



(Dùng ngôn ngữ lập trình C++)

MỤC LỤC

CHƯƠNG 0 : NHẬP MÔN LẬP TRÌNH	5
§1 : THỜI GIAN THỰC HIỆN CHƯƠNG TRÌNH	5
① ĐỘ PHỨC TẠP CỦA THUẬT TOÁN THEO THỜI GIAN	5
② HỆ SỐ KHÔNG ĐỔI.....	6
§2 : XỬ LÝ DỮ LIỆU	6
① ĐỌC – XUẤT	6
② DEBUG TAY.....	7
CHƯƠNG 1 : ĐẠI SỐ.....	8
§1 : SỐ NGUYÊN TỐ	8
① KIỂM TRA TÍNH NGUYÊN TỐ.....	8
② Sàng ERATOSTHENES.....	9
③ PHÂN TÍCH THỪA SỐ NGUYÊN TỐ, ĐẾM SỐ ƯỚC CỦA SỐ N	9
§2 : ƯỚC CHUNG LỚN NHẤT, BỘI CHUNG NHỎ NHẤT.....	10
§3 : ĐỒNG DƯ THỨC, LŨY THỪA.....	11
① ĐỒNG DƯ THỨC.....	11
② NHỊ PHÂN LŨY THỪA.....	11
③ LOGARITHM	12
§4 : XỬ LÝ SỐ NGUYÊN LỚN.....	12
① THAO TÁC SO SÁNH VÀ CHUẨN HOÁ	12
② THAO TÁC CỘNG, TRỪ, NHÂN.....	12
§5 : HỆ CƠ SỐ.....	14
① HỆ CƠ SỐ.....	14
② CHUYỂN ĐỔI QUA LẠI TỪ HỆ CƠ SỐ A VÀ HỆ CƠ SỐ B	15
③ BẢNG MÃ ASCII	15
④ BIT.....	16
CHƯƠNG 2 : DÃY SỐ	18
§1 : DÃY SỐ CÓ QUY LUẬT	18
① BIỂU DIỄN DÃY SỐ CÓ QUY LUẬT	18
② DÃY CẤP SỐ CỘNG VÀ CẤP SỐ NHÂN	18
§2 : TÌM KIẾM TUYẾN TÍNH.....	18
① INDEX – ITERATOR.....	18
② FOR – FOR-EACH	19
§3 : CÁC HÀM CÓ SẴN TRONG C++	19
① CÁC HÀM ĐƠN GIẢN	19
② THUẬT TOÁN SẮP XẾP.....	20
§4 : CẤU TRÚC DỮ LIỆU TUYẾN TÍNH.....	21
① MẢNG ĐỘNG (VECTOR)	21
② NGĂN XẾP (STACK).....	21

③ HÀNG ĐỢI (QUEUE).....	23
④ MAP VÀ SET.....	24
§5 : THUẬT TOÁN TÌM KIẾM NHỊ PHÂN.....	24
① THUẬT TOÁN TÌM KIẾM NHỊ PHÂN CƠ BẢN	25
② MỞ RỘNG ỨNG DỤNG THUẬT TOÁN TÌM KIẾM NHỊ PHÂN	25
§6 : DÃY CON LIÊN TIẾP (ĐOẠN CON).....	27
① CỬA SỐ TRƯỢT	27
② THUẬT TOÁN KADANE	28
§7 : MẢNG CỘNG DỒN	29
§8 : KỸ THUẬT BẮM DỮ LIỆU.....	29
① BẢNG BẮM (HASH TABLE).....	29
② ỨNG DỤNG KỸ THUẬT BẮM TRONG XỬ LÝ XẤU	30
CHƯƠNG 3 : ĐẠI SỐ TỔ HỢP.....	31
§1 : LÝ THUYẾT ĐẠI SỐ TỔ HỢP	31
① QUY TẮC CỘNG, QUY TẮC NHÂN.....	31
② HOÁN VỊ, CHỈNH HỢP, TỔ HỢP	31
§2 : ĐỆ QUY – QUAY LUI – NHÁNH – CẬN	32
① ĐỆ QUY – QUAY LUI.....	32
② NHÁNH – CẬN	34
§3 : QUY HOẠCH ĐỘNG	34
① TỔNG QUAN.....	34
② QUY HOẠCH ĐỘNG THÔNG DỤNG.....	35
③ QUY HOẠCH ĐỘNG CẤU HÌNH	39
④ QUY HOẠCH ĐỘNG CHỮ SỐ (DIGIT).....	39
⑤ QUY HOẠCH ĐỘNG TRẠNG THÁI (BITMASK):.....	40
§4 : MEET IN THE MIDDLE	47
§5 : THUẬT TOÁN THAM LAM	48
CHƯƠNG 4 : HÌNH HỌC	50
§1 : MA TRẬN	50
① BIỂU DIỄN MA TRẬN.....	50
② DI CHUYỂN CON TRỎ TRONG MA TRẬN.....	50
§2 : THUẬT TOÁN ĐƯỜNG QUÉT.....	51
① CÁC KHÁI NIỆM CƠ BẢN TRÊN HỆ TRỤC TỌA ĐỘ DESCARTES	51
② THUẬT TOÁN ĐƯỜNG QUÉT	51
CHƯƠNG 5 : LÝ THUYẾT ĐỒ THỊ.....	53
§1 : ĐẠI CƯƠNG.....	53
① KHÁI NIỆM.....	53
② BIỂU DIỄN ĐỒ THỊ.....	54
§2 : CÁC THUẬT TOÁN TÌM KIẾM TRÊN ĐỒ THỊ.....	55

❶ DFS – THUẬT TOÁN TÌM KIẾM THEO CHIỀU SÂU.....	55
❷ BFS – THUẬT TOÁN TÌM KIẾM THEO CHIỀU RỘNG.....	57
§3 : CHU TRÌNH.....	57
❶ CHU TRÌNH.....	58
❷ KIỂM TRA CHU TRÌNH.....	58
§4 : THỨ TỰ TOPO, SẮP XẾP TOPO.....	59
❶ THỨ TỰ TOPO.....	59
❷ THUẬT TOÁN KAHN.....	59
§5 : VÙNG LIÊN THÔNG.....	61
❶ ĐỊNH NGHĨA.....	61
❷ TÌM VÙNG LIÊN THÔNG TRONG ĐỒ THỊ VÔ HƯỚNG BẰNG THUẬT TOÁN BFS/DFS.....	62
❸ TÌM VÙNG LIÊN THÔNG TRONG ĐỒ THỊ CÓ HƯỚNG.....	62
❹ ĐỈNH KHỚP, CẠNH CẦU.....	64
§6 : ĐƯỜNG ĐI NGẮN NHẤT.....	66
❶ THUẬT TOÁN LOANG (BFS).....	66
❷ THUẬT TOÁN DIJKSTRA.....	67
§7 : CÂY KHUNG CỰC TIỂU.....	68
❶ CẤU TRÚC DỮ LIỆU CÁC TẬP HỢP RỜI NHAU.....	68
❷ KHÁI NIỆM CÂY KHUNG CỰC TIỂU.....	71
❸ TÌM CÂY KHUNG CỰC TIỂU.....	71

CHƯƠNG 0 : NHẬP MÔN LẬP TRÌNH

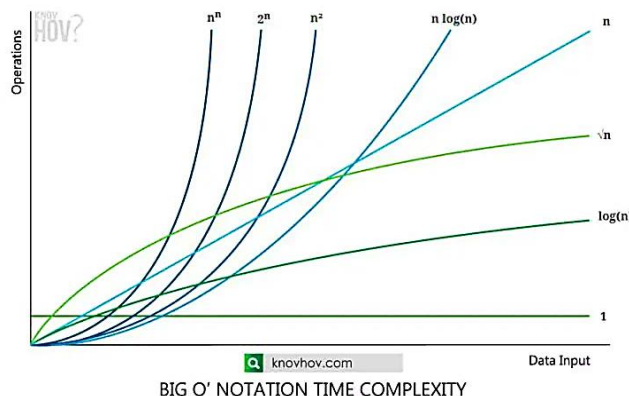
§1 : THỜI GIAN THỰC HIỆN CHƯƠNG TRÌNH

- **Loop, repeat:** Vòng lặp, sự lặp lại.
- **Time complexity:** Độ phức tạp của thuật toán theo thời gian.
- **Constant factor:** Hệ số không đổi.

① ĐỘ PHỨC TẠP CỦA THUẬT TOÁN THEO THỜI GIAN

- **Ý nghĩa:**
 - Độ phức tạp của thuật toán giúp ta phỏng đoán tốc độ của thuật toán (nằm trong chương trình), qua đó đưa ra những tối ưu phù hợp nhằm cải tiến sao cho chương trình chạy càng nhanh càng tốt.
 - Độ phức tạp của thuật toán tương đối thể hiện số lượng phép tính của thuật toán so với khối lượng của dữ liệu vào. Như vậy, độ phức tạp của thuật toán phụ thuộc vào khối lượng của dữ liệu vào.
 - Độ phức tạp càng nhỏ (nhỏ nhất là 1) thì thuật toán càng nhanh.
- **Kí hiệu:** $O(f(n))$ và $f(n)$ là hàm theo n thể hiện khối lượng của dữ liệu vào. Nếu $f(n) = 1$ thì $O(f(n)) = O(1)$ và được gọi là độ phức tạp hằng số.
- **Cách xác định độ phức tạp:** Dựa trên số lần lặp.

Thao tác	Độ phức tạp	Điều kiện của n để không TLE (tương đối)
$a = b;$ $a = \min(x, y);$ $\Rightarrow$ Không lặp lần nào (hoặc cũng có thể hiểu là lặp 1 lần).	$\approx O(1)$	Bất kì n (do n là mấy thì $O(1)$ vẫn là 1).
Lặp từ 1 đến n . $\Rightarrow$ Lặp n lần.	$\approx O(n)$	$n \leq 10^6$.
Lặp từ 1 đến n : Lặp từ 1 đến m . $\Rightarrow$ Lặp nm lần.	$\approx O(nm) \approx O(n^2)$	$n \leq 10^3$.
Lặp i từ 1 đến n : Lặp j từ $i + 1$ đến n . $\Rightarrow$ Coi như lặp n^2 lần.	$\approx O(n^2)$	$n \leq 10^3$.
Lặp $\frac{n}{2}$ lần, rồi lặp $\frac{n}{4}$ lần, rồi lặp $\frac{n}{6}$ lần,... $\Rightarrow$ Lặp $\log_2 n$ lần.	$\approx O(\log_2 n) \approx O(\log n)$	$n \leq 10^{12}$.



Đồ thị cho thấy tốc độ của các loại phức tạp (trục hoành biểu thị độ tăng INPUT, trục tung biểu thị thời gian tương ứng) (đường cong càng ngang là càng nhanh). Dữ liệu càng lớn, thời gian càng có sự khác biệt. Có thể thấy, cùng một dữ liệu nhưng $O(1)$, $O(\log n)$ có tốc độ rất nhanh.

② HỆ SỐ KHÔNG ĐỔI

Khái niệm:

- Hệ số không đổi k là hệ số được gắn vào $f(n)$, trở thành $O(k \cdot f(n))$, thể hiện số lượng phép tính của thuật toán.
- Như vậy, k không phụ thuộc vào khối lượng dữ liệu vào mà phụ thuộc vào số lượng phép tính.

VD1: Có dòng code:

```
a = b; ++ a;
```

- Do không lặp nên độ phức tạp là $O(1)$. Tuy nhiên, lại có đến 2 phép tính chứ không phải 1, nên $k = 2$, tức $O(2)$.

VD2: Trong vòng lặp n lần có dòng code là:

```
a = b; ++ a;
```

- Do lặp n lần nên độ phức tạp là $O(n)$. Tuy nhiên, lại có đến 2 phép tính ở mỗi lần lặp, nên $k = 2$, tức $O(2n)$. Dù n có tăng giảm ra sao đi nữa (độ phức tạp thay đổi), nhưng k thì vẫn là 2.

Kết luận:

- Dù trong 1 giây máy tính có thực hiện đến 10^9 , 10^{12} ,... phép tính, có khi còn hơn, nhưng vì hệ số không đổi, nên điều kiện $n \leq 10^6$ (như bảng trên) là một điều kiện khá tốt để đảm bảo chương trình không TLE.
- Bên cạnh việc cố gắng giảm số lần lặp, cũng cần tinh chỉnh code sao cho ngắn gọn, ít phép tính nhất có thể để hệ số không đổi thấp.

§2 : XỬ LÝ DỮ LIỆU

① ĐỌC – XUẤT

Fast I/O: Đoạn code tăng tốc độ đọc – xuất (chỉ thực sự có tác dụng khi input và output lớn):

```
1. ios_base::sync_with_stdio(0);
2. cin.tie(0); cout.tie(0);
```

Đọc chuỗi có dấu cách:

- Do đối tượng `>>` không thể đọc chuỗi có dấu cách như chuỗi `s = "Baor Ming 2k5"` nên cần dùng hàm `getline`:
`getline(cin, s);`
- Nhưng `getline` sẽ đọc cả dấu enter, để xóa dấu này, ta dùng `cin.ignore();` liền sau `getline` nếu có nhu cầu đọc thêm 1 chuỗi khác.

Đọc – xuất với file FILE.INP và FILE.OUT qua đối tượng `cin` và `cout` bằng hàm `freopen` (không cần đóng file):

- Đọc: `freopen ("FILE.INP", "r", stdin);`
- Xuất: `freopen ("FILE.OUT", "w", stdout);`

Đọc vào `x` cho đến khi không thể đọc nữa (không cho biết số lượng `x`):

```
while (cin >> x) {};
```

Xuất ra `x` chữ số thập phân của số `n`:

```
cout << fixed << setprecision(x) << n;
```

Vài hàm làm tròn khác:

<code>round(n)</code>	Làm tròn đến chữ số thập phân đầu tiên của <code>n</code> .
<code>ceil(n)</code>	Trả về số nguyên nhỏ nhất lớn hơn hoặc bằng <code>n</code> .
<code>floor(n)</code>	Trả về số nguyên lớn nhất bé hơn hoặc bằng <code>n</code> (bỏ đi phần thập phân của <code>n</code>).

Lưu ý:

- Các hàm trả về số thực khi `n` khá lớn. Trong trường hợp đó, ta ép kiểu về nguyên, chẳng hạn `int(floor(n))`.
- Nếu `ceil(n) = floor(n)` thì `n` là 1 số nguyên.

② DEBUG TAY

Ý nghĩa: Hướng đến việc in ra màn hình giá trị các biến cần xem (`cout <<`) mà không dùng chức năng debug có sẵn trong trình biên dịch.

Các cú pháp hỗ trợ:

- `system("pause");` : Tạm dừng thông dịch chương trình cho đến khi nhận dấu enter.
- `cout << flush;` : Xóa bộ nhớ đệm sau mỗi lần thực hiện `system("pause")`.

CHƯƠNG 1 : ĐẠI SỐ

§1 : SỐ NGUYÊN TỐ

- **Prime number:** Số nguyên tố.
- **Prime factor:** Thừa số nguyên tố.
- **Sieve of Eratosthenes:** Sàng Eratosthenes.

Định nghĩa: Số nguyên tố là số chỉ chia hết cho 1 và chính nó.

Nhận xét về số nguyên tố: Gồm 3 tính chất quan trọng sau:

1. $n = 1$ thì n không phải số nguyên tố.
2. n là số chia hết cho 2 hoặc chia hết cho 3 và $n > 3$ thì n không phải số nguyên tố.
3. n là số nguyên tố > 3 thì n có dạng $6k \pm 1$ ($k \in \mathbb{N}^*$). **Lưu ý:** Điều ngược lại là sai.
4. n không chia hết cho số nào thì n là số nguyên tố (suy ra từ thuật toán Eratosthenes).

① KIỂM TRA TÍNH NGUYÊN TỐ

Ý tưởng 1: Do số nguyên tố là những số không chia hết cho bất kì số nào. Mà ước lớn nhất (khác n) của n là $\frac{n}{2}$. Nên ta chỉ cần kiểm tra xem n có chia hết cho số nào từ 2 đến $\frac{n}{2}$ không. Nếu có, n không phải là số nguyên tố.

Độ phức tạp: $O\left(\frac{n}{2}\right)$.

Ý tưởng 2:

- Giả sử $n = ab$.
- Điều đó có nghĩa a và b là ước của n . Nếu n chia hết cho a thì chắc chắn chia hết cho b .
- Mà a, b cùng $\leq \sqrt{n}$.
- Trong VD dưới, $n = 20$ và $\sqrt{20} \approx 4$. Ở $a = 1; 2; 4$ thì n chắc chắn chia hết cho $b = 20; 10; 5$. Nên, ta chỉ cần kiểm tra xem n có chia hết cho số nào từ 2 đến $\sqrt{n}$ không chứ không cần kiểm tra đến $\frac{n}{2}$.

$\sqrt{n}$					$n/2$					n									
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Các nghiệm của n "đối xứng" nhau qua $\sqrt{n}$ nên chỉ cần duyệt từ 2 đến $\sqrt{n}$.										n không có nghiệm nào lớn hơn $\frac{n}{2}$ (trừ n) nên không cần duyệt trong đoạn này.									

- Tóm lại, nếu muốn kiểm tra n có phải số nguyên tố không, chỉ cần kiểm tra n có chia hết cho số nào từ 2 đến $\sqrt{n}$ không.

Độ phức tạp: $O(\sqrt{n})$.

Ý tưởng 3:

- Kiểm tra sàng lọc theo tính chất 1 và 2.
- Dựa trên tính chất 2 và 3: Lần lượt kiểm tra những số i có dạng $6k - 1 = 5$ đến $\sqrt{n}$, nếu n không chia hết cho bất kì i nào hoặc không chia hết cho bất kì $i + 2$ nào thì n là số nguyên tố.

Độ phức tạp: $O\left(\frac{\sqrt{n}}{6}\right)$.

```
1. //Hàm trả về true nếu n là số nguyên tố
2. bool isPrime(int n){
```



```

3.     if(n == 1 || n > 3 && (n % 3 == 0 || n % 2 == 0))
4.         return false;
5.     for(int i = 5; i*i <= n; i+=6)
6.         if(n % i == 0 || n % (i+2) == 0)
7.             return false;
8.     return true;
9. }

```

② SÀNG ERATOSTHENES

Ứng dụng: Tìm hàng loạt những số nguyên tố trong đoạn đã cho.

a. Sàng Eratosthenes trong đoạn $[0; n]$

Ý tưởng:

- Coi mọi số trong dãy đều là số nguyên tố (trừ số 0 và số 1), đánh dấu mọi số trong dãy.
- Lần lượt duyệt i từ 2 đến $\sqrt{n}$. Nếu i được đánh dấu thì i là số nguyên tố, bỏ đánh dấu mọi bội số của i còn lại trong dãy.

Lưu ý tối ưu: Để loại bỏ các bội số của i , thay vì duyệt từ $2i$, có thể duyệt từ i^2 đến n để tối ưu thời gian.

Độ phức tạp: $O(n \log n)$.

```

1. bool prime[n+1];
2. void sieve(){
3.     fill(&prime[2], &prime[n], true);
4.     for(int i = 2; i*i <= n; ++i)
5.         if(prime[i]) //Nếu i là số nguyên tố thì bỏ đánh dấu mọi bội số của i trong dãy
6.             for(int j = i*i; j <= n; j+=i)
7.                 prime[j] = false;
8. }

```

b. Sàng Eratosthenes trong đoạn $[l, r] \neq [0, r]$

Vấn đề:

- Bình thường, ta tạo ra mảng prime để đánh dấu xem đâu là số nguyên tố ($\text{prime}[i] = \text{true}$ thì i là số nguyên tố). Nhưng ở bài này, đề không yêu cầu ta xét tính nguyên tố của các số $l-1$, $l-2$, mà chỉ yêu cầu ta xét các số trong đoạn $[l, r] \Rightarrow$ Tạo mảng prime có $r-l+1$ phần tử.
- Nhiệm vụ của ta là tìm cách đánh dấu vào mảng prime mới này. Không còn cách nào khác ngoài cách duyệt i từ 2 đến $\sqrt{r}$. Biết bội của số i nhỏ nhất lớn hơn hoặc bằng l tính bằng công thức: $\frac{l+i-1}{i}i$ (phép chia nguyên), ta sẽ dùng công thức này để đánh dấu vào mảng prime.

Các bước:

- Gọi $\text{prime}[r-l+1]$ là mảng chứa các số nguyên tố trong đoạn $[l, r]$. Ban đầu, giả sử tất cả các số trong $[l, r]$ đều là số nguyên tố.
- Duyệt i từ 2 đến $\sqrt{r}$. Ở mỗi i , cho j chạy từ $\max(i^2, \frac{l+i-1}{i}i)$ đến r ($j += i$ cho mỗi bước) để loại bỏ bội số của i trong $[l, r]$ khỏi prime (j là bội số của i , $\text{prime}[j] = \text{false}$).

Lưu ý: Nếu coi $\text{prime}[i] = \text{true} \Rightarrow i$ là số nguyên tố thì đối với sàng đoạn, nếu i là số nguyên tố thì $\text{prime}[i-l] = \text{true}$.

Độ phức tạp: $O(r)$.

```

1. bool prime[r-l+1];
2. void sieve(){
3.     fill(&prime[0], &prime[r-l+1], true);
4.     for(int i = 2; i*i <= r; ++i)
5.         for(int j = max(i*i, (l+i-1)/i*i); j <= r; j+=i)
6.             prime[j-l] = false;
7. }

```

③ PHÂN TÍCH THỪA SỐ NGUYÊN TỐ, ĐẾM SỐ ƯỚC CỦA SỐ N

a. Phân tích thừa số nguyên tố

Định nghĩa:

- Phân tích thừa số nguyên tố của số n là đưa n về dạng tích của các số nguyên tố. VD $50 = 2 * 5^2$.
- Mỗi số nguyên dương đều có thể phân tích thành thừa số nguyên tố.

Ý tưởng:

- Tạo một biến tên p dùng để lấy n chia p . Ban đầu, $p = 2$.
- Nếu n chia hết cho p :
 - p là thừa số nguyên tố.
 - Giảm giá trị của n : $n /= p$.
- Nếu n không chia hết cho p : Tăng p lên 1 đơn vị.
- Lặp đi lặp lại 2 bước trên cho đến khi $n = 1$ (n đã hoàn thành chia cho chính nó).

Lưu ý: Có thể tối ưu bằng cách lấy p là những số nguyên tố, nên áp dụng với bài có dữ liệu khủng.

Độ phức tạp: $O(\log n)$.

```
1. //Thuật toán xuất ra danh sách các thừa số nguyên tố của n
2. void fact(int n){
3.     int p = 2;
4.     while(n != 1)
5.         if(n % p == 0){
6.             cout << p << ' ';
7.             n /= p;
8.         } else
9.             ++ p;
10. }
```

b. Danh sách ước của n

Có hai cách tìm ước của n :

- Cách 1:** Đã biết nếu $n = ij$ thì i và j là ước của n , mà $i \leq \sqrt{n}$ nên ta duyệt i từ 1 đến $\sqrt{n}$ để tìm ước.
- Cách 2:** Nếu n có thể phân tích thành thừa số nguyên tố: $n = a_0^{k_0} * a_2^{k_2} * a_3^{k_3} \dots$ Thì n có $(k_0 + 1)(k_2 + 1)(k_3 + 1) \dots$ ước. VD $50 = 2^1 * 5^2 \Rightarrow 50$ có $(1 + 1)(2 + 1)$ ước.

```
1. //Hàm xuất ra các ước của n theo cách 1
2. void divCnt(int n){
3.     for(int i = 1; i*i < n; ++i)
4.         if(n % i == 0)
5.             cout << i << ' ' << n/i << ' ';
6.     //Nếu làm tròn lên của căn n và làm tròn xuống của căn n bằng nhau thì căn n là 1 số nguyên và là 1 ước
   c của n
7.     if(ceil(sqrt(n)) == floor(sqrt(n)))
8.         cout << sqrt(n);
9. }
```

§2 : ƯỚC CHUNG LỚN NHẤT, BỘI CHUNG NHỎ NHẤT

- Greatest common divisor (GCD):** Ước chung lớn nhất.
- Lowest common multiple (LCM):** Bội chung nhỏ nhất.

Khái niệm:

- Ước chung lớn nhất của a và b , kí hiệu $GCD(a, b)$, là số lớn nhất mà cả a và b đều chia hết. VD: $GCD(4, 6) = 2$.
- Bội số nhỏ nhất của a và b , kí hiệu $LCM(a, b)$, là số nhỏ nhất mà chia hết cho cả a và b . VD: $LCM(4, 6) = 12$.

Tìm $GCD(a, b)$ bằng thuật toán Euclid:

- GCD có 2 tính chất sau:
 - $GCD(a, 0) = a$.

- $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$.

- Dựa vào đó viết hàm tính $\text{GCD}(a, b)$ như sau:

```
1. int gcd(int a, int b){
2.     if (b == 0)
3.         return a;
4.     else
5.         return gcd(b, a % b);
6. }
```

- Độ phức tạp: $O(\log \max(a, b))$.

Thuật toán Euclid còn được viết như sau (không nhanh bằng cách trên):

- Nếu $a > b$ thì $a -= b$.
- Nếu $a < b$ thì $b -= a$.
- Lặp đi lặp lại 2 bước trên cho đến khi $a = b$. Lúc đó, $a = b = \text{GCD}(a, b)$.
- Độ phức tạp: $O(\log \max(a, b))$.

Tìm $\text{LCM}(a, b)$ thông qua $\text{GCD}(a, b)$: $\text{LCM}(a, b) = \frac{a}{\text{GCD}(a, b)} b$.

§3 : ĐỒNG DƯ THỨC, LŨY THỪA

- **Modulo (mod)**: Phép chia lấy dư.
- **Exponentiation (exp)**: Phép lũy thừa.
- **Power of k**: Lũy thừa bậc k.

① ĐỒNG DƯ THỨC

a. Modulo thông thường

Phép modulo thông thường:

- Đồng dư thức (phép modulo) là phép chia lấy dư của a cho modulo m, kí hiệu $a \% m$. VD: $4 \% 2 = 0$.
- Tính chất:
 - Kết quả của phép modulo (số dư) luôn bé hơn số chia.
 - Modulo có tính phân phối với phép cộng – trừ và nhân:
 - $(a \pm b) \% m = (a \% m \pm b \% m) \% m$.
 - $ab \% m = (a \% m)(b \% m) \% m$.
 - $(a \% b) \% m = a \% m + b \% m$.

b. Nghịch đảo modulo

- Có $\frac{b}{a} \% m = b \cdot a^{-1} \% m = (b \% m)(a^{-1} \% m) \% m$ (theo tính chất phân phối). Nhưng không thể tính $a^{-1} \% m$ như thông thường mà cần phải tính theo cách đặc biệt là nghịch đảo modulo.
- Nghịch đảo modulo có dạng $a^{-1} \% m$ và không phải lúc nào cũng tồn tại kết quả của nghịch đảo modulo.
- **Cách tính nghịch đảo modulo**: Nếu $k \cdot a \% m = 0$ ($0 \leq k < m$) thì $a^{-1} \% m = k$.

② NHỊ PHÂN LŨY THỪA

Tính chất của lũy thừa:

- $a^0 = 1$.
- $a^k = (a^{\frac{k}{2}})^2$ (k chẵn).
- $a^k = a(a^{\frac{k-1}{2}})^2$ (k lẻ).

Tính lũy thừa dựa theo tính chất (đệ quy):

- Có hàm tính lũy thừa $\text{pow}(a, k)$:
 - Nếu $k = 0$, trả về 1.
 - Nếu k chẵn: $\text{pow}(a, k) = \text{pow}(a, \frac{k}{2})^2$.

- Nếu k lẻ: $\text{pow}(a, k) = a \cdot \text{pow}(a, \frac{k}{2})^2$ (phép chia nguyên).

Độ phức tạp: $O(\log k)$.

3 LOGARITHM

- Nếu $a^k = b \Rightarrow \log_a b = k$, đọc là logarithm cơ số a của b ($\log a$ của b).
- Logarithm cơ số e của b còn gọi là logarithm tự nhiên, với hằng số e xấp xỉ 2,70828. Kí hiệu $\log_e b$ hoặc $\ln b$.
- Trong C++ chỉ hỗ trợ sẵn hàm tính logarithm tự nhiên của x : **log(x)**.
- Công thức tính logarithm cơ số a bất kỳ của b : $\log_a b = \frac{\log(b)}{\log(a)}$.

§4 : XỬ LÝ SỐ NGUYÊN LỚN

- **Bignum**: Số nguyên lớn.

Các kiểu dữ liệu thường dùng để chứa số nguyên:

Kiểu dữ liệu / cấu trúc dữ liệu	Giá trị tối đa		Tương đương
	Cú pháp	Giá trị	
int	INT_MAX	Khoảng 10^9 .	9 chữ số.
long long	LLONG_MAX	Khoảng 10^{18} .	18 chữ số.
string (chứa toàn chữ số)	Gần như vô hạn (dựa vào bộ nhớ máy tính)		
vector (mỗi phần tử chứa 1 chữ số)			

Kết luận: Nếu đề bài dẫn đến việc thao tác với số nguyên rất lớn (hơn 18 chữ số), có thể dùng kiểu string hoặc vector để thay thế int và long long.

Một số thao tác với string (chứa toàn chữ số):

1 THAO TÁC SO SÁNH VÀ CHUẨN HOÁ

a. So sánh

- So sánh trực tiếp 2 xâu bằng các toán tử so sánh.

b. Chuẩn hoá

Chuẩn hoá là làm cho số nguyên hợp lệ. Có 2 loại:

- Xoá các chữ số 0 ở đầu số (delZero).
- Thêm số 0 (addZero) ở đầu chữ số để thuận tiện tính toán. VD có phép toán $124 + 1$, để thuận tiện, ta nên biến đổi thành "124" + "001" để chỉ cần duyệt và cộng tuần tự từng chữ số.

```

1. //thêm số 0 vào trước a cho đến khi a có độ dài là len
2. void addZero(string &a, int len){
3.     while(a.size() < len)
4.         a = '0' + a;
5. }
6.
7. void delZero(string &a){
8.     while(a.size() > 1 && a.front() == '0')
9.         a.erase(a.begin());
10. }
```

2 THAO TÁC CỘNG, TRỪ, NHÂN

a. Phép cộng

Lưu ý: Phải addZero đến khi 2 toán hạng có độ dài bằng nhau.

VD có phép cộng:

Biến nhớ (rem)	1	1	
Toán hạng a		9	9
Toán hạng b		2	8
Cộng từng chữ số (tmp)		12	17
Kết quả (ans)	1	2	7

- Thực hiện phép tính $9 + 8 = 17$ không hợp lệ (do chỉ nhận 1 chữ số), nên ta chỉ lấy số 7 ($\text{tmp} \% 10$) để gắn vào kết quả và nhớ 1 ($\text{tmp} / 10$) đến cộng cho phép tính tiếp theo. Cuối cùng, cộng dồn rem vào ans rồi xóa các chữ số 0 dư thừa.

```
1. string add(string a, string b){
2.     int len = max(a.size(), b.size());
3.     addZero(a, len);
4.     addZero(b, len);
5.     string ans = "";
6.     int rem = 0;
7.     for(int i = len-1; i >= 0; --i){
8.         int tmp = (a[i]-'0') + (b[i]-'0') + rem;
9.         rem = tmp / 10;
10.        tmp %= 10;
11.        ans = (char)(tmp + '0') + ans;
12.    }
13.    ans = (char)(rem + '0') + ans;
14.    delZero();
15.    return ans;
16. }
```

b. Phép trừ

- Lưu ý:** Chỉ đúng khi $a \geq b$.

VD có phép trừ:

Toán hạng a	1	3	0
Toán hạng b	0	2	8
Biến nhớ (rem)		1	
Trừ từng chữ số (tmp)		0	-2
Kết quả (ans)	1	0	2

- Ở hàng phải cùng, ta thấy $0 - 8 = -2$ không hợp lệ, nên ta mượn thêm 1 để tạo thành $10 - 8 = 2$. Hay nếu $\text{tmp} < 0$, ta cộng thêm 10 rồi gắn số đó vào kết quả, đồng thời nhớ số 1 để tạo thành phép tính $3 - 2 - 1 = 0$. Nếu $\text{tmp} \geq 0$, trực tiếp đem tmp gắn vào kết quả.

```
1. string sub (string a, string b){
2.     int len = max(a.size(), b.size());
3.     addZero(a, len);
4.     addZero(b, len);
5.     string ans = "";
6.     int rem = 0;
7.     for(int i = len-1; i >= 0; --i){
8.         int tmp = (a[i]-'0') - (b[i]-'0') - rem;
9.         if(tmp < 0){
10.            tmp += 10;
11.            rem = 1;
12.        } else
13.            rem = 0;
14.        ans = (char)(tmp + '0') + ans;
15.    }
16.    delZero(ans);
17.    return ans;
}
```

c. Phép nhânVD có phép nhân: 12×28 .• Nhân lần 1: 12×8 :

Toán hạng a	1	2
Toán hạng b	2	8
Kết quả 8×2		16
	1	6
Kết quả 8×1	8	
Cộng dồn 2 kết quả lại	9	6

• Nhân lần 2: 12×2

Toán hạng a	1	2
Toán hạng b	2	8
Kết quả nhân lần 1	9	6
Kết quả 12×2	2	4
		13
Cộng dồn 2 kết quả lại	1	3
	3	
Kết quả cuối cùng	3	3

- Giá sử có $ans = a * b$, vậy $|ans| \leq |a| + |b|$ với $|ans|$ là số lượng chữ số của ans . Tạo vector `<int> tmp (|a|+|b|)` là vector lưu các chữ số của ans .
- Ta thực hiện duyệt kép ngược từ phải sang (duyệt b rồi duyệt a), lấy tuần tự từng chữ số của a và b nhân lại và gán cho $tmp[i+j]$. Lưu ý chỉ giữ lại ở $tmp[i+j]$ 1 chữ số, chữ số dư thừa sẽ cộng dồn sang $tmp[i+j-1]$.
- Lưu ý: Ở lần nhân đầu, ta thấy kết quả 2×8 được đặt vào $tmp[2]$, tuy nhiên tmp có tối đa đến vị trí $tmp[4]$ nên ta dịch vị trí thêm 1 về bên phải ở mỗi lần nhân ($tmp[i+j] \rightarrow tmp[i+j+1]$).
- Sau khi cộng và đưa vào vector tmp , ta chuyển sang chuỗi string.

```

1. string mul (string a, string b){
2.     vector <int> tmp(a.size() + b.size());
3.     for(int j = b.size()-1; j >= 0; --j)
4.         for(int i = a.size()-1; i >= 0; --i){
5.             tmp[i+j+1] += (a[i]-'0') * (b[j]-'0');
6.             tmp[i+j-1+1] += tmp[i+j+1] / 10;
7.             tmp[i+j+1] %= 10;
8.         }
9.     string ans = "";
10.    for(int i = 0; i < tmp.size(); ++i)
11.        ans += (char)(tmp[i] + '0');
12.    delZero(ans);
13.    return ans;
14. }
```

Lưu ý: Ở các thuật toán trên, có thể đảo ngược xâu lại trước khi tính toán để tối ưu thời gian.

§5 : HỆ CƠ SỐ

- Binary (bin):** Hệ nhị phân.
- Decimal (dec):** Hệ thập phân.

1 HỆ CƠ SỐ

- Hệ cơ số x thì dùng x kí tự để biểu diễn các giá trị số (những kí tự này có thể là a, b, 0, 1,...). Tức mỗi số là một dãy các kí tự.
- Các kí tự trong dãy được đánh số thứ tự từ phải sang, bắt đầu từ vị trí 0. VD trong dãy kí tự **abcdef** thì kí tự thứ 0 là f, thứ 1 là e.
- Hệ cơ số thường gặp là hệ nhị phân: Dùng 2 kí tự 0 và 1 để biểu diễn. Mỗi giá trị 0 hoặc 1 được gọi là một bit.

② CHUYỂN ĐỔI QUA LẠI TỪ HỆ CƠ SỐ A VÀ HỆ CƠ SỐ B

Vấn đề: Có số n ở hệ cơ số A , kí hiệu n_A , yêu cầu chuyển sang hệ cơ số B , hay n_B .

a. Nếu $A < B$

Công thức tổng quát:

$$n_B = \sum_{i=0}^{|n_A|-1} A^i n_{A[i]}$$

(với $|n_A|$ là số lượng kí tự của n_A).

Công thức tính theo thứ tự từ trái sang:

$$n_B = \sum_{i=0}^{|n_A|-1} A^i n_{A[|n_A|-i-1]}$$

VD: $1011_2 = ?_{10}$

i =	3	2	1	0
$N_2 =$	1	0	1	1
$N_{10} =$	$2^3 \cdot 1 +$	$2^2 \cdot 0 +$	$2^1 \cdot 1 +$	$2^0 \cdot 1 + = 11$

b. Nếu $A > B$

Lần lượt lấy n_A và các kết quả chia nguyên của nó chia cho A . n_B sẽ là phép cộng (theo chuỗi) của số dư của từng phép tính theo chiều ngược lại.

VD: $11_{10} = ?_2$

11	2								
1	5	2							
	1	2	2						
		0	1	2					
			1	0					
				1	0				

Vậy $11_{10} = 1011_2$.

③ BẢNG MÃ ASCII

Bảng mã ASCII là bảng mã hóa các ký tự thông dụng thành các dãy bit. Mỗi dãy bit có thể được chuyển sang số thập phân. Trong C++, chỉ cần viết 'a' là compiler sẽ tự hiểu là 97 (mã ASCII của 'a').

BẢNG MÃ ASCII
Các ký tự và mã thập phân tương ứng

32	33 !	34 "	35 #	36 \$	37 %	38 &	39 '	40 (	41)
42 *	43 +	44 ,	45 -	46 .	47 /	48 0	49 1	50 2	51 3
52 4	53 5	54 6	55 7	56 8	57 9	58 :	59 ;	60 <	61 =
62 >	63 ?	64 @	65 A	66 B	67 C	68 D	69 E	70 F	71 G
72 H	73 I	74 J	75 K	76 L	77 M	78 N	79 O	80 P	81 Q
82 R	83 S	84 T	85 U	86 V	87 W	88 X	89 Y	90 Z	91 [
92 \	93]	94 ^	95 _	96 `	97 a	98 b	99 c	100 d	101 e
102 f	103 g	104 h	105 i	106 j	107 k	108 l	109 m	110 n	111 o
112 p	113 q	114 r	115 s	116 t	117 u	118 v	119 w	120 x	121 y
122 z	123 {	124	125 }	126 ~					

- Mã của chữ cái thường lớn hơn mã chữ cái hoa là ('a' - 'A') $\Rightarrow$ Lấy mã chữ cái hoa + ('a' - 'A') sẽ ra chữ cái thường và ngược lại.
- Mã của chữ số lớn hơn giá trị của chữ số đó '0' $\Rightarrow$ Lấy kí tự chữ số - '0' sẽ ra giá trị của chữ số và ngược lại.

4 BIT

a. Dãy bit

- **Khái niệm:** Dãy bit là dãy các bit 1 hoặc 0 (hệ cơ số 2).
- **Số bit trong dãy bit:**
 - Có các kiểu dữ liệu:
 - Kiểu dữ liệu có 8 bit (char, bool).
 - Kiểu dữ liệu có 32 bit (int).
 - Kiểu dữ liệu có 64 bit (long long).
 - Các bit 0 cuối dãy bit là để lấp đầy dãy bit theo kiểu dữ liệu, không tham gia tạo giá trị thập phân, trừ biểu diễn nhị phân của số thập phân 0. **VD:** giá trị TRUE (bool) tương ứng 00000001.
 - Cho số thập phân n, biểu diễn nhị phân của n có $|n_2|$ bit với:

$$|n_2| = \text{ceil}(\log_2 n) = \text{ceil}\left(\frac{\ln n}{\ln 2}\right)$$

- **Sử dụng dãy bit:** Trong C++, các dữ liệu được thể hiện dưới dạng thập phân trước mắt người lập trình, nhưng ta cũng có thể coi đó như một dãy bit và thao tác như thường.

b. Toán tử bit (bitwise)

- **Ứng dụng:** Toán tử bit thực hiện trên lần lượt từng bit của dãy bit.
- **Ưu điểm:** Giảm bộ nhớ và thời gian.
- **Lưu ý:**
 - Vài toán tử bit có độ ưu tiên thấp hơn các toán tử khác, nên tốt nhất là dùng dấu (và) cho mọi bitwise.
 - Dạng nhị phân của các số nguyên không dấu (unsigned) và số thực tương đối phức tạp và không cần thiết nên ta nên thao tác bitwise đối với các số nguyên có dấu (signed).

Toán tử	Ký hiệu	Tác dụng	Ví dụ	
AND	$a \& b$	Trả về 1 nếu cả hai bit đều là 1.	$(10110 \& 01010)$	00010
OR	$a b$	Trả về 1 nếu có ít nhất một bit là 1.	$(10110 01010)$	11110
XOR	$a \wedge b$	Trả về 1 nếu hai bit khác nhau.	$(10110 \wedge 01010)$	11100
NOT	$\sim a$	Trả về thể đảo của bit (0 $\rightarrow$ 1; 1 $\rightarrow$ 0)	(~ 10110)	01001
Dịch trái	$a \ll n$	Thêm n bit 0 vào bên phải. Nếu kiểu dữ liệu còn dư vị trí thì giữ nguyên n bit bên trái, nếu không thì xoá n bit bên trái. Sau khi $a \ll n$, $a = a * 2^n$.	$(10110 \ll 2)$	1011000
Dịch phải	$a \gg n$	Thêm n bit 0 vào bên trái, xoá n bit bên phải. Sau khi $a \gg n$, $a = a / 2^n$.	$(10110 \gg 2)$	00101

- **Một số thao tác hỗn hợp:**
 1. **Cách lấy bit thứ i của dãy bit a:** $((a \gg i) \& 1)$ (bit thứ i sẽ trở thành bit cuối sau khi dịch, nếu bit cuối là 1, kết quả toàn phép tính là 1 và ngược lại, phần còn lại của sẽ bị tiêu biến do các bit 0 của 1).

VD: Lấy bit thứ 2 của dãy bit 10110:

$$\begin{array}{r} 10110 \\ \gg 2 \quad 00101 \\ \& \quad 00001 \\ \hline 00001 \end{array}$$

2. **Cách đặt bit thứ i của dãy bit a thành bit 1 (set):** $(a | (1 \ll i))$ (đưa bit 1 của số 1 tương ứng vị trí với bit cần set; khi đấy, dù bit cần set có là 0 hay 1 thì vẫn cho ra kết quả là 1 sau khi thực hiện OR với bit 1 tương ứng vị trí).

VD: Set bit thứ 3 của dãy bit 10110:

$$\begin{array}{r} 1 == 00001 \\ \ll 3 \quad 01000 \\ a == 10110 \\ | \quad 01000 \\ \hline 11110 \end{array}$$

3. End.

- **Các thao tác kiểm tra đơn giản:**

1. **Kiểm tra tính chẵn lẻ: $(a \& 1)$** (Ở các số lẻ thì bit đầu tiên là 1, nên nếu biểu thức trên là true thì a lẻ).
2. **Kiểm tra 2 dãy bit a, b đan xen bit 0 – 1: $(a \& b)$** (Nếu 2 dãy bit có đan xen thì cho kết quả là 0 (do trong từng cặp bit tương ứng của 2 dãy bit thì không có cặp bit nào đều là 1), ngược lại thì cho kết quả > 0).

- **Các hàm:**

- **`__builtin_clz(n)`**: Đếm số lượng bit 0 ở cuối dãy bit nhị phân của số thập phân n.
- **`__builtin_ctz(n)`**: Đếm số lượng bit 0 ở đầu dãy bit nhị phân của số thập phân n.
- **`__builtin_popcount(n)`**: Đếm số lượng bit 1 trong dãy bit nhị phân của số thập phân n.

c.Lưu bit qua *bitset*

- Bitset là cấu trúc dữ liệu trong C++ tương đương với mảng bool nhưng được rút gọn bộ nhớ.
- Thứ tự các phần tử trong bitset cũng giống như dãy bit.
- Bitset có thể coi là 1 biến duy nhất và cũng có thể được chuyển thành kiểu dữ liệu khác. Nhưng nó có thể được tham chiếu đến từng bit riêng lẻ.

Khai báo:

<code>bitset <n> B;</code>	Tạo bitset B có n bit 0.
<code>bitset <n> B (d);</code>	Tạo bitset B có n bit 0, các bit đầu của B biểu diễn nhị phân của số thập phân d.
<code>bitset <n> B ("b");</code>	Tạo bitset B có n bit 0, các bit đầu của B tương ứng với các bit của dãy bit b.

Các hàm:

<code>B.set();</code>	Đưa toàn bộ bit thành 1.
<code>B.reset();</code>	Đưa toàn bộ bit thành 0.
<code>B.all();</code>	Trả về true nếu toàn bộ là bit 1.
<code>B.none();</code>	Trả về true nếu toàn bộ là bit 0.
<code>B.flip();</code>	Đảo toàn bộ bit.
<code>B.any();</code>	Trả về true nếu có bit 1.
<code>B.count();</code>	Trả về số lượng bit 1.

CHƯƠNG 2 : DÃY SỐ

§1 : DÃY SỐ CÓ QUY LUẬT

① BIỂU DIỄN DÃY SỐ CÓ QUY LUẬT

Một dãy số có quy luật có thể được biểu diễn bằng cách cho biết công thức tính giá trị của phần tử thứ i trong dãy đó:

- **Công thức tổng quát:**
 - Chỉ cần biết i là sẽ biết giá trị của $A_{[i]}$.
 - VD: Công thức tổng quát của phần tử thứ i của dãy A: $A_{[i]} = \frac{5}{i}$.
- **Công thức truy hồi:**
 - Để biết giá trị của $A_{[i]}$, cần biết giá trị các phần tử trước nó. Giá trị của những phần tử đầu tiên được cho trước.
 - VD: Công thức truy hồi của dãy Fibonacci với quy luật: Phần tử thứ i bằng tổng hai phần tử liền trước nó, với $F_{[0]} = F_{[2]} = 0$, $F_{[i]} = F_{[i-0]} + F_{[i-2]}$.

② DÃY CẤP SỐ CỘNG VÀ CẤP SỐ NHÂN

Lưu ý: Coi dãy số U bắt đầu từ vị trí $U_{[1]}$.

	Cấp số cộng	Cấp số nhân
Công thức truy hồi	$U_{[i]} = U_{[i-1]} + d$ (d gọi là công sai, và $U_{[1]}$ có sẵn)	$U_{[i]} = qU_{[i-1]}$ (q gọi là công bội, và $U_{[1]}$ có sẵn)
Công thức tổng quát	$U_{[i]} = U_{[1]} + (i - 1)d$	$U_{[i]} = U_{[1]}q^{i-1}$
Tổng từ $U_{[1]}$ đến $U_{[i]}$	$S_{[i]} = iU_{[1]} + \frac{di(i-1)}{2} = \frac{i(U_{[1]} + U_{[i]})}{2}$	$S_{[i]} = \frac{U_{[1]}(q^i - 1)}{q - 1}$
Tính chất đặc biệt	$U_{[i]} = \frac{U_{[i-k]} + U_{[i+k]}}{2}$ $d = \frac{U_{[i]} - U_{[j]}}{i - j}$ (i, j bất kì)	$U_{[i]} = \sqrt{U_{[i-1]}U_{[i+1]}}$

§2 : TÌM KIẾM TUYẾN TÍNH

- **Linear search:** Tìm kiếm tuyến tính.
- Tìm kiếm tuyến tính là duyệt tuần tự từng phần tử một trong danh sách.
- **Độ phức tạp:** $O(n)$.

① INDEX – ITERATOR

Nói về vị trí của phần tử trong danh sách, có 2 kiểu vị trí:

	Index (i)	Iterator (it)
Dùng để	Duyệt mảng, string, vector.	Duyệt các danh sách không cố định chiều dài (vector, string, set,...).
Ý nghĩa	Cho biết vị trí của phần tử so với mảng. VD: Vị trí thứ 0, 1, 2,...	Cho biết địa chỉ của phần tử trong bộ nhớ. VD: 0x7fff5fbff80c, ...

Có dạng	i có dạng là 1 số tự nhiên. VD: Vị trí thứ 3 trong mảng A có vị trí index $i = 3$.	$it = A.begin() + i$. Với A là danh sách mà iterator trở đến, $A.begin()$ là hàm iterator. VD: Vị trí thứ 3 trong mảng A là $it = A.begin() + 3$. it có thể được thay bằng các hàm như sau: $A.begin()$: Vị trí thứ 0 của A. $A.rbegin()$: Vị trí cuối cùng của A. $A.end()$: Vị trí sau vị trí cuối cùng của A (ra khỏi giới hạn của A). $A.rend()$: Vị trí trước vị trí cuối cùng của A (ra khỏi giới hạn của A).
Cách lấy dữ liệu	$A[i]$	$*it$ Lưu ý: $*A.end()$ và $*A.rend()$ mang giá trị rác.
Tốc độ	Nhanh hơn.	Chậm hơn.

② FOR – FOR-EACH

Có 2 cách duyệt trong C++:

- For:** Dùng vòng for thông thường, sử dụng biến lưu index hoặc iterator để duyệt.
- For-each:** Chỉ có từ C++11 trở về sau, là cách viết vắn tắt của for.

Bốn cách duyệt sau có cùng tác dụng:

```

1. vector<int> A;
2.
3. //FOR
4. //index
5. for(int i = 0; i < A.size(); ++i){
6.     cout << A[i];
7. }
8. //iterator
9. for(vector<int> :: iterator it = A.begin(); it != A.end(); ++it){
10.    cout << *it;
11. }
12. //(từ C++11 trở đi mới có từ khoá auto)
13. for(auto it = A.begin(); it != A.end(); ++it){
14.    cout << *it;
15. }
16.
17. //FOR-EACH (từ C++11 trở đi)
18. for(int i : A){
19.    cout << i;
20. }
```

Lưu ý: Biến i trong for-each tương tự như tham số của hàm. Có thể biến nó thành tham biến như sau:

```

1. for(int &i : A){
2.     cin >> i;
3. }
```

Như vậy, khi i thay đổi thì phần tử tương ứng trong A cũng sẽ thay đổi.

§3 : CÁC HÀM CÓ SẴN TRONG C++

① CÁC HÀM ĐƠN GIẢN

- Các hàm thao tác trên danh sách thường có cú pháp dạng **function(s, e, func)**. Có ý nghĩa là: thực hiện thao tác trong danh sách A trong phạm vi [s; e). Như vậy, giả sử ta muốn hàm thao tác trên A trong đoạn [2, 3], thì $s = 2$ và $e = 4$.
- Có những cách ghi s và e như sau:

Phạm vi	
s	e

Mảng	<code>A + s</code>	<code>A + e</code>
Vector	<code>A.begin() + s</code>	<code>A.begin() + e</code>
Duyệt hết vector A	<code>A.begin()</code>	<code>A.end()</code>
Dùng chung cho các danh sách	<code>&A[s]</code>	<code>&A[e]</code>

- **Lưu ý:** Hàm `A.end()` đã trả về vị trí sau vị trí cuối cùng của vector A.

<code>max_element(s, e)</code>	Trả về iterator vị trí của phần tử lớn nhất trong phạm vi.
<code>min_element(s, e)</code>	Trả về iterator vị trí của phần tử nhỏ nhất trong phạm vi.
<code>accumulate(s, e, value)</code>	Tính tổng các phần tử trong phạm vi và cộng thêm value.
<code>fill(s, e, value);</code>	Gán cho mọi phần tử trong phạm vi bằng giá trị value.
<code>find_if(s, e, check)</code>	Trả về iterator vị trí của phần tử thỏa mãn điều kiện trong hàm check trong phạm vi. Hàm check có kiểu bool và 1 tham số x tượng trưng cho 1 phần tử trong A trong quá trình tìm kiếm.
<code>find(s, e, x)</code>	Trả về iterator vị trí đầu tiên của x trong phạm vi.
<code>search(s, e, s', e')</code>	Trả về iterator vị trí đầu tiên của phạm vi B xuất hiện trong phạm vi A. VD: • A: 4 ; 3 ; 2 ; 1. • B: 2 ; 1. • Vậy hàm search trả về iterator trở đến vị trí 2 trong A.
<code>reverse(s, e);</code>	Đảo ngược vị trí các phần tử trong phạm vi.

<code>A.clear();</code>	Xoá hết mọi phần tử của A.
<code>A.size()</code>	Trả về số lượng phần tử của A.
<code>A.empty()</code>	Trả về true nếu A rỗng.

Lưu ý: Đối với các hàm trả về iterator, nếu không thể tìm thấy phần tử cần tìm thì trả về `e`.

② THUẬT TOÁN SẮP XẾP

Trong C++ có 2 hàm tương ứng 2 kiểu sort khác nhau:

- **Unstable sort (intro sort):** Vị trí các phần tử giống nhau không được bảo toàn. Unstable có thể có nhiều kết quả.
- **Stable sort (merge sort):** Vị trí các phần tử giống nhau được bảo toàn.

Ví dụ: Sort mảng sau theo thứ tự tăng dần của giá trị tuyệt đối:

Danh sách	1	<u>-1</u>	2	3	<u>-2</u>
Unstable sort	<u>-1</u>	1	<u>-2</u>	2	3
Stable sort	1	<u>-1</u>	2	<u>-2</u>	3

Lưu ý: Có thể có nhiều phương án để sort là sort là unstable sort nhưng chỉ có 1 phương án là stable sort.

Độ phức tạp: $O(n \log n)$.

Cú pháp:

Unstable sort	<code>sort(s, e, func);</code>
Stable sort	<code>stable_sort(s, e, func);</code>

Trong đó, func là hàm phụ, có thể có hoặc không. Hàm này có thể được người dùng tự xây dựng hoặc dùng các hàm có sẵn:

- Hàm func được xây dựng theo cú pháp `bool tên_hàm_phụ (type a, type b) {}`. Trong đó:
 - **type a, type b:** 2 phần tử cùng loại trong danh sách, mang kiểu của danh sách (không thể là auto).
 - Nếu hàm trả về true thì a sẽ được xếp trước b, và ngược lại.
 - Trong {} thường chỉ có duy nhất dòng return.
- VD khi muốn sắp xếp mảng `int A[n]` theo kiểu giảm dần.
 - Khai báo hàm: `bool cmp (int a, int b) {return a < b;}`

- Cách dùng: `sort(&A[0], &A[n], cmp);`
- Lưu ý:
 - Khi khai báo comparator là `comp` hoặc `greater<type>()` thì dãy sẽ được xếp giảm dần.
 - Nếu không khai báo hàm này, thì mặc định sắp xếp tăng dần.

§4 : CẤU TRÚC DỮ LIỆU TUYẾN TÍNH

- *Linear data structure: Cấu trúc dữ liệu tuyến tính.*
- Cấu trúc dữ liệu là cách lưu trữ dữ liệu (danh sách).
- Cấu trúc dữ liệu tuyến tính là danh sách lưu trữ các phần tử liên tiếp nhau.
- Có 2 loại cấu trúc thông dụng là mảng và vector.
- Ưu điểm: Dễ thao tác, quản lý, tham chiếu.
- Nhược điểm: Tốn thời gian duyệt.

① MẢNG ĐỘNG (VECTOR)

- Vector là mảng động có thể tự động thay đổi kích thước (số lượng phần tử) mà không cần cố định trước kích thước như mảng.
- Lưu ý: Mảng chỉ có thể chứa khoảng 10^5 phần tử (không báo lỗi khi build), nhưng vector thì có thể.

Cú pháp:

<code>vector <type> vt;</code>	Khai báo vector tên vt, kiểu type.
<code>vt[pos]</code>	Trả về phần tử thứ pos của vector.
<code>vt.push_back(x);</code>	Đưa x vào cuối vector.
<code>vt.insert(it, x);</code>	Đưa x vào vị trí trước vị trí mà iterator it trỏ đến (riêng chuỗi thì it là index).
<code>vt.clear();</code>	Xóa toàn bộ phần tử của vector.
<code>vt.pop_back();</code>	Xóa phần tử cuối cùng khỏi vector.
<code>vt.erase(s);</code> <code>vt.erase(s, e);</code>	Xóa phần tử ở iterator s. Xóa các phần tử trong phạm vi.
<code>vt.back()</code>	Trả về phần tử cuối cùng của vector.
<code>vt.front()</code>	Trả về phần tử đầu tiên của vector.
<code>swap(a, b);</code>	Hoán đổi các phần tử giữa 2 vector a và b.

② NGĂN XẾP (STACK)

Nguyên tắc hoạt động:

- Stack là một vector hoạt động theo nguyên tắc LIFO (last in first out) – phần tử nào vào sau thì phần tử đó ra trước.
⇒ Các phần tử vào ra từ một đầu của stack, tạm coi như ra vào từ cuối stack.
- Không thể tham chiếu đến vị trí hay duyệt stack.

Ứng dụng: Như một kho lưu trữ tạm thời và được xét ngược lại với thứ tự ban đầu. VD:

- Ta đang duyệt mảng A bằng biến i. Ở vị trí i, ta muốn duyệt ngược lại (i-1, i-2, i-3,...).
- Thay vì tốn thêm một vòng for duyệt ngược nữa, ta chỉ cần lần lượt đưa A[i] vào stack. Khi nào muốn duyệt ngược, ta lấy A[i] ra khỏi stack.
- Tuy độ phức tạp không đổi so với for ngược, nhưng thực tế, việc khởi tạo một vòng for có thể tốn nhiều thời gian hơn việc lấy 1 đỉnh ra khỏi stack.

Cú pháp (độ phức tạp $O(1)$):

<code>stack <type> stk;</code>	Khai báo stack tên stk, kiểu type.
<code>stk.push(x);</code>	Đưa x vào cuối stack.
<code>stk.pop();</code>	Xóa phần tử cuối cùng ra khỏi stack.
<code>stk.top()</code>	Trả về phần tử cuối cùng của stack.

Bài toán: Dãy ngoặc đúng

- Dãy ngoặc đúng là dãy ngoặc phù hợp với các điều kiện sau:
 - Nếu A là dãy ngoặc đúng thì (A) là dãy ngoặc đúng.
 - Nếu A và B là 2 dãy ngoặc đúng thì AB là dãy ngoặc đúng.
- VD: ((()))() là dãy ngoặc đúng,)()(()) là dãy ngoặc sai.
- Cho một xâu chỉ gồm các dấu (và). Kiểm tra xem xâu đó có phải dãy ngoặc đúng không.

INPUT	OUTPUT
Dãy ngoặc S	- Xuất ra 1 nếu dãy ngoặc đúng. - Xuất ra 0 nếu dãy ngoặc sai.
((()))()	1

- Ý tưởng:
 - Duyệt S:
 - Nếu $S[i-1] = '('$ và $S[i] = ')'$ $\Rightarrow$ Thỏa mãn cặp ngoặc đúng $\Rightarrow$ Xóa $S[i-1]$ và $S[i]$ ra khỏi S. Sau đó duyệt lại từ đầu và tiếp tục kiểm tra điều kiện.
 - Nếu kết thúc một lần duyệt nào đó mà không xóa được cặp $S[i-1]$ và $S[i]$ nào $\Rightarrow$ S sai. Ngược lại, nếu cứ lặp và xóa mãi, đến khi S rỗng thì S đúng.
 - Quá trình thực hiện như sau: Duyệt S = "((()))()":
 - Có $S[1] = '('$ và $S[2] = ')'$ $\Rightarrow$ Xóa $S[1]$ và $S[2]$, còn lại S = "(()())".
 - Có $S[0] = '('$ và $S[1] = ')'$ $\Rightarrow$ Xóa $S[0]$ và $S[1]$, còn lại S = "(())".
 - Có $S[0] = '('$ và $S[1] = ')'$ $\Rightarrow$ Xóa $S[0]$ và $S[1]$, còn lại S = "()".
 - Cuối cùng, S rỗng nên S đúng.
 - Ý tưởng dùng stack:
 - Có thể thấy ý tưởng trên có nhược điểm là phải duyệt đi duyệt lại tốn thời gian. Ta có thể dùng stack để không cần phải duyệt đi duyệt lại.
 - Giả sử ta xóa $S[i-1]$ và $S[i]$ bằng cách đánh dấu (có thể dùng mảng bool, như $check[i-1] = check[i] = false$). Vậy nên, việc xóa $S[i-1]$ và $S[i]$ cũng giống như xóa $S[j]$ và $S[x]$ ($j < i-1, x > i$) $\Rightarrow$ x tăng dần theo chiều duyệt của S, còn j chính là duyệt ngược.
 - Vậy, ở mỗi lần duyệt, thay vì xét $S[i-1]$ và $S[i]$, có thể xét $stk.top()$ và $S[i]$:
 - Nếu $stk.top() = '('$ và $S[i] = ')'$ thì đưa $S[i]$ vào stack rồi pop 2 lần (xóa $stk.top()$ cũ và $S[i]$ cũ).
 - Nếu không, chỉ đơn thuần đưa $S[i]$ vào stack.
 - Cuối cùng, trong stack chỉ còn những dấu ngoặc không thể xóa $\Rightarrow$ Nếu stack rỗng thì S đúng, nếu không thì S sai.

Bài toán: Xóa k chữ số

- Cho số tự nhiên N và k. Hãy xóa k chữ số trong số N 2 lần sao cho thu được 2 số là số lớn nhất và nhỏ nhất.

INPUT	OUTPUT	Giải thích test
N k	Số lớn nhất sau khi xóa k chữ số. Số nhỏ nhất sau khi xóa k chữ số.	
12093 3	93 3	Các chữ số bị xóa được tô cam: 12093 12093

- Ý tưởng:
 - Xét $N[i]$ và $N[i-1]$.
 - Tìm số lớn nhất:
 - Duyệt các chữ số của N: Nếu $N[i-1] < N[i] \Rightarrow$ Dãy số tăng dần $\Rightarrow$ Xóa $N[i]$ để được số lớn nhất có thể. Sau đó duyệt lại từ đầu và tiếp tục kiểm tra điều kiện.
 - Nếu kết thúc các lần duyệt mà chưa xóa đủ k chữ số $\Rightarrow$ Không tìm thấy $N[i-1] < N[i] \Rightarrow$ Dãy số giảm dần $\Rightarrow$ Lần lượt xóa thêm các chữ số cuối cùng có cho đến khi đủ k chữ số.
 - Quá trình thực hiện như sau: Duyệt N = 12093:
 - Có $1 < 2 \Rightarrow$ Xóa 1, còn lại 2093.
 - Có $0 < 9 \Rightarrow$ Xóa 0, còn lại 293.

- Có $2 < 9 \Rightarrow$ Xoá 2, còn lại 93.
- Đã xoá đủ 3 chữ số, kết thúc quá trình.
 - Tìm số bé nhất: Cũng tương tự tìm số lớn nhất nhưng ta kiểm tra điều kiện $N[i-1] > N[i]$.
- Ý tưởng dùng stack:
 - Có thể thấy ý tưởng trên có nhược điểm là phải duyệt đi duyệt lại tốn thời gian. Ta có thể dùng stack để không cần phải duyệt đi duyệt lại.
 - Giả sử ta “xoá” $N[i-1]$ bằng cách đánh dấu. Vậy nên, việc kiểm tra $N[i-1] < N[i]$ thực chất là $N[j] < N[i]$ ($j < i$), và nó chính là duyệt ngược.
 - Vậy, có thể lần lượt đưa $A[i]$ vào stack và kiểm tra $stk.top() < A[i]$.
 - Tóm lại:
 - Duyệt từng chữ số của N: Kiểm tra $stk.top() < A[i]$.
 - Nếu đúng, xoá $stk.top()$ khỏi stack và liên tục kiểm tra lại điều kiện trên cho đến khi sai.
 - Nếu sai, đưa $A[i]$ vào stack.
 - Nếu sau khi duyệt hết N mà vẫn chưa xoá đủ k chữ số, ta lần lượt xoá các chữ số cuối stack cho đến khi đủ k chữ số.
 - Cuối cùng, trong stack chỉ còn lại những chữ số không bị loại bỏ.
 - **Lưu ý:** Việc lấy các chữ số trong stack ra để xuất thành kết quả sẽ khiến thứ tự các chữ số bị đảo ngược. Để khắc phục, ta dùng hàm reverse. Tuy tốn khá nhiều thao tác nhưng vẫn nhanh hơn so với cách lặp đi lặp lại cũ.

3 HÀNG ĐỢI (QUEUE)

Nguyên tắc hoạt động:

- Queue là một vector hoạt động theo nguyên tắc FIFO (first in first out) – phần tử nào vào trước thì phần tử đó ra sau $\Rightarrow$ Các phần tử vào ra từ hai đầu của queue.
- Không thể tham chiếu đến vị trí hay duyệt queue.

Cú pháp:

<code>queue <type> qu;</code>	Khai báo queue tên qu, kiểu type.
<code>qu.push(x);</code>	Đưa x vào cuối queue.
<code>qu.pop();</code>	Xoá phần tử đầu tiên ra khỏi queue.
<code>qu.front();</code>	Trả về phần tử đầu tiên của queue.
<code>qu.back();</code>	Trả về phần tử cuối cùng của queue.

Bài toán: Số may mắn

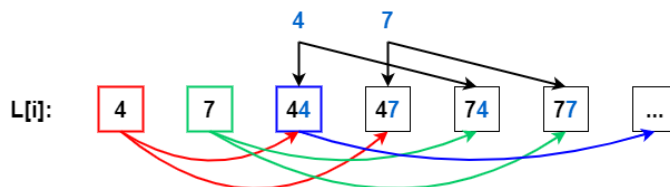
- <https://claoj.edu.vn/problem/lucky>
- Trong lĩnh vực số học Vicky coi các số ở hệ 10 chỉ chứa các chữ số 4 và 7, tức là các số chỉ chứa toàn số 4, chứa toàn số 7 hoặc cả 4 lẫn 7 là những số may mắn. Các số may mắn được Vicky đánh số theo giá trị tăng dần của chúng, bắt đầu từ 1. Các số may mắn đầu tiên là 4, 7, 44, 47,...
- **INPUT:** n ($0 < n < 1010$).
- **OUTPUT:** Số may mắn thứ n.

INPUT	OUTPUT
3	44

- Ý tưởng:
 - Bài toán này nếu dùng for rồi lần lượt kiểm tra tính may mắn của số i thì sẽ rất lâu. Ta nghĩ đến việc sinh ra số may mắn chứ không dùng for.
 - Gọi dãy số may mắn là L. Khảo sát giá trị đến $L[8]$:

$L[1] = 4$	$L[2] = 7$
$L[3] = 44$	$L[4] = 47$
$L[5] = 74$	$L[6] = 77$
$L[7] = 444$	$L[8] = 447$

- Suy ra, với hai số đầu là 4 và 7, lần lượt cộng thêm chuỗi 4 và 7 cho $L[1]$ và $L[2]$, sinh ra $L[3]$ và $L[4]$. Rồi từ $L[3]$ và $L[4]$, lần lượt cộng thêm chuỗi 4 và 7 sinh ra $L[5]$ và $L[6]$... Vậy với $L[1]$ và $L[2]$ ta cho sẵn, lần lượt cộng thêm chuỗi 4 và 7 để sinh ra các L tiếp theo cho đến khi sinh ra $L[n]$. Ta dùng queue để thực hiện ý tưởng bài toán:
 - Cho trước $L[1] = 4$ và $L[2] = 7$, $cnt = 2$. Lần lượt đưa hai phần tử này vào queue.
 - Lần lượt lấy $queue[0]$ ra, lần lượt cộng thêm chuỗi 4 và 7 vào $queue[0]$ để tạo thành $L[i]$ và $L[i+1]$. Đưa hai số mới vừa sinh ra này lần lượt vào queue.
 - Lặp lại bước trên cho đến khi đã sinh ra n số, hay $L.size() - 1 = n$, xuất ra $L[n]$.
 - Sơ đồ sinh số từ $L[1]$ đến $L[6]$:



4 MAP VÀ SET

a.Map

Giới thiệu:

- Map là vector các pair, tức là mỗi phần tử của map là một cặp giá trị {key, value}, hay $map[key] = value$. Các cặp được sắp xếp theo chiều tăng dần của key và không có cặp nào trùng key với nhau.
- Nếu chủ động gán trước giá trị cho key, value mặc định được gán bằng 0.
- Có thể duyệt set bằng for-each và iterator. Do mỗi phần tử của map là pair, nên biến duyệt i phải có kiểu pair. Tham chiếu đến key của $map[i]$ thông qua từ khóa first, và tham chiếu đến value qua từ khóa second.

Cú pháp:

<code>map <type1, type2> mp;</code>	Khai báo map tên mp, có key kiểu type1 và value kiểu type 2.
<code>mp[key] = value;</code>	Gán giá trị value cho key.
<code>mp.erase(key);</code>	Xóa pair {key, value} ra khỏi map.
<code>mp.find(key);</code>	Trả về iterator trỏ đến vị trí của pair {key, value}.

Độ phức tạp: $O(n \log n)$.

b.Set

Giới thiệu:

- Set là vector mà trong đó không có phần tử trùng nhau, kể cả có cố tình đẩy phần tử trùng nhau vào set.
- Các phần tử trong set được sắp xếp theo thứ tự tăng dần.
- Có thể duyệt set bằng for-each và iterator.
- Không thể thay đổi giá trị các phần tử trong set.

Cú pháp:

<code>set <type> st;</code>	Khai báo set tên st, kiểu type.
<code>st.insert(x);</code>	Đưa x vào set.
<code>st.erase(x);</code>	Xóa x ra khỏi set.
<code>st.clear();</code>	Xóa hết các phần tử trong set.
<code>st.find(x);</code>	Trả về iterator trỏ đến vị trí của x.

Độ phức tạp: $O(n \log n)$.

§5 : THUẬT TOÁN TÌM KIẾM NHỊ PHÂN

- *Binary search: Tìm kiếm nhị phân..*

① THUẬT TOÁN TÌM KIẾM NHỊ PHÂN CƠ BẢN

Vấn đề: Cho một dãy A có n phần tử, kiểm tra xem x có nằm trong A không (tìm kiếm x).

Tìm kiếm nhị phân (chặt nhị phân):

- Là thuật toán cải tiến từ tìm kiếm tuyến tính (duyệt tuyến tính).
- **Điều kiện:** Dãy A đã được sắp xếp theo chiều tăng dần hoặc giảm dần. Ở đây hướng dẫn thứ tự tăng dần:
- **Ý tưởng:**
 - Trong dãy A tăng dần, nếu $x > A[i]$ ($1 \leq i < n$) thì dĩ nhiên mọi $A[j]$ ($0 \leq j < i$) đều $<$ hơn x.
 - Như vậy, ta không cần phải tìm kiếm trên đoạn $[0; i]$ mà chỉ cần tìm kiếm trên khoảng $(i; n)$.
- **Thực hiện:**
 - Gọi l và r lần lượt là vị trí tận cùng bên trái và tận cùng bên phải của đoạn cần xét. Gọi mid là vị trí ở giữa l và r, hay $mid = \frac{l+r}{2}$.
 - Nếu $l > r$ thì không còn đoạn cần xét, tức không tìm thấy x trong dãy A.
 - Xét $A[mid]$:
 - Nếu $x > A[mid] \Rightarrow$ Không cần tìm kiếm trên $[l; mid]$ mà chỉ cần tìm kiếm trên $(mid; r] \Rightarrow$ Đặt $l = mid + 1$ và quay về bước đầu.
 - Nếu $x < A[mid] \Rightarrow$ Không cần tìm kiếm trên $[mid; r]$ mà chỉ cần tìm kiếm trên $[l; mid) \Rightarrow$ Đặt $r = mid - 1$ và quay về bước đầu.
 - Nếu $x = A[mid]$
 - $\Rightarrow$ Đã tìm thấy x, kết thúc thuật toán.
- **Độ phức tạp:** $O(\log n)$.
- **Quy trình thực hiện như sau (những ô tô màu cam là phạm vi tìm kiếm):**

				2								:x
				0	1	2	3	4	5	6	7	:i
l r mid				0	1	1	1	2	4	7	9	:A
Bước 1:	0	7	3	0	1	1	<u>1</u>	2	4	7	9	$x < A[mid] \Rightarrow l = mid + 1.$
Bước 2:	4	7	5	0	1	1	1	2	<u>4</u>	7	9	$x < A[mid] \Rightarrow r = mid - 1.$
Bước 3:	4	4	4	0	1	1	1	<u>2</u>	4	7	9	$x = A[mid] \Rightarrow$ Đã tìm thấy x.

Như vậy, chỉ với 3 bước ta đã kiểm tra thành công kiểm tra xem x có thuộc A (có 8 phần tử) hay không.

```

1. //Hàm trả về true nếu tìm thấy x thuộc A
2. bool binSearch(int l, int r){
3.     if(l > r)
4.         return false;
5.     int mid = (l+r) / 2;
6.     if(x > A[mid])
7.         return binSearch(mid+1, r);
8.     else if(x < A[mid])
9.         return binSearch(l, mid-1);
10.    else
11.        return true;
12. }
```

② MỞ RỘNG ỨNG DỤNG THUẬT TOÁN TÌM KIẾM NHỊ PHÂN

- **Phương pháp:** Đặt mid là đáp án cần tìm. Lần lượt tối ưu mid bằng tìm kiếm nhị phân.

Ứng dụng equal-range:

Bài toán: Độ lệch k

- Nguồn: Sở giáo dục và đào tạo, tỉnh Bà Rịa – Vũng Tàu, đề thi Olympic 27 tháng 4 học sinh giỏi lớp 10, 11, năm học 2022 – 2023.
- Cho 2 số tự nhiên n và k và một dãy số tự nhiên A có n chữ số. Hãy tìm số lượng lớn nhất các cặp số có chung độ lệch k.

INPUT	OUTPUT	Giải thích test
n k A[i]	Như yêu cầu	
6 2 3 5 5 5 10 7	6	6 cặp số thỏa đề là {3; 5}, {3; 5}, {3; 5}, {7; 5}, {7; 5}, {7; 5}.

- Ý tưởng:
 - Duyệt i hết dãy A:
 - Duyệt j đến hết A ($j \neq i$):
 - Nếu $|A[j] - A[i]| = k$ thì tăng biến đếm kết quả lên 1.
 - Xuất ra biến đếm trên.
- Ý tưởng dùng tìm kiếm nhị phân:
 - Ý tưởng trên tốn khoảng $O(n^2)$ nên có thể TLE, có thể dùng tìm kiếm nhị phân để tối ưu.
 - Với A[i], nếu A[i] + k tồn tại trong A thì có 1 cặp hợp lệ.
 - Tuy nhiên, có thể có nhiều A[i] + k cùng tồn tại. Vậy, ta duy trì 2 biến s và e lần lượt là điểm đầu và điểm cuối của đoạn con $= A[i] + k$. VD test mẫu: Với A[0] = 3, ta tìm được A[0] + 2 = 5. Giả sử khi tìm được 5, mid = 3, vậy ta cho s = 3, e = 3, while(A[s-1]==A[mid]) --s, while(A[e+1]==A[mid]) ++e để s và e lần lượt thành 1, 3. Cuối cùng, cộng dồn s-3+1 vào biến lưu đáp án cần xuất ra.
 - Tóm lại: Duyệt i đến hết dãy A, ở mỗi A[i], tìm xem có tồn tại A[i] + k trong A không. Nếu có, tìm số lượng A[i] + k trong A và cộng dồn số lượng đếm được vào biến kết quả.

Ứng dụng lower-bound, upper-bound:

Bài toán: Robot

- https://claoj.edu.vn/problem/thtla_22c_3
- Kiến thức nền tảng: Lý thuyết đồ thị.
- Một bản đồ hình vuông kích thước nxm được chia thành lưới các ô vuông, trong mỗi ô vuông có một số nguyên không âm thể hiện độ cao của ô vuông đó.
- Một robot có khả năng nhảy cao d đơn vị sẽ đi được sang một trong 4 ô kề cạnh nếu chênh lệch độ cao của ô đang đứng và ô sẽ đến không quá d.
- Hãy tìm d nhỏ nhất sao cho với d này, robot xuất phát từ 1 ô bất kì có thể đi hơn một nửa bản đồ (nếu tổng số ô của bản đồ là số lẻ thì một nửa được làm tròn lên).

INPUT	OUTPUT
n k A[i]	Như yêu cầu
5 0 0 0 3 3 0 0 0 0 3 0 9 9 3 3 9 9 9 3 3 9 9 9 9 3	3

- Ý tưởng:
 - Gọi mid là độ chênh lệch để robot có thể đi hơn một nửa bản đồ. Vậy $0 \leq mid \leq d_{\max}$, với $d_{\max}$ là độ chênh lệch lớn nhất trong bản đồ (thu được bằng cách thực hiện BFS từ 1 ô bất kì trên bản đồ), tức là $l = 0$ và $r = d_{\max}$.
 - Ở mỗi $mid = \frac{l+r}{2}$, ta kiểm tra xem với mid đó thì robot có đi hơn một nửa bản đồ không (kiểm tra bằng cách BFS từ ô chưa được đi trên bản đồ). Nếu không, ta loại mid này và kiểm tra mid khác bằng cách cho $l = mid + 1$.
 - Tuy nhiên, sau khi tìm được mid, ta vẫn chưa thể khẳng định đây là đáp án cần tìm vì đề yêu cầu tìm độ chênh lệch nhỏ nhất có thể. Vậy, ta tạm lưu mid vào danh sách các mid (mỗi mid trong danh sách này đều có thể là đáp án), cho $r = mid - 1$ và tiếp tục tìm mid cho đến khi không tìm được nữa ($l > r$).
 - Trong danh sách các mid, tìm ra mid bé nhất và xuất ra.
 - VD: Trong test mẫu, có thể tìm ra mid = 9, 6, 3. Cả 3 mid này đều có thể đi hơn một nửa bản đồ, nhưng ta chọn xuất ra 3 vì nó là mid nhỏ nhất.

Bài toán: Đường đi an toàn

- <https://claoj.edu.vn/problem/safepath>
- Kiến thức nền tảng: Lý thuyết đồ thị.
- Hôm nay các em ở trường Tiểu học & THCS La Ngâu đã tham gia một chuyến dã ngoại cùng các anh chị tình nguyện viên. Vì sơ suất nên các bạn không may đã lạc vào nơi ở của một loài thú dữ. Là một trong số những tình nguyện viên, bạn sẽ hướng dẫn các em đến khu vực an toàn. Bạn biết trước vị trí của các con thú dữ, để di chuyển an toàn thì phải chọn đường đi càng xa chúng càng tốt!
- Khu rừng có thể được mô tả bằng một hình chữ nhật kích thước $n \times m$. Những ô trống được đánh dấu bằng ký hiệu $.$, những ô có thú dữ là $\#$, vị trí ban đầu của bạn là S và khu vực an toàn là E . Bạn có thể di chuyển từ ô đang đứng sang các ô kề cạnh.
- Khoảng cách giữa ô $A(x_A, y_A)$ và $B(x_B, y_B)$ là $|x_A - x_B| + |y_A - y_B|$.
- Đường đi an toàn nhất được hiểu là đường đi mà khoảng cách bé nhất từ một ô nào đó trên đường đi đến con thú dữ gần nhất là lớn nhất. Xuất ra khoảng cách lớn nhất này.

INPUT	OUTPUT	Giải thích test
$n \ m$ $A[i][j]$	Như yêu cầu	
4 5###. ..#.#. S#.E#	0	• Đường đi an toàn đi qua các ô $(4; 1) \rightarrow (4; 2) \rightarrow (4; 3) \rightarrow (4; 4)$. • Khoảng cách từ các ô trên đến con thú gần nhất lần lượt là 1; 0; 1; 1. • Đáp án là 0 vì đây là khoảng cách ngắn nhất trong đường đi trên, đồng thời cũng là khoảng cách lớn nhất trong số các khoảng cách bé nhất của tất cả các đường đi khác.

- Ý tưởng:
 - Bài này cũng tương tự bài trên.
 - Để cho dễ, ta đánh đồng "ô có khoảng cách từ ô đó đến con thú dữ gần nhất là nhỏ nhất" là "ô nhỏ nhất", và ngược lại.
 - Gọi mid là ô nhỏ nhất trên đường đi từ s đến e . Vậy $0 \leq mid \leq d_{\max}$ ($d_{\max}$ là khoảng cách lớn nhất từ 1 ô bất kì đến con thú dữ gần nhất, hoặc đơn giản là n^2 , con số này ảnh hưởng tốc độ tính toán chứ không ảnh hưởng đến quả). Ở mỗi $mid = \frac{l+r}{2}$, ta kiểm tra xem nếu BFS(s) thì có thể đến được e không. Nếu không, tức là chẳng có đường đi nào mà ô nhỏ nhất của nó $\geq mid$, ta loại mid này và tìm mid khác bằng cách cho $l = mid + 1$. Nếu có, ta tạm thời đưa mid này vào danh sách các mid .
 - Cuối cùng, ta xuất ra mid lớn nhất trong danh sách các mid .
 - Lưu ý: Có thể dùng BFS để tính khoảng cách từ 1 ô đến con thú gần nhất (Ban đầu, $d[i][j] = +\infty$. $d[\text{ô có thú}] = 0$, BFS(ô có thú). Ở mỗi ô, xét ô kề cạnh, nếu $d[\text{ô kề cạnh}] > d[\text{ô hiện tại}] + 1$ thì $d[\text{ô kề cạnh}] = d[\text{ô hiện tại}] + 1$).

§6 : DÂY CON LIÊN TIẾP (ĐOẠN CON)

- *Sliding window*: Cửa sổ trượt.

❶ CỬA SỔ TRƯỢT

Giới thiệu: Dùng để tìm dãy con có độ dài cố định thỏa điều kiện nào đó trong dãy cho trước với độ phức tạp $O(n)$.

Bài toán: Dãy con liên tiếp có tổng lớn nhất có độ dài là k

- **INPUT:**
 - Dòng đầu chứa số nguyên k và n ($n > k$).
 - Dòng sau chứa n phần tử trong dãy A .
- **OUTPUT:** Tìm tổng của dãy con gồm k phần tử liên tiếp có giá trị cao nhất trong dãy A có n phần tử:

INPUT	OUTPUT
3 6	16

4 3 1 2 8 6	
-------------	--

- Giải thích test: Các dãy con có 3 phần tử liên tiếp:

Dãy con	Tổng
4 ; 3 ; 1	8
3 ; 1 ; 2	6
1 ; 2 ; 8	10
2 ; 8 ; 6	16 (nhận)

- Ý tưởng:

- Gọi ans là tổng lớn nhất thỏa đề. Ban đầu, ans = 0.
- Cửa sổ đầu tiên: Từ A[1] đến A[3], tổng là s.
- Cho “trượt” cửa sổ bằng cách dùng for i chạy từ 4 đến n:
 - Cộng thêm giá trị của A[i] vào s : $s += A[i]$.
 - Loại bỏ phần tử thứ A[i - k] do phần tử này hiện đã không nằm trong cửa sổ nữa: $s -= A[i-k]$.
 - So sánh ans và s. Nếu $ans < s$ thì $ans = s$.

Bài toán: Dãy số

- https://claoj.edu.vn/problem/hsg02_2020_dayso
- Dãy số đẹp là dãy số có các phần tử nguyên dương liên tiếp nhau, $A[i] = A[i-1] + 1$. Cho trước dãy B là một dãy số đẹp có giá trị từ 1 đến n và dãy A. Hãy chọn tối đa S phần tử từ B ghép vào A để dãy con sau khi đã chọn là một dãy số đẹp.

- INPUT:

- Dòng đầu chứa 3 số nguyên n, m, s có ý nghĩa lần lượt là:
 - Giá trị lớn nhất trong hai dãy.
 - Số lượng phần tử trong dãy A.
 - Số lượng phần tử tối đa có thể được chọn từ B.
- Dòng sau chứa M phần tử của dãy A.

- OUTPUT: Số lượng các dãy con đẹp được tạo ra theo cách trên.

INPUT	OUTPUT
10 5 2 7 1 3 5 6	5
10 6 2 5 5 5 5 5 5	0

- Giải thích test 1:

STT

B	1 2 3 4 5 6 7 8 9 10	
A	1 3 5 6 7	
1	1 <u>2</u> 3 <u>4</u> 5	Hợp lệ.
2	<u>2</u> 3 <u>4</u> 5 6	Hợp lệ.
3	3 <u>4</u> 5 6 7	Hợp lệ.
4	<u>4</u> 5 6 7 <u>8</u>	Hợp lệ.
5	5 6 7 <u>8</u> <u>9</u>	Hợp lệ.
6	6 7 <u>8</u> <u>9</u> <u>10</u>	Không hợp lệ.

- Ý tưởng:

- Gọi check[i] = true nếu i không có trong dãy A tức i được lấy từ B.
- Tạo cửa sổ đầu tiên: Cho i chạy từ 0 đến m, đếm số lượng phần tử true và gán vào count. Nếu count ≤ s thì đó là dãy hợp lệ.
- Trượt: Xét các cửa sổ còn lại bằng cách cho i chạy từ k + 1 đến n. Nếu check[i] = true thì ++ count, nếu check[i-k] = true thì -- count. Nếu count ≤ s thì đó là dãy hợp lệ.

THUẬT TOÁN KADANE

Giới thiệu: Dùng để tìm dãy con có độ dài không cố định thỏa điều kiện nào đó trong dãy cho trước với độ phức tạp $O(n)$.

📌 Bài toán: Dãy con liên tiếp có tổng lớn nhất có độ dài không cố định

- Cho mảng các số nguyên A. Tìm dãy con liên tiếp của A sao cho tổng các phần tử trong đoạn con là lớn nhất.

INPUT	OUTPUT	Giải thích test
7 -1 2 8 -3 9 -2 5	19	Đoạn con thỏa đề là {2; 8; -3; 9; -2; 5}.

• Ý tưởng:

- Gọi tmp là tổng dãy con hiện tại, ans là tổng dãy con thỏa đề (kết quả bài toán).
- Duyệt i từ 0 đến n-1 (duyet dãy A):
 - tmp += A[i] (Nếu tmp < 0 thì gán tmp = 0 (bỏ qua kết quả âm)).
 - ans = max(tmp, ans).
- Lưu ý:** Thuật toán không giải quyết trường hợp toàn bộ dãy A là số âm. Để khắc phục, ta tìm maxA là giá trị lớn nhất của A, nếu maxA < 0 thì xuất maxA, không sử dụng thuật toán.

§7 : MẢNG CỘNG DỒN

• Prefix sum: Mảng cộng dồn.

- Đối với những bài toán với dãy đòi hỏi xử lý dữ liệu lớn hay thao tác nhiều hành động như nhau trên cùng dãy, có thể sử dụng mảng cộng dồn thay vì for để tiết kiệm thời gian.
- Gọi S là mảng cộng dồn của mảng A có n phần tử, $S[0] = A[0]$, $S[i] = S[i-1] + A[i] (i > 0)$.
- VD0: n = 5.

i	1	2	3	4	5
A	3	5	2	6	2
S	3	3 + 5 = 8	8 + 2 = 10	10 + 6 = 16	16 + 2 = 18

📌 Ứng dụng 1: Tính tổng các phần tử trong mảng A từ vị trí l đến vị trí r

- Gọi sum là tổng của các phần tử của A từ l đến r. Với S là mảng cộng dồn của A, $sum = S[r] - S[l - 1]$.
- VD: Cho S như VD0, l = 2, r = 4: $sum = S[4] - S[1] = 13$.

📌 Ứng dụng 2: Tăng giá trị của từng phần tử trong mảng A từ vị trí l đến vị trí r lên k đơn vị

- Tạo mảng temp có $temp[l] = k$ và $temp[r+1] = -k$, các phần tử còn lại của temp = 0.
- Tạo mảng S là mảng cộng dồn của temp.
- Cộng mảng A và mảng S lại, thỏa yêu cầu của đề.
- VD: Cho A như VD0, l = 2; r = 4; k = 10.

i	1	2	3	4	5
A	3	5	2	6	2
temp	0	10	0	0	-10
S	0	10	10	10	0
A sau khi cộng	3	15	12	16	2

§8 : KỸ THUẬT BĂM DỮ LIỆU

• Hash: Băm.

① BẢNG BĂM (HASH TABLE)

a. Quy trình băm dữ liệu

- Băm dữ liệu là biến dữ liệu cần được lưu trữ (value) thành key. Key có vai trò như index. Mô hình này gọi là bảng băm. Gọi hashTable là bảng băm:

Dữ liệu cần được lưu trữ (value) $\xrightarrow{\text{hàm băm}}$ key $\Rightarrow$ hashTable[key] = value.

- Để tìm kiếm value trong bảng băm tốn thời gian tốt nhất $O(n)$ và tệ nhất $O(n)$.

b. Hàm băm

- Hàm băm hash(value) là hàm có chức năng biến đổi value thành key. Bảng băm hoạt động có hiệu quả hay không nhờ vào hàm băm
- Thông thường, hàm hash(value) dùng để chia dư value cho số nguyên tố p nào đó. Khi đó, key trả về luôn rơi vào giá trị từ 0 đến p – 1. Do đó, p là kích thước của bảng băm.
- VD1:** Dùng hashTable để lưu trữ các số tự nhiên.
 - Có hàm băm key = hash(value) = value % 2.
 - Có value = 9. Vậy hash(9) = 1 và hashTable[1] = 9.
 - Khi đó, muốn biết 9 nằm ở đâu trong hashTable, ta chỉ cần lấy hashTable[hash(9)], do đó cũng không cần duyệt.
- VD2:** Dùng hashTable để lưu trữ tên các công ty: hash(value) = value[0] – ‘a’ (lấy chữ cái đầu tiên). Có value = “bitis” thì hash(value) = 1 do ‘b’ – ‘a’ = 1.

c. Hiện tượng đụng độ

- Trong trường hợp lý tưởng nhất, nếu có thể xây dựng hàm hash(value) sao cho với mỗi value khác nhau thông qua hash(value) sẽ luôn cho ra một key khác nhau và đạt đến độ phức tạp $O(1)$. Tuy nhiên, không phải lúc nào cũng có thể xây dựng một hàm băm lý tưởng như thế. Hiện tượng đụng độ là hiện tượng với các value khác nhau, qua hash(value) đưa ra 1 key giống nhau.
- VD** như VD2, giả sử như value1 = “google” và value2 = “grab”, hash(value1) = hash(value2) = 6.
- Cách xử lý đơn giản nhất là tạo một vector ở mỗi key, và ta lần lượt đưa các value cùng key vào vector đó.

② ỨNG DỤNG KỸ THUẬT BĂM TRONG XỬ LÝ XÂU

CHƯƠNG 3 : ĐẠI SỐ TỔ HỢP

§1 : LÝ THUYẾT ĐẠI SỐ TỔ HỢP

Ý nghĩa: Tính toán nhanh số cách để tạo thành tập hợp mà không cần phải kiểm tra từng trường hợp.

① QUY TẮC CỘNG, QUY TẮC NHÂN

a. Quy tắc cộng

Để hoàn thành công việc X thì có nhiều phương án. Chỉ cần hoàn thành một phương án trong số đó là hoàn thành công việc. Vậy theo quy tắc cộng, số cách có thể làm để thỏa công việc X là tổng các cách ở mỗi phương án.

VD: Hôm nay là sinh nhật Tiên, Tân muốn tặng Tiên bánh kem. Tân có thể ra chợ mua, vào tiệm, tự làm hoặc nhờ bạn làm dùm. Vậy Tân có bao nhiêu cách để hoàn thành nhiệm vụ?

- Tân có thể:
 - Ra chợ mua: 1 cách.
 - Vào tiệm mua: 1 cách.
 - Tự làm: 1 cách.
 - Nhờ bạn làm dùm: 1 cách.
- Mà chỉ cần thỏa 1 trong 4 phương án trên là được. Vậy Tân có $1 + 1 + 1 + 1 = 4$ cách để hoàn thành nhiệm vụ.

b. Quy tắc nhân

Để hoàn thành công việc X thì có nhiều bước. Mọi bước đều phải được hoàn thành thì công việc X mới được hoàn thành. Vậy theo quy tắc nhân, số cách có thể làm để thỏa công việc X là tích các cách ở mỗi bước.

VD: Từ tập hợp $A = \{0; 1; 2; 3; 4; 5; 6; 7; 8; 9\}$, có mấy cách sinh ra các số tự nhiên có 4 chữ số khác nhau?

- Gọi dạng của số tự nhiên là : $\overline{a_1a_2a_3a_4a_5}$.
 - a_1 : Có thể chọn bất kỳ số nào từ A, trừ số 0 : 9 cách.
 - a_2 : Có thể chọn bất kỳ số nào từ A, trừ a_1 : 9 cách.
 - a_3 : Có thể chọn bất kỳ số nào từ A, trừ a_1 và a_2 : 8 cách.
 - a_4 : Có thể chọn bất kỳ số nào từ A, trừ a_1, a_2 và a_3 : 7 cách.
- Vậy có $9 \cdot 9 \cdot 8 \cdot 7 = 4536$ cách.

② HOÁN VỊ, CHỈNH HỢP, TỔ HỢP

Vấn đề: Cho một tập hợp A có n phần tử. Hỏi có mấy cách tạo ra tập hợp bằng cách chọn các phần tử của tập A?

Số cách	Ý nghĩa	Công thức ($k \leq n$)
Hoán vị của n (n giai thừa)	Số cách để sắp xếp lại vị trí các phần tử của tập A (có n phần tử).	$n! = \prod_{i=1}^n (i) = A_n^n$
Tổ hợp chập k của n	Số cách chọn k phần tử của tập A (có n phần tử), <u>không phân biệt thứ tự</u> và mỗi phần tử <u>chỉ chọn một lần</u> (không lặp).	$C_n^k = nCk = \frac{n!}{k!(n-k)!}$
Tổ hợp lặp chập k của n	Số cách chọn k phần tử của tập A (có n phần tử), <u>không phân biệt thứ tự</u> và mỗi phần tử <u>có thể chọn nhiều lần</u> (có lặp).	$K_n^k = C_{n+k-1}^k$
Chỉnh hợp chập k của n	Số cách chọn k phần tử của tập A (có n phần tử), phân biệt thứ tự và mỗi phần tử <u>chỉ chọn một lần</u> (không lặp).	$A_n^k = nPk = \frac{n!}{(n-k)!}$

Chỉnh hợp lặp chập k của n	Số cách chọn k phần tử của tập A (có n phần tử), phân biệt thứ tự và mỗi phần tử <u>có thể chọn nhiều lần</u> (có lặp).	$F_n^k = n^k$
----------------------------	---	---------------

VD:

- VD1: Cho số tự nhiên $x = 3526$ có 4 chữ số, hỏi có mấy số được tạo ra bằng cách thay đổi vị trí của các chữ số của x ?
 - Đáp án: Có $4!$ cách.
- VD2: Cho số tự nhiên $x = 3526$ có 4 chữ số, hỏi có mấy số có 2 chữ số được tạo ra từ x ?
 - Bản chất: Chọn 2 chữ số từ x , mỗi chữ số chỉ chọn 1 lần $\Rightarrow$ Vậy đáp án là C_4^2 .

§2 : ĐỆ QUY – QUAY LUI – NHÁNH – CẬN

- **Recursion:** Đệ quy.
- **Backtracking:** Quay lui.
- **Branch and bound:** Nhánh và cận.

① ĐỆ QUY – QUAY LUI

a. Giới thiệu

- Đệ quy là một hàm gọi chính nó, được xây dựng từ các bài toán con. Ví dụ: $n! = n(n-1)!$, vậy $(n-1)!$ là bài toán con.
- Quay lui là việc trở lại lần đệ quy trước đó. Đệ quy – quay lui giúp tìm ra mọi cấu hình có thể có. Do mang tính vét cạn nên tốn nhiều thời gian và bộ nhớ, nhưng là phương pháp giải nhiều bài toán.
- **Ưu điểm:** Tương đối dễ thiết lập, dễ hiểu.
- **Nhược điểm:** Dễ bị TLE.

Đệ quy: Các bước chung:

- Tạo điều kiện dừng đệ quy.
- Gọi đệ quy bài toán con.

Đệ quy – quay lui: Các bước chung:

- Tạo điều kiện dừng đệ quy.
- Bổ sung cấu hình:
 - Kiểm tra cấu hình: Nếu đúng, gọi đệ quy với cấu hình vừa kiểm tra.
- Trả về cấu hình trước khi được bổ sung (quay lui).

b. Một vài ứng dụng sinh số kinh điển

Ứng dụng 1: Sinh chỉnh hợp có lặp

INPUT	OUTPUT
n	Các chỉnh hợp lặp có số phần tử từ 1 đến n của A.
A[i]	
2	4
4 5	5
	4 4
	4 5
	5 4
	5 5

- **Ý tưởng:**
 - Nhận xét: Mỗi cấu hình ứng với một chỉnh hợp sẽ được sinh ra.
 - Gọi k là số phần tử của cấu hình. Thấy k tăng dần từ 1 đến n nên ta sẽ dùng vòng lặp for.

- Gọi num là vector chứa các phần tử trong cấu hình. Gọi Try (num) là hàm sinh số: Duyệt mảng A, mỗi lần truy cập đến một phần tử trong A:
 - Đưa thêm A[i] vào num, xét:
 - Nếu độ dài num = k thì đã sinh ra được một chỉnh hợp, hoàn tất cấu hình. Xuất A.
 - Nếu không, tiếp tục gọi đệ quy Try (num) để tiếp tục hoàn chỉnh cấu hình.
 - Lấy A[i] ra khỏi num để sinh ra một cấu hình mới.

```

1. #include <bits/Pdc++.h>
2. using namespace Pd;
3.
4. vector <int> A;
5. int n, k;
6.
7. void write (vector <int> num){
8.     for (int i : num) cout << i << ' ';
9.     cout << '\n';
10. }
11.
12. void Try (vector <int> num){
13.     for (int i = 0; i < n; ++i){
14.         num.push_back(A[i]); //Hoàn thiện cấu hình
15.         if (num.size() == k) write(num); //Hoàn tất cấu hình, xuất ra
16.         else Try(num); //Đệ quy
17.         num.pop_back(); //Quay lui
18.     }
19. }
20.
21. int main() {
22.     cin >> n;
23.     for (int i = 1; i <= n; ++i){
24.         int tmp; cin >> tmp;
25.         A.push_back(tmp);
26.     }
27.
28.     for (k = 1; k <= n; ++k) {
29.         vector <int> num;
30.         Try(num);
31.     }
32.
33.     return 0;
34. }

```

Ứng dụng 2: Sinh tổ hợp có lặp

INPUT	OUTPUT
n	Các tổ hợp có số phần tử từ 1 đến n của A.
A[i]	
2	4
4 5	5
	4 4
	4 5
	5 5

- **Ý tưởng:** Cũng tương tự bài trên, nhưng khác ở chỗ ta không sinh các cấu hình trùng nhau. VD {4;5} và {5;4} đều là một tổ hợp, ta chỉ chọn in {4;5} (theo INPUT). Có nghĩa là ta chỉ tiếp tục bổ sung những phần tử chưa từng được đưa vào num (trừ phần tử mới được đưa vào để tạo nên tổ hợp lặp): Trong hàm void, thêm tham số start là vị trí bắt đầu duyệt đến n của A, ban đầu, start = 0.

```

1. void Try (vector <int> num, int start){
2.     for (int i = start; i < n; ++i){
3.         num.push_back(A[i]);
4.         if (num.size() == k) write(num);
5.         else Try(num, i);
6.         num.pop_back();
7.     }
8. }

```

Ứng dụng 3: Sinh hoán vị

INPUT	OUTPUT
n A[i]	Các hoán vị của A.
3 4 5 6	4 5 6 4 6 5 5 4 6 5 6 4 6 4 5 6 5 4

- Ý tưởng:

- Do bản chất của hoán vị là chỉnh hợp không lặp, ta dựa trên ứng dụng 1 để giải.
- Hoán vị là mỗi phần tử trong A chỉ chọn một lần. Ý tưởng của ta là dùng một mảng đánh dấu. Nếu đưa A[i] vào num thì check[i] = 1, nếu lấy ra thì check[i] = 0. Ta chỉ chọn những phần tử thứ i mà check[i] = 0, tức chưa được chọn.

```
1. int n;  
2. vector <int> A;  
3. char check[999999] = {0};  
4.  
5. void Try (vector <int> num){  
6.     for (int i = 0; i < n; ++i){  
7.         if (check[i] == 1) continue;  
8.  
9.         num.push_back(A[i]);  
10.        check[i] = 1;  
11.  
12.        if (num.size() == n) write(num);  
13.        else Try(num);  
14.  
15.        num.pop_back();  
16.        check[i] = 0;  
17.    }  
18. }
```

Từ 3 ứng dụng trên, ta có thể suy ra để giải các bài tập khác nhau về sinh số, như chỉnh hợp không lặp, hoán vị không lặp... Ngoài sinh số, đệ quy – quay lui còn có thể giải được nhiều bài toán khác.

2 NHÁNH – CẬN

Nhánh – cận là việc đánh giá chất lượng cấu hình được tạo ra, hay nói cách khác là thêm điều kiện để không phải đệ quy cấu hình đó nữa, giảm thời gian và không gian.

§3 : QUY HOẠCH ĐỘNG

- Dynamic programming: Quy hoạch động.*

0 TỔNG QUAN

Giới thiệu:

- Có thể coi quy hoạch động là khử đệ quy. Áp dụng trong bài toán có thể có các bài toán con tối ưu gối nhau: kết quả của bài toán con này làm tiền đề để thực hiện bài toán con kế tiếp và kết quả của chúng là kết quả của bài toán lớn.
- Ưu điểm:** Tốc độ tính toán nhanh hơn rất nhiều so với đệ quy – quay lui.
- Nhược điểm:**
 - Thường gây tốn bộ nhớ, tràn dữ liệu do thường phụ thuộc vào các mảng (không quan trọng lắm). Tuy nhiên, có thể tối ưu không gian.
 - Khó hiểu, khó thiết lập.

Các bước chung để giải bài toán theo quy hoạch động:

- Quy hoạch động thường hướng đến việc sử dụng một mảng 1 chiều hoặc nhiều chiều, thường gọi là dp hoặc F, duyệt hết mảng đó, coi mỗi $F[i]$ là bài toán con và xuất ra bài toán lớn cuối cùng (kết quả cần tìm) là phần tử cuối cùng trong mảng:
 - Định nghĩa bài toán con tối ưu $F[i]$. Suy ra kết quả bài toán, giá trị của bài toán con cơ sở.
 - Khảo sát giá trị, tìm các thành phần tạo ra kết quả của bài toán, tìm các trường hợp có thể có để suy ra công thức tổng quát (công thức truy hồi) của bài toán con tối ưu $F[i]$ (giả sử các bài toán con trước đó đã tối ưu).
 - Duyệt qua các bài toán con tối ưu (duyệt mảng F) rồi đưa ra kết quả.
 - Giải quyết các bước phụ (như truy vết).

1 QUY HOẠCH ĐỘNG THÔNG DỤNG

Để tránh độ phức tạp xử lý, các dãy số, chuỗi trong các bài toán dưới đây đều bắt đầu từ vị trí 1.

a. Tìm dãy con

Bài toán: Dãy con tăng dần

- Cho dãy A có n phần tử, tìm độ dài của dãy con tăng dần dài nhất trong dãy (các phần tử trong dãy con có thể không liên tiếp).

INPUT	OUTPUT	Giải thích test
n $A[i]$	Như yêu cầu.	
7 1 2 3 1 4 8 5	5	Dãy con thỏa đề là {1; 2; 3; 4; 5} có độ dài là 5.

- Ý tưởng:
 - Gọi $F[i]$ là độ dài của dãy con tăng dần dài nhất khi có i phần tử đầu tiên của dãy A $\Rightarrow$ Đáp án là $F[n]$; $F[0] = 0$ và $F[1] = 1$ (hiển nhiên).
 - Do dãy con có thể không liên tiếp nên với mỗi $F[i]$, lần lượt xét $A[j]$ với $(0 < j < i)$, nếu $A[i] > A[j]$ thì $A[i]$ và dãy con kết thúc tại $A[j]$ nằm trong một dãy con mới lớn hơn: $F[i] = F[j] + 1$. VD: $A[3] > A[2]$ nên $A[3]$ là phần tử nằm trong dãy con kết thúc tại $A[2]$, nên $F[3] = F[2] + 1$.
 - Cứ duyệt như thế sẽ sinh ra n^2 trường hợp, nhưng vì đề yêu cầu tìm dãy con dài nhất, nên ta lấy max tất cả các trường hợp ấy: $F[i] = \max(F[j] + 1) (A[j] < A[i]) (0 < j < i)$.

Bài toán: Dãy con có tổng bằng s

- Cho dãy A có n phần tử, xuất ra TRUE nếu tìm thấy dãy con có tổng các phần tử bằng s trong dãy, ngược lại là FALSE (các phần tử trong dãy con có thể không liên tiếp).

INPUT	OUTPUT	Giải thích test
n s $A[i]$	Như yêu cầu.	
4 3 2 5 3 1	TRUE	Dãy con thỏa đề có thể là {2; 1} hoặc {3}.

- Ý tưởng:
 - Gọi $F[i][j] = \text{true}$ khi với i số đầu tiên trong dãy A có thể tạo ra dãy con có tổng là j $\Rightarrow$ Đáp án là $F[n][s]$; $F[i][0] = \text{true}$ (không chọn phần tử trong A để tạo thành dãy con) $(0 \leq i \leq n)$.
 - Với mỗi $F[i][j] = \text{true}$ nếu thỏa ít nhất 1 trong 2 điều kiện sau:
 - $F[i-1][j] = \text{true}$ (Nếu dãy con có ít phần tử hơn (i - 1) mà vẫn có thể tạo thành tổng j thì thêm dãy con có nhiều phần tử hơn (i) chắc chắn có thể tạo thành tổng j).
 - $F[i-1][j-A[i]] = \text{true}$ (Chứng tỏ $A[i]$ nằm trong dãy con tạo thành tổng s ($j = A[i] + \text{<dãy con>}$, mà khả năng xuất hiện <dãy con> lại là $F[i-1][j-A[i]]$)).

b. Tìm xâu

Bài toán: Xâu con chung dài nhất

- Có 2 chuỗi A và B chỉ gồm các chữ cái in thường, tìm 1 chuỗi chung dài nhất của chúng (có thể không liên tiếp nhưng phải theo thứ tự). Kết quả có thể rỗng.

INPUT	OUTPUT	Giải thích test
A B	Như yêu cầu.	
axyb abyxb	axb	Lưu ý: axb hoặc ayb đều đúng.

Ý tưởng phương án sơ khai:

- Gọi IA là độ dài chuỗi A, và IB là độ dài chuỗi B. Gọi $F_{[i][j]}$ là chuỗi chung dài nhất khi có i ký tự đầu của A và j ký tự đầu của B $\Rightarrow F_{[IA][IB]}$ là đáp án, $F_{[i][0]} = F_{[0][j]} = ""$ (Do không có ký tự sẵn có nào từ chuỗi A hoặc B).
- Ở mỗi $F_{[i][j]}$ có 2 trường hợp:
 - $A[i] = B[j] = x \rightarrow$ Bổ sung x vào chuỗi chung $\Rightarrow F_{[i][j]} = F_{[i-1][j-1]} + x$;
 - $A[i] \neq B[j] \rightarrow$ Có 2 trường hợp con sau:
 - $F_{[i][j]} = F_{[i-1][j]}$.
 - $F_{[i][j]} = F_{[i][j-1]}$.
 - Do đề yêu cầu tìm chuỗi chung dài nhất, ta lấy:
 - $F_{[i][j]} = (1) \text{ if } (1).size() > (2).size()$.
 - $F_{[i][j]} = (2) \text{ if } (2).size() > (1).size()$.
 - Lưu ý: Do 1 INPUT có thể cho nhiều OUTPUT, nên khi $(1).size() = (2).size()$, $F[i][j]$ mang giá trị (1) hay (2) đều được.

Ý tưởng phương án truy vết:

- Thay vì gọi $F_{[i][j]}$ là chuỗi chung dài nhất khi có i ký tự đầu của A và j ký tự đầu của B, ta gọi $F_{[i][j]}$ là độ dài lớn nhất của chuỗi chung khi có i ký tự đầu của A và j ký tự đầu của B. Vậy, $F_{[IA][IB]}$ không còn là chuỗi chung dài nhất nữa mà chỉ là độ dài của chuỗi chung dài nhất.
- Theo đó, có công thức truy hồi như sau:
 - $A[i] = B[j] \Rightarrow F_{[i][j]} = F_{[i-1][j-1]} + 1$.
 - $A[i] \neq B[j] \Rightarrow F_{[i][j]} = \max(F_{[i-1][j]}, F_{[i][j-1]})$.
- Sau khi đã hoàn thành duyệt F , ta truy vết để có được chuỗi chung có độ dài $F_{[IA][IB]}$:
 - Gọi res là chuỗi chung dài nhất cần được xuất ra.
 - Gọi $i = IA, j = IB$. Lần lượt lặp cho đến khi i hoặc $j = 0$: Nếu:
 - $A[i] = B[j] = x \Rightarrow x$ nằm trong chuỗi chung $\Rightarrow$
 - $res += x$.
 - $--i, --j$.
 - $A[i] \neq B[j] \Rightarrow$ Không tìm thấy ký tự chung $\Rightarrow$ Bỏ qua quá trình nạp ký tự chung vào res :
 - $F_{[i-1][j]} > F_{[i][j-1]} : --i$.
 - $F_{[i][j-1]} > F_{[i-1][j]} : --j$.
 - Do bản chất của truy vết là đi ngược, cho nên chuỗi res sau khi được sinh ra bị ngược so với thứ tự vốn có. Ta đảo ngược chuỗi res trước khi xuất ra.

c. Chọn phần tử trong dãy được cho để có phương án tối ưu

Bài toán: Atcoder Educational DP Contest A – Frog 1

- https://oj.vnoi.info/problem/atcoder_dp_a
- Có N tầng đá được đánh số từ 1 đến N , tầng đá thứ i có chiều cao là $H_{[i]}$. Con ếch đang ở tầng đá thứ nhất và muốn nhảy sang tầng đá thứ N theo quy tắc sau: Ở tầng đá thứ i , nó chỉ có thể nhảy sang tầng đá thứ $j = i + 1$ hoặc $j = i + 2$ và nó sẽ tốn $|H_{[j]} - H_{[i]}|$ năng lượng. Hãy tìm mức năng lượng tiêu thụ thấp nhất có thể.

INPUT	OUTPUT	Giải thích test
$n (n \geq 2)$ $H_{[i]}$	Như yêu cầu	
6 30 10 60 10 60 50	40	Con ếch có thể nhảy đến hòn đá thứ 3, 5 và 7. Tổng năng lượng là: $ 60 - 30 + 60 - 60 + 50 - 60 = 40$.

• **Ý tưởng:**

- Gọi $F[i]$ là tổng mức năng lượng nhỏ nhất để nhảy từ cột 1 đến cột $i \Rightarrow$ Kết quả bài toán là $F[n]$; $F[1] = 0$ (do đứng yên tại chỗ), $F[2] = |H[2] - H[1]|$ (chỉ có 1 lựa chọn này). Lưu ý không định nghĩa $F[0]$ do $F[0]$ vô nghĩa (giả thiết con ếch luôn đứng sẵn ở tảng đá đầu tiên nên luôn có ít nhất 2 hòn đá do $n \geq 2$).
- Ở mỗi bài toán con $F[i]$, có 2 trường hợp như sau:
 - Nếu trước đó nó ở cột $i - 1 \Rightarrow F[i] = F[i-1] + |H[i] - H[i-1]|$.
 - Nếu trước đó nó ở cột $i - 2 \Rightarrow F[i] = F[i-2] + |H[i] - H[i-2]|$.
- Vậy, $F[i]$ mang 2 giá trị. Nhưng do ta đã định nghĩa $F[i]$ là nhỏ nhất nên ta lấy min giữa các giá trị. Vậy, công thức là: $F[i] = \min((1), (2))$.

 Bài toán: Phân quà

- Vào dịp cuối năm, một công ty quyết định thưởng quà cho nhân viên. Có n phần quà được đánh số từ 1 đến n và phần quà thứ i có giá trị là $A[i]$ với quy tắc không được chọn ba món liên tiếp nhau. Hãy chọn quà sao cho thỏa điều kiện mà vẫn lấy được giá trị cao nhất, xuất ra giá trị ấy.

INPUT	OUTPUT
n $A[i]$	Như yêu cầu.
5 6 9 1 3 5	23

• **Ý tưởng:**

- Gọi $F[i]$ là cách chọn tối ưu nhất có thể khi có i món quà đầu tiên $\Rightarrow$ Đáp án là $F[n]$; $F[0] = 0$ (do không chọn món quà nào), $F[1] = A[1]$ (chỉ có thể chọn $A[1]$), $F[2] = A[1] + A[2]$ (cách chọn tối ưu nhất).
- Ở mỗi $F[i]$, xét liên tiếp $A[i-2]$, $A[i-1]$ và $A[i]$. Với điều kiện không được lấy 3 món liên tiếp, vậy muốn được tối ưu, ta phải lấy 2 món. 2 món đó là kết quả khi xét liên tiếp 1 đoạn như trên, để thỏa mãn định nghĩa về $F[i]$, ta cộng thêm kết quả của $F[j]$ ($j < i$). Vậy có 3 trường hợp sau:
 1. Chỉ lấy $A[i-1]$ và $A[i] \Rightarrow$ Bỏ $A[i-2] \Rightarrow F[i] = A[i-1] + A[i] + F[i-3]$.
 2. Chỉ lấy $A[i-2]$ và $A[i] \Rightarrow$ Bỏ $A[i-1] \Rightarrow F[i] = A[i] + F[i-2]$.
 3. Chỉ lấy $A[i-2]$ và $A[i-1] \Rightarrow$ Bỏ $A[i] \Rightarrow F[i] = F[i-1]$.
- Để có được đáp án tối ưu, ta lấy max: $F[i] = \max((1), (2), (3))$.

 Bài toán: Bài toán cái túi

- https://oj.vnoi.info/problem/atcoder_dp_d

- Có một tên trộm đột nhập vào một ngôi nhà có N đồ vật, đồ vật thứ i có trọng lượng $W[i]$ và có giá trị $V[i]$. Cái túi của tên trộm chỉ có thể chứa tối đa trọng lượng M . Tìm tổng giá trị lớn nhất mà tên trộm có thể lấy mà không vượt quá tải trọng M .

INPUT	OUTPUT	Giải thích test
N M $W[i]$ $V[i]$	Như yêu cầu.	
6 10 5 4 9 10 2 5 4 9 1 1 10 5	14	Chọn món đầu tiên ($\{5; 4\}$) hai món đồ áp chót ($\{4; 9\}, \{1; 1\}$). Tổng giá trị là $4 + 9 + 1 = 14$.

• **Ý tưởng:**

- Có 2 thành phần là W và $V \rightarrow$ tạo mảng 2 chiều $F \rightarrow$ gọi $F[i][j]$ là tổng giá trị cao nhất có thể có khi trong ngôi nhà có i đồ vật (từ 1 đến i) và cái túi của tên trộm có tải trọng là $j \Rightarrow$ Đáp án là $F[N][M]$. $F[0][j] = 0$ (do không có đồ vật) và $F[i][0] = 0$ (do tải trọng của túi là 0).
- Ở mỗi bài toán con $F[i][j]$ có 2 trường hợp như sau:

1. Không lấy đồ vật thứ $i \Rightarrow F_{[i][j]} = F_{[i-1][j]}$.
 2. Lấy đồ vật thứ $i \Rightarrow F_{[i][j]} = F_{[i-1][j-w[i]]} + V[i]$ ($j - W[i]$ là trọng lượng còn lại mà cái túi có thể chứa).
- Do cần lấy được nhiều tiền nhất nên ta lấy max giữa 2 trường hợp: $F_{[i][j]} = \max((1), (2))$.

d. Tìm số lượng các phương án thỏa đề

Bài toán: Atcoder Educational DP Contest H – Gird 1

- https://oj.vnoi.info/problem/atcoder_dp_h

- Cho một bản đồ A kích thước $n \times m$, $A_{[i][j]}$ hoặc = $\square$, hoặc = $\blacksquare$. Có một robot ở vị trí $A_{[1][1]}$ muốn đến vị trí $A_{[n][m]}$, hỏi có mấy đường đi khả dĩ, biết robot chỉ có thể đi sang phải, xuống dưới và chỉ đi trên ô $\square$. (lưu ý số lượng thỏa mãn yêu cầu có thể rất lớn nên ta chỉ lấy kết quả là số đường đi sau khi chia dư cho $10^9 + 7$) (Giả thiết $A_{[1][1]} = \square$).

INPUT	OUTPUT	Giải thích test
n m $A_{[i][j]}$	Như yêu cầu.	
3 4 ...# .#..	3	Có 3 đường đi thỏa mãn đề như sau: 

• Ý tưởng:

- Gọi $F_{[i][j]}$ là số lượng đường đi từ $A_{[0][0]}$ đến $F_{[i][j]}$ chia dư cho $10^9 + 7$ trên bản đồ con có kích thước $i \times j \Rightarrow$ Đáp án là $F_{[n][m]}$. $F_{[1][1]} = 1$ do chỉ có 1 cách duy nhất để đến ô $A_{[1][1]}$. Tất cả các phần tử còn lại trong mảng $F = 0$ do chưa biết có cách nào để đến $A_{[i][j]}$ hay không.
- Ở mỗi $F_{[i][j]}$, có thể có 2 cách để đi đến $A_{[i][j]}$:
 - Cách 1: Từ $A_{[i][j-1]} \rightarrow A_{[i][j]}$ (trái sang phải) $\Rightarrow F_{[i][j]} += F_{[i][j-1]}$ ($F_{[i][j-1]} = \square$).
 - Cách 2: Từ $A_{[i-1][j]} \rightarrow A_{[i][j]}$ (trên xuống dưới) $\Rightarrow F_{[i][j]} += F_{[i-1][j]}$ ($F_{[i-1][j]} = \square$).
- Do đề yêu cầu tìm tất cả đường đi, nên ta cứ duyệt và cộng dồn vào theo công thức (nếu có thể) rồi chia dư mà không cần lấy max hay min gì cả.

Bài toán: Atcoder Educational DP Contest I – Coins

- https://oj.vnoi.info/problem/atcoder_dp_i

- Có n đồng xu (n lẻ). Xác suất để đồng xu thứ i ngửa là $P[i]$, và xác suất để đồng xu đó sấp là $1 - P[i]$. Tung n đồng xu đó, tính tổng xác suất các trường hợp có số đồng xu ngửa cao hơn số đồng xu sấp.

INPUT	OUTPUT	Giải thích test
n $P[i]$	Như yêu cầu.	
3 0.3 0.6 0.8	0.612	Có 4 trường hợp mà số đồng xu ngửa (N) nhiều hơn sấp (S): 1. N – N – N : $0.3 * 0.6 * 0.8 = 0.14$. 2. S – N – N : $(1 - 0.3) * 0.6 * 0.8 = 0.336$. 3. N – S – N : $0.3 * (1 - 0.6) * 0.8 = 0.096$. 4. N – N – S : $0.3 * 0.6 * (1 - 0.8) = 0.036$. Tổng 4 trường hợp trên là 0.612.

• Ý tưởng:

- Gọi $F_{[i][j]}$ là xác suất để có j mặt ngửa trong số i đồng xu $\Rightarrow \sum_{j=\frac{n}{2}+1}^n F_{[n][j]}$ là đáp án của bài toán. $F_{[0][0]} = 1$ (do khi có 0 đồng xu thì dĩ nhiên cũng có 0 mặt sấp, đây là biến cố hiển nhiên đúng nên xác suất của nó là 1), $F_{[i][j]} = 0$ ($j > i$) (do chỉ có i đồng xu mà lại yêu cầu nhiều hơn i mặt ngửa là điều vô lý, không thể xảy ra nên xác suất là 0).
- Với mỗi $F_{[i][j]}$, có 3 trường hợp đặc biệt sau:
 - Có i đồng xu và 0 mặt ngửa $\rightarrow$ Toàn bộ đều sấp $\Rightarrow F_{[i][0]} = (1 - P[i])F_{[i-1][0]}$.
 - Có i đồng xu và i mặt ngửa $\rightarrow$ Toàn bộ đều ngửa $\Rightarrow F_{[i][i]} = P[i]F_{[i-1][i-1]}$ ($i = j$).
 - Có i đồng xu và ít hơn i mặt ngửa, có 2 trường hợp con như sau:

- Đồng xu thứ i ngửa $\rightarrow$ Cần lấy thêm xác suất của $i - 1$ đồng xu còn lại, trong số đó chỉ cần $j - 1$ mặt ngửa do đồng xu thứ i đã ngửa sẵn $\Rightarrow F_{[i][j]} += P_{[i]}F_{[i-1][j-1]}$ ($1 \leq j < i$).
- Đồng xu thứ i sấp $\rightarrow$ Cần lấy thêm xác suất của $i - 1$ đồng xu còn lại, trong số đó cần j mặt ngửa do đồng xu thứ i không ngửa $\Rightarrow F_{[i][j]} += (1 - P_{[i]})F_{[i-1][j]}$ ($1 \leq j < i$).
- Lấy tổng 2 trường hợp trên, tính ra được xác suất của $F_{[i][j]}$ khi ($1 \leq j < i$).

2 QUY HOẠCH ĐỘNG CẤU HÌNH

Vấn đề: Cho vị trí k , tìm cấu hình của dãy ở vị trí k ấy.

3 QUY HOẠCH ĐỘNG CHỮ SỐ (DIGIT)

(Hướng dẫn trong phần này coi chuỗi số bắt đầu từ vị trí 0).

Vấn đề: Đếm số lượng các số trong phạm vi $[begin, end]$ thỏa mãn điều kiện liên quan đến các chữ số trong số đó.

Phương pháp: Chia end thành các khoảng nhỏ hơn để tính toán, gồm 2 phần:

• Phần 1:

- Gọi $F_{[i][...]}$ là số lượng các số thỏa đề từ 0 đến $9 \cdot \frac{10^i - 1}{9}$, hay nói cách khác là tổng số lượng các số có từ 0 đến i chữ số.
- Nhận xét: Các số nguyên có i chữ số được tạo thành bằng việc lấy các số nguyên có $i-1$ chữ số lần lượt ghép với các số nguyên x từ 0 đến 9.
- Từ nhận xét, ta duyệt $i, j, \dots$ Duyệt x từ 0 đến 9.
- **Lưu ý:**
 - Số 0 là trường hợp đặc biệt. Số 0 không thể ghép với số x nào để tạo thành số khác với số đã tạo, nên khi duyệt từ $F_{[1][...]}$, ta không tính số 0.
 - Sau này, số 0 cũng tham gia việc tạo thành số nên ta kiểm tra số 0 có thỏa đề không và $++ F_{[i][...]}$ sau khi duyệt bảng F .

• Phần 2:

- Tính số lượng số cùng chữ số bằng những phân đoạn từ $[l; r]$, mỗi l hoặc r có dạng $\overline{ab}$ với $a \geq 1$, b cố gắng có dạng $9 \cdot \frac{10^k - 1}{9}$. VD với $n = 3248$, $n = 30023$ và $n = 3100$ (Gọi sa là tổng các chữ số của a).

n = 3248					n = 30023					n = 3100							
	l		r		sa		l		r		sa		l		r		sa
	a	b	a	b			a	b	a	b			a	b			
3	1	000	1	999	1	3	1	0000	1	9999	1	3	1	000	1	999	1
	2	000	2	999	2		2	0000	2	9999	2		2	000	2	999	2
	3	000	3	999			3	0000	3	9999			3	000	3	999	
	30	00	30	99	3		30	000	30	999	3		30	00	30	99	3
2	31	00	31	99	4	2	300	00	300	99		1	31	00	31	99	
	32	00	32	99			3000	0	3000	9	3		310	0	310	9	
	320	0	320	9	5		3001	0	3001	9	4		0	310	0	310	0
4	321	0	321	9	6	3	3002	0	3002	9		3	321	0	321	9	
	322	0	322	9	7		3002	0	3002	9			322	0	322	9	
	323	0	323	9	8		3002	0	3002	3	5		323	0	323	9	
	324	0	324	9									324	0	324	9	
8	324	0	324	8	9												

○ Trong đó:

- Ô trắng: Những đoạn theo quy luật ổn định.
- Ô cam: Những đoạn không hợp lệ do $r > n$.

- Ô xanh: Những đoạn được đưa ra để khắc phục ô đỏ.
- Ô vàng: Đoạn đặc biệt không theo quy luật trên.

○ **Nhận xét:**

- Hậu tố b của l luôn là 0, còn hậu tố b của r luôn có dạng có dạng $9 \cdot \frac{10^k - 1}{9}$ (trừ trường hợp ô vàng).

Cho nên, ta có thể tận dụng bảng F để tính vì bảng F cũng đếm các số từ 1 đến $9 \cdot \frac{10^k - 1}{9}$. Trong trường hợp đề yêu cầu tính cả số 0 nếu số 0 thỏa điều kiện, ta tính thủ công 1 trường hợp đó.

- Ở ô vàng, ta không thể dùng bảng F nhưng có thể dùng vòng lặp để tính toán thủ công vì số vòng lặp tối đa ở ô vàng là 9, rất nhỏ.

- Tính tổng kết quả đếm được trên 2 phần (nếu có) để đưa ra kết quả cuối cùng.

📌 Bài toán: Tổng chia hết

- Cho 2 số tự nhiên n và d ($n \leq 10^{10000}$). Hãy tìm số lượng các số trong phạm vi từ 1 tới n có tổng các chữ số của nó chia hết cho d (xuất ra đáp án chia dư cho $10^9 + 7$).

INPUT	OUTPUT	Giải thích test
n d	Như yêu cầu.	
30 4	6	Có 6 số thỏa mãn điều kiện là: 4; 8; 13; 17; 22; 26.

• **Ý tưởng:**

○ Nhận xét:

1. Số dư luôn bé hơn số chia.
2. Theo tính chất của đồng dư thức, nếu số $n \% d = x$, thì tổng các chữ số của $n \% d = x$.

○ Phần 1:

- Gọi $F[i][j]$ là số lượng số có i chữ số và có j là số dư khi chia cho d $\Rightarrow$ Vậy kết quả sau khi duyệt F là $F[|n|-1][d-1]$ (do nhận xét 1). Ban đầu, mọi $F[i][j]$ đều bằng 0 do ta chưa xét gì cả.
- Xét số x có 1 chữ số ($1 \leq x \leq 9$): $++F[1][x \% d]$.
- Ở mỗi $F[i][j]$ xét chữ số x ($0 \leq x \leq 9, i \geq 2$):
 - Do tính chất của đồng dư thức (nhận xét 2), để tính số lượng số có i chữ số, cần biết số lượng số có $i-1$ chữ số và chia 4 dư y, với $(x+y) \% d = 0$ (có thể có nhiều y và ta chỉ duyệt y từ 0 đến $d-1$). Tóm lại, $F[i][j] = F[i-1][y]$ với $(x+y) \% d = j$ và $0 \leq y < d$.
 - Sau khi duyệt F, vì $\sum_{i=1}^{|n|-1} F[i][0]$ chỉ tiềm cận đáp án đúng nên ta xử lý phần 2.

○ Phần 2:

4 QUY HOẠCH ĐỘNG TRẠNG THÁI (BITMASK):

• **Đặc trưng:**

- Xử lý các bài toán có nhiều lớp trạng thái. Mỗi trạng thái có thể dùng bit 1 (TRUE) và bit 0 (FALSE) để biểu diễn mỗi thành phần trong trạng thái đó. Các bit 1 và bit 0 hợp lại tạo thành dãy bit, mỗi dãy bit là một trạng thái. Nói cách khác, bitmask là kiểm tra tất cả trường hợp.
- Cần sử dụng mảng có 2^n hay $(1 \ll n)$ phần tử, tức $n \leq 30$. Đây cũng là đặc trưng của dạng bài bitmask khi input thường nhỏ.

- **Phương pháp:** Ta duyệt qua tất cả các trạng thái, ứng dụng các thao tác với bit, kết hợp với mảng F để đưa ra kết quả cuối cùng (Với $F[\text{mask}][...] = F[\text{premask}][...]$, premask là trạng thái trước đó của mask).

📌 Bài toán: TRIP

- <https://oj.vnoi.info/problem/lem3>

- Sherry muốn đi du lịch qua tất cả n đất nước bằng máy bay. Giá vé từ đất nước i đến đất nước j là C_{ij} (C_{ij} có thể khác với C_{ji} , và $C_{ii} = 0$).
- Sherry xuất phát từ đất nước bất kì, tìm lộ trình đi qua tất cả N đất nước với chi phí rẻ nhất có thể. Xuất ra giá tiền của lộ trình đó.

INPUT	OUTPUT	Giải thích test
n ($5 < n < 16$)	Như yêu cầu.	

$C_{[i][j]}$																																																			
6 0 1 2 1 3 4 5 0 3 2 3 4 4 1 0 2 1 2 4 2 5 0 4 3 2 5 3 5 0 2 5 4 3 3 1 0	8	Lộ trình: $2 \rightarrow 5 \rightarrow 4 \rightarrow 0 \rightarrow 1 \rightarrow 3$. Chi phí cho các giá vé được chọn được in đậm: <table><tr><th></th><th>0</th><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th></tr><tr><th>0</th><td>0</td><td>1</td><td>2</td><td>1</td><td>3</td><td>4</td></tr><tr><th>1</th><td>5</td><td>0</td><td>3</td><td>2</td><td>3</td><td>4</td></tr><tr><th>2</th><td>4</td><td>1</td><td>0</td><td>2</td><td>1</td><td>2</td></tr><tr><th>3</th><td>4</td><td>2</td><td>5</td><td>0</td><td>4</td><td>3</td></tr><tr><th>4</th><td>2</td><td>5</td><td>3</td><td>5</td><td>0</td><td>2</td></tr><tr><th>5</th><td>5</td><td>4</td><td>3</td><td>3</td><td>1</td><td>0</td></tr></table>		0	1	2	3	4	5	0	0	1	2	1	3	4	1	5	0	3	2	3	4	2	4	1	0	2	1	2	3	4	2	5	0	4	3	4	2	5	3	5	0	2	5	5	4	3	3	1	0
	0	1	2	3	4	5																																													
0	0	1	2	1	3	4																																													
1	5	0	3	2	3	4																																													
2	4	1	0	2	1	2																																													
3	4	2	5	0	4	3																																													
4	2	5	3	5	0	2																																													
5	5	4	3	3	1	0																																													

• Ý tưởng:

- Gọi $F_{[mask][i]}$ là lộ phí thấp nhất khi đang ở trạng thái mask và đang thăm đến đất nước thứ i .
- Gọi bit 1 tương ứng với việc thành phố được đi qua, ngược lại là bit 0. Vậy ta sẽ có n bit tương ứng với trạng thái, bắt đầu từ $\overline{000} \dots ((1 \ll 0) - 1)$ đến $\overline{111} \dots ((1 \ll n) - 1)$. Tổng cộng, dãy bit có $(1 \ll n)$ bit.
- Giả sử ta có trạng thái 110010 , tương ứng với $F_{[50][1]}$. Để có $F_{[50][1]}$, ta cần trạng thái chưa xuất hiện bit 1 ở vị trí 1, hay chính là trạng thái 110000 , hay $F_{[48][...]}$. Nhưng ta chưa biết trong trạng thái này thì ta đã thăm đến đất nước nào, những trường hợp có thể có là:
 - Đã thăm nước thứ 5, hay $F_{[48][5]}$.
 - Đã thăm đến nước thứ 4, hay $F_{[48][4]}$.
- Có được các trường hợp, ta cộng các trường hợp đó với chi phí là và xét min là xong: $F_{[50][1]} = \min($
 - $F_{[48][5]} + C_{[5][1]}$,
 - $F_{[48][4]} + C_{[4][1]}$.
- Tổng quát hóa, $F_{[mask][i]} = F_{[mask - (1 \ll i)][j]} + C_{[j][i]}$. mask - $(1 \ll i)$ chính là premask (ta không nhất thiết phải tính toán premask bằng phép dịch mà duyệt lại mask với điều kiện $j \neq i$ cũng được).
- Lưu ý:
 - Ở trạng thái mask = 0, $F_{[mask][i]} = 0$.
 - Nếu chỉ có một bit 1 trong mask, ta không cần tìm premask vì đó là quốc gia được thăm đầu tiên. Ta dùng hàm `__builtin_popcount(mask)` để đếm số lượng bit 1 trong dãy bit.
- Sau khi duyệt xong mask, ở trạng thái $(1 \ll n) - 1$ có nhiều trường hợp ta đã hoàn thành trạng thái này nhờ việc đến thăm đất nước nào, nên ta duyệt qua tất cả $F_{[(1 \ll n) - 1][i]}$ và tìm min.

```

1. #include <bits/stdc++.h>
2. #define bit(i, a) ((a >> i) & 1) //Bit thứ i của a
3. using namespace std;
4.
5. int C[15][15];
6. int F[1 << 16][15];
7.
8. int main(){
9.     int n;
10.    cin >> n;
11.    for(int i = 0; i < n; ++i)
12.        for(int j = 0; j < n; ++j)
13.            cin >> C[i][j];
14.
15.    for(int mask = 1; mask < (1 << n); ++mask){
16.        if(__builtin_popcount(mask) != 1)
17.            for(int i = 0; i < n; ++i)
18.                if(bit(i, mask)){
19.                    F[mask][i] = 1e9; //Cực tiểu hoá
20.                    int premask = mask - (1 << i);
21.                    for(int j = 0; j < n; ++j)
22.                        if(bit(j, premask))
23.                            F[mask][i] = min(F[mask][i], F[premask][j] + C[j][i]);
24.                }
25.    }

```

```

26.     int ans = 1e9;
27.     for(int i = 0; i < n; ++i)
28.         ans = min(ans, F[(1 << n) - 1][i]);
29.     cout << ans;
30.     return 0;
31. }

```

Bài toán: The problem for kid

- <https://oj.vnoi.info/problem/maugiao>
- Người Tày có một câu đố dân gian như sau: Một gia đình kia có n người con gái cần được gả cho n người con trai nhưng không biết phải gả như thế nào nên đã hỏi một nhà hiền triết. Nhà hiền triết ấy đã tạo ra một bảng vuông $n \times n$ và nói rằng: “Nếu cô gái thứ i kết hôn với chàng trai thứ j thì sẽ tạo ra lượng tài sản là $C_{[i][j]}$ đến hết đời”. Nhưng bản thân nhà hiền triết cũng không biết phải sắp xếp hôn sự như thế nào để có được lượng tài sản lớn nhất.
- Hãy tính toán tổng tài sản lớn nhất mà gia đình có thể có được và số cách để đạt được số tài sản ấy.

INPUT	OUTPUT
n ($1 \leq n \leq 20$) $C_{[i][j]}$	Như yêu cầu.
4 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16	34 24

- Ý tưởng:
 - Gọi k là số bit 1 trong mask. Giả sử có mask = 110110, khi ấy, cô gái thứ $k-1$ (nếu mảng C bắt đầu từ 0 thì phải -1) đã được ghép với một trong bốn chàng trai tương ứng với bốn bit 1 ở các vị trí 1, 2, 4, 5.
 - Gọi $F_{[mask]}$ là số tiền có được khi cô gái thứ $k-1$ đã được ghép.
 - Tuy nhiên, ta không biết cụ thể cô gái thứ $k-1$ đã ghép với chàng trai nào, nên ta lần lượt giả sử cô gái thứ $k-1$ đã kết hôn với chàng trai thứ i (với i lần lượt là vị trí các bit 1 trong mask). Mà nếu cô gái thứ $k-1$ kết hôn với chàng trai thứ i , ta cần lấy premask là mask nhưng không có bit 1 ở vị trí i (hay $premask = mask - (1 << i)$), cộng với $C_{[k][i]}$. Tóm lại, $F_{[mask]} = \max(F_{[mask]}, F_{[premask]} + C_{[k][i]})$.
 - VD: Có mask = 1101**1**0, giả sử cô gái thứ 3 kết hôn với chàng trai thứ 1, tức ta cần xem $premask = 1101$ **0**0, và tương tự đối với các i khác.
 - Gọi $cnt_{[mask]}$ là số cách để có được $F_{[mask]}$:
 - $cnt_{[0]} = 1$.
 - Nếu $F_{[mask]} < F_{[premask]} + C_{[k][i]}$ thì $F_{[mask]} = F_{[premask]}$ nên $cnt_{[mask]} = cnt_{[premask]}$.
 - Nếu $F_{[mask]} = F_{[premask]} + C_{[k][i]}$ thì $cnt_{[mask]} += cnt_{[premask]}$.
 - Lưu ý: cnt nên có kiểu long long vì số cách có thể rất lớn.

```

1. #include <bits/stdc++.h>
2. #define bit(i, a) ((a >> i) & 1)
3. using namespace std;
4.
5. int C[20][20];
6. int F[1 << 20];
7. long long cnt[1 << 20];
8.
9. int main(){
10.     int n; cin >> n;
11.     for(int i = 0; i < n; ++i)
12.         for(int j = 0; j < n; ++j)
13.             cin >> C[i][j];
14.
15.     cnt[0] = 1;
16.
17.     for(int mask = 1; mask < (1 << n); ++mask){

```

```

18.     int k = __builtin_popcount(mask)-1;
19.     for(int i = 0; i < n; ++i)
20.         if(bit(i, mask)){
21.             int premask = mask - (1 << i);
22.             int pref = F[premask] + C[k][i];
23.             if(pref > F[mask]){
24.                 F[mask] = pref;
25.                 cnt[mask] = cnt[premask];
26.             } else if(pref == F[mask])
27.                 cnt[mask] += cnt[premask];
28.         }
29.     }
30.     cout << F[(1<<n)-1] << ' ' << cnt[(1<<n)-1];
31.
32.     return 0;
33. }

```

Bài toán: Người chăn bò

- <https://oj.vnoi.info/problem/cowgirl>
- Một cánh đồng hình vuông có diện tích $m.n$, cứ mỗi ô có diện tích 1×1 là tạo thành 1 vị trí đứng cho 1 con bò. Người chăn bò cần xếp bò vào cánh đồng theo quy tắc sau: Không được để các con bò đứng cạnh nhau tạo thành một ô vuông 2×2 hoặc để trống 1 ô vuông 2×2 .
- Cho t test case, mỗi test case có m và n . Xác định số cách xếp bò thỏa mãn quy tắc trên cho mỗi test case (số lượng bò ≥ 0).

INPUT	OUTPUT	Giải thích test
t ($t \leq 111$) m n ($m.n \leq 30$)	Như yêu cầu.	
1 1 1	2	Có thể có 1 hoặc 0 con bò đứng vào ô 1×1 .

- Ý tưởng:
 - Gọi $F_{[i][mask]}$ là số cách thỏa đề khi có trạng thái là $mask$ và tính đến hàng thứ i .
 - Các trạng thái bắt đầu từ từ $\overline{000} \dots ((1 << 0) - 1)$ đến $\overline{111} \dots ((1 << n) - 1)$, vậy có tổng cộng $(1 << n)$ trạng thái ở mỗi hàng.
 - Duyệt các hàng ($1 \leq i < n$):
 - Duyệt các $mask$ ($0 \leq mask < (1 << n)$):
 - Duyệt các $premask$ ($0 \leq premask \leq (1 << n) - 1$) là trạng thái của hàng $i - 1$:
 - Xét điều kiện: Nếu như không có cặp 2 bit ở $mask$ giống với cặp 2 bit ở $premask$ có vị trí tương ứng thì sai điều kiện của đề.
 - Cách kiểm tra:
 - Các trường hợp không hợp lệ là khi bốn bit đó là 1111 hoặc 0000.
 - Vậy, tổng của chúng tối đa là 4. Vậy khi tổng của các bit đó chia hết cho 4 thì 2 dãy bit đó không hợp lệ.
 - Nếu đúng điều kiện, $F_{[i][mask]} += F_{[i-1][premask]}$.
 - Trường hợp cơ sở: Ở $F_{[0][mask]} = 0$, ta chỉ $++F_{[0][mask]}$ khi điều kiện đúng chứ không gọi $F_{[i-1][premask]}$.
 - Lưu ý: Giả sử $n = 1, m = 30$, khi ấy, ta gọi đến $F_{[1][1 << 30]}$ sẽ bị tràn dữ liệu. Để khắc phục, ta hoán đổi n và m vì vai trò của 2 biến này giống nhau (khi $n = 2, m = 15$, không tràn dữ liệu).
 - Cuối cùng, ta cộng tất cả $F_{[n-1][mask]}$ rồi xuất ra.

```

1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int m, n;
5.
6. //Không nên dùng #define vì tham số truyền vào là biểu thức
7. inline bool bit(int i, int a){
8.     return ((a >> i) & 1);

```

```

9.  }
10.
11. //Kiểm tra xem 2 trạng thái có
12. bool valid(int a, int b){
13.     for(int pos = 0; pos < n-1; ++pos){
14.         int tmp = bit(pos, a) + bit(pos, b) + bit(pos+1, a) + bit(pos+1, b);
15.         if(tmp % 4 == 0)
16.             return false;
17.     }
18.     return true;
19. }
20.
21. int main(){
22.     int t;
23.     cin >> t;
24.     while(t--){
25.         cin >> m >> n;
26.         if(n > m) //Tránh tràn bộ nhớ
27.             swap(n, m);
28.         int F[m][1 << n];
29.         //Xử lý đặc biệt khi i = 0 (không tồn tại i - 1)
30.         for(int mask = 0; mask < (1 << n); ++mask){
31.             F[0][mask] = 1;
32.         }
33.         for(int i = 1; i < m; ++i)
34.             for(int mask = 0; mask < (1 << n); ++mask){
35.                 F[i][mask] = 0;
36.                 for(int premask = 0; premask < (1 << n); ++premask)
37.                     if(valid(mask, premask))
38.                         F[i][mask] += F[i-1][premask];
39.             }
40.         int ans = 0;
41.         for(int mask = 0; mask < (1 << n); ++mask)
42.             ans += F[m-1][mask];
43.         cout << ans << '\n';
44.     }
45.     return 0;
46. }

```

Bài toán: Chọn ô

- <https://oj.vnoi.info/problem/qbselect>
- Cho một bảng A có $4 \times n$ ô vuông được đánh số từ 0 đến 3 theo chiều dọc và từ 1 đến $n-1$ theo chiều ngang. Mỗi ô $A_{[i][j]}$ chứa một số nguyên.
- Hãy tìm cách chọn các ô sao cho các ô đó không chung cạnh và tổng các số trên các ô đó là lớn nhất có thể (không giới hạn số lượng ô được chọn). Xuất ra tổng này.

INPUT	OUTPUT	Giải thích test
$n\ (n \leq 10^4)$ $A_{[i][j]}$	Như yêu cầu.	
3 -1 9 3 -4 5 -6 7 8 9 9 7 2	32	Các ô được chọn được tô màu xanh lá: 0 1 2 (...) 0 -1 9 3 1 -4 5 -6 2 7 8 9 3 9 7 2

- Ý tưởng:
 - Định nghĩa:
 - Gọi $F_{[i][mask]}$ là tổng lớn nhất khi tính đến cột thứ j (đếm từ 0) và có trạng thái là $mask$.

- Giải thích về trạng thái:
 - Gọi bit 1 tương trưng cho ô được chọn, bit 0 là không được chọn.
 - Như vậy, ở mỗi cột sẽ tương ứng với một dãy bit. Có nhiều dãy bit có thể có, bắt đầu từ trạng thái 0000 (0) đến trạng thái 1111 (15). Vậy suy ra có $15 - 0 + 1 = 16$ dãy bit.
 - $F_{ij}[\dots]$ có nhiệm vụ lưu lại tổng của từng trạng thái hợp lệ khi tính đến cột j.
- Kiểm tra trạng thái: Trong các trạng thái, có một vài trạng thái không thỏa đề vì các trạng thái đó có cặp bit 1 cạnh nhau. Cách giải quyết:
 - Gọi $check_{ij} = true$ nếu trạng thái i không có cặp bit cạnh nhau.
 - Duyệt mask từ 0 đến 15 (duyệt các trạng thái), nếu trong đó không có cặp bit nào cạnh nhau bằng 1 thì $check_{mask} = true$.
- Quy hoạch động: Duyệt j từ 0 đến n-1 (duyệt cột):
 - Duyệt mask từ 0 đến 15 (duyệt các trạng thái). Nếu trạng thái thứ mask hợp lệ hay $check_{mask} == true$:
 - Tính tổng: Duyệt i từ 0 đến 4 (duyệt hàng): Nếu bit thứ i của trạng thái mask là bit 1 thì $sum += A_{ij}$.
 - Gọi pmask là trạng thái của của cột j - 1. Duyệt pmask từ 0 đến 15: Nếu trạng thái pmask hợp lệ và đan xen theo bit với trạng thái mask, lúc đó:
 - $F_{ij}[\text{mask}]$ giữ nguyên.
 - $F_{ij}[\text{mask}] = F_{ij-1}[\text{pmask}] + sum$.
 - Do yêu cầu tìm tổng lớn nhất, ta lấy max cả 2.
- Kết thúc: Kết quả cuối cùng là 1 trong các trạng thái của $F_{n}[\text{mask}]$. Để tìm ra trạng thái có tổng lớn nhất, duyệt mask từ 0 đến 15: $ans = \max(ans, F_{n}[\text{mask}])$.
- Lưu ý: Trong bài toán này có 1 trường hợp đặc biệt là khi số nguyên lớn nhất trong mảng $A \leq 0$. Để đề phòng, ta kiểm tra xem số lớn nhất trong mảng có ≤ 0 không. Nếu có, xuất luôn số lớn nhất đó và không cần thực hiện các bước trên.

```

1. #include <bits/stdc++.h>
2. #define bit(i, a) ((a >> i) & 1)
3. using namespace std;
4.
5. int A[4][1e4+1];
6. int F[1e4+1][16];
7.
8. int main(){
9.     int n; cin >> n;
10.    int tmp = -1e8;
11.    for(int i = 0; i < 4; ++i)
12.        for(int j = 1; j <= n; ++j){
13.            cin >> A[i][j];
14.            tmp = max(tmp, A[i][j]);
15.        }
16.    //Trường hợp đặc biệt
17.    if(tmp <= 0){
18.        cout << tmp;
19.        return 0;
20.    }
21.    //KIỂM TRA TÍNH HỢP LỆ CỦA CÁC TRẠNG THÁI
22.    bool check[16];
23.    for(int i = 0; i < 16; ++i)
24.        check[i] = true;
25.    for(int mask = 0; mask < 16; ++mask)
26.        //Nếu tồn tại 2 cặp bit cạnh nhau bằng 1 thì không hợp lệ
27.        for(int j = 0; j < 4-1; ++j)
28.            if(bit(j, mask) && bit(j+1, mask)){
29.                check[mask] = false;
30.                break;
31.            }
32.

```

```

33. //QUY HOẠCH ĐỘNG
34. for(int j = 1; j <= n; ++j)
35.     for(int mask = 0; mask < 16; ++mask)
36.         if(check[mask]){
37.             //Tính tổng
38.             int sum = 0;
39.             for(int i = 0; i < 4; ++i)
40.                 if(bit(i, mask))
41.                     sum += A[i][j];
42.             //Liên hệ với trạng thái của cột j - 1
43.             for(int pmask = 0; pmask < 16; ++pmask)
44.                 if(check[pmask] && ((mask & pmask) == 0))
45.                     F[j][mask] = max(F[j][mask], F[j-1][pmask] + sum);
46.         }
47.
48. //KẾT THÚC
49. int ans = 0;
50. for(int mask = 0; mask < 16; ++mask)
51.     if(check[mask])
52.         ans = max(ans, F[n][mask]);
53. cout << ans;
54. return 0;
55. }

```

Bài toán: THECODE – Subset with all digits

- Có t test case. Mỗi test case có n dãy số, mọi dãy số có độ dài d. Xuất ra số lượng ít nhất dãy số mà các chữ số trong dãy số đó có tất cả chữ số từ 0 đến 9.

INPUT	OUTPUT	Giải thích test
t ($1 \leq t \leq 10^2$) n d ($1 \leq n, d \leq 10^3$) n dãy số	Như yêu cầu.	
2 4 5 01234 56789 01456 13452 4 5 11234 56789 01456 13452	2 3	Test case 1: 01234, 56789. Test case 2: 01456, 11234, 56789.

- Ý tưởng:
 - o s

Bài toán: Bóng đèn

- Hệ thống bóng đèn trang trí của nhà bạn An có 2n bóng được xếp đều thành 2 hàng ngang và được kí hiệu là 2 dãy bit (bit 0 tương ứng bóng đèn tắt, bit 1 tương ứng bóng đèn sáng). An muốn bật/tắt hệ thống bóng đèn trên để tạo thành hình ảnh đẹp mắt. Quy tắc bật/tắt như sau:
 - o Mỗi thao tác bật/tắt là đảo bit (biến bit 0 thành bit 1 hoặc bit 1 thành bit 0).
 - o Bật/tắt 2 bóng đèn trên cùng 1 cột.
 - o Bật/tắt một loạt bóng liên tiếp trên cùng 1 hàng ngang (tối đa n bóng).
- Cho 2 dãy bit như trên là trạng thái bóng đèn mà bạn An mong muốn có được. Ban đầu, từ 2 dãy đèn đều tắt, hãy đếm số lần bật/tắt ít nhất để đạt được trạng thái đó.

INPUT	OUTPUT
n (số lượng bóng đèn ở mỗi hàng) ($1 \leq n \leq 10^4$) Dãy bit thứ nhất.	Như yêu cầu.

Dãy bit thứ hai.	
20 11101101111000101010 01111101100000010100	7

- **Giải thích test:** Gồm 7 lần bật/tắt như sau (các bit thay đổi được highlight vàng và các bit không bị thay đổi có màu nhạt):

Ban đầu	00000000000000000000 00000000000000000000
Lần 1	11100000000000000000 00000000000000000000
Lần 2	11100010000000000000 00000010000000000000
Lần 3	11100010000000000000 01111101100000000000
Lần 4	11101101111000000000 01111101100000000000
Lần 5	11101101111000111110 01111101100000000000
Lần 6	11101101111000101110 01111101100000010000
Lần 7	11101101111000101010 01111101100000010100

- **Ý tưởng:**
 - sa

\$4 : MEET IN THE MIDDLE

Ý tưởng: Thay vì duyệt trên một tập khá nhỏ và đưa ra một lượng rất lớn các trường hợp có thể có (trong khi không thể dùng các phương pháp tối ưu khác như quy hoạch động) có thể cân nhắc đến phương pháp Meet in the middle với ít số bước giải hơn so với véc cạn đơn thuần.

Các bước:

- Chia tập dữ liệu thành hai nửa, tạm gọi là A và B.
- Ta duyệt và đưa ra các trường hợp từ tập con A.
- Duyệt tập con B đồng thời kết hợp đối chiếu với các trường hợp của tập A để đưa ra kết quả.

Bài toán: Tìm đường đi ngắn nhất

- **Bài toán:** Có N địa điểm nằm giữa công ty và kho (N tương đối nhỏ). Tìm độ dài đường đi ngắn nhất để người giao hàng từ kho, đi qua N địa điểm và đến công ty.
- **Ý tưởng:** Chia N thành 2 nửa A và B. Tìm đường đi ngắn nhất trong A, B rồi tìm đường đi ngắn nhất nối giữa A và B.

Bài toán: Bài toán cái túi

- Có một tên trộm đột nhập vào một ngôi nhà có N đồ vật, đồ vật thứ i có trọng lượng $W[i]$ và có giá trị $V[i]$. Cái túi của tên trộm chỉ có thể chứa tối đa trọng lượng M. Tìm tổng giá trị lớn nhất mà tên trộm có thể lấy mà không vượt quá tải trọng M.

INPUT	OUTPUT	Giải thích test
N M ($1 \leq N \leq 30; 1 \leq M \leq 10^9$) $W[i] V[i]$ ($1 \leq W[i], V[i] \leq 10^9$)	Như yêu cầu.	
6 10 5 4 9 10 2 5 4 9 1 1	14	Chọn món đầu tiên ({5; 4}) hai món đồ áp chót ({4; 9}, {1; 1}). Tổng giá trị là $4 + 9 + 1 = 14$.

• **Ý tưởng:**

- Nhận xét: Không thể áp dụng quy hoạch động do không thể tạo ra mảng có 10^9 phần tử $\Rightarrow$ Cân nhắc chuyển hướng sang Meet in the middle.
- **Tập A:** Xét các trường hợp trên 3 đồ vật đầu tiên $\{5; 4\}$, $\{9; 10\}$ và $\{2; 5\}$:
 - Chọn 1 đồ vật:
 - $\{5; 4\}$.
 - $\{9; 10\}$.
 - $\{2; 5\}$.
 - Chọn 2 đồ vật:
 - $\{5; 4\} + \{9; 10\} = \{14; 14\} \rightarrow$ loại do $W > M$.
 - $\{5; 4\} + \{2; 5\} = \{7; 9\}$.
 - $\{9; 10\} + \{2; 5\} = \{11; 15\} \rightarrow$ loại do $W > M$.
 - Chọn 3 đồ vật: $\{5; 4\} + \{9; 10\} + \{2; 5\} = \{16; 19\} \rightarrow$ loại do $W > M$.
- **Tập B:** Xét tiếp 3 đồ vật còn lại đồng thời đối chiếu:
 - Gọi các trường hợp của tập A là $\{W_a, V_a\}$ và các trường hợp của tập B là $\{W_b, V_b\}$.
 - Sau khi tìm ra các $\{W_a, V_a\}$, ta tìm thêm cặp $\{W_b, V_b\}$ sao cho $W_a + W_b$ gần bằng M nhất có thể. Ta tiến hành cộng dồn các cặp này lại, tạo thành các cặp $\{W_x, V_x\}$.
 - Đáp án cần tìm là V_x sao cho V_x lớn nhất.
- Có các trường hợp đã nhận từ tập A như sau (đã xếp theo độ tăng dần về trọng lượng): $\{2; 5\}$, $\{5; 4\}$, $\{7; 9\}$, $\{9; 10\}$.
 - Chọn 1 đồ vật:
 - $\{4; 9\} \rightarrow + \{5; 4\} = \{9; 13\}$.
 - $\{1; 1\} \rightarrow + \{9; 10\} = \{10; 11\}$.
 - $\{10; 5\} \rightarrow$ giữ nguyên do không thể cộng thêm do $W = M$.
 - Chọn 2 đồ vật:
 - $\{4; 9\} + \{1; 1\} = \{5; 10\} \rightarrow + \{5; 4\} = \{10; 14\}$.
 - $\{4; 9\} + \{10; 5\} = \{14; 24\} \rightarrow$ loại do $W > M$.
 - $\{1; 1\} + \{10; 5\} = \{11; 6\} \rightarrow$ loại do $W > M$.
 - Chọn 3 đồ vật: $\{4; 9\} + \{1; 1\} + \{10; 5\} = \{15; 29\} \rightarrow$ loại do $W > M$.
- Vậy đáp án là 14 do đây là V lớn nhất trong số các cặp được chọn (in màu cam).

§5 : THUẬT TOÁN THAM LAM

• **Greedy algorithm:** Thuật toán tham lam.

- Tham lam là sort lại mảng theo thứ tự tăng dần / giảm dần rồi dựa vào đó để đưa ra kết quả bài toán, không cần xét hết tất cả trường hợp.
- Do không xét hết tất cả trường hợp nên dễ sai. Tuy nhiên, cũng có những bài toán được thiết kế để tham lam luôn đúng.

 Bài toán: Tặng quà

- Phú ông có 2 cô con gái song sinh. Lúc đến sinh nhật họ, phú ông muốn tặng quà chúc mừng cho 2 cô gái. Vì yêu thương cô út hơn mà phú ông định tặng quà cho cô út nhiều hơn.
- Có n món quà và $A[i]$ là giá trị của món quà thứ i . Hãy tìm giá trị cô út nhận được sao cho:
 - Số quà cô út nhận được phải bằng số quà của cô chị (nếu số lượng món quà lẻ thì cô út được quyền lấy thêm 1 món quà).
 - Giá trị cô út nhận được là lớn nhất.

INPUT	OUTPUT	Giải thích test
-------	--------	-----------------

n A[i]	Như yêu cầu.	
6 2 4 1 3 3 5	12	Phú ông có thể đưa cho cô út món quà thứ 2, 4 và 6.

• **Ý tưởng:**

- Sắp xếp A theo thứ tự tăng dần. Sau đó xuất ra tổng của $\frac{n}{2}$ (làm tròn lên) món quà đầu tiên.

Bài toán: Xếp sách

- Có N quyển sách được đánh số thứ tự từ 1 đến r. Tuy nhiên, các quyển sách ấy được đặt lộn xộn không theo thứ tự gì cả. Hãy tìm số bước ít nhất để xếp các quyển sách đó lại theo thứ tự tăng dần.

INPUT	OUTPUT	Giải thích test
l r A[i]	Như yêu cầu.	
3 6 5 3 4 6	2	• Bước 1: Hoán đổi 5 và 4, được {4; 3; 5; 6}. • Bước 2: Hoán đổi 4 và 4, được {3; 4; 5; 6}.

• **Ý tưởng:**

- Gọi B là A nhưng đã được sắp xếp tăng dần.
- Duyệt đến hết dãy A:
 - Nếu $A[i] \neq B[i]$ thì hoán đổi $A[i]$ và $B[i]$ (kết quả hoán đổi lưu trên A), tăng biến đếm kết quả lên 1.
 - Lặp lại thao tác trên cho đến khi $A[i] = B[i]$.
- Minh hoạ (những số được hoán đổi ở từng bước được tô màu cam):

i =	1	2	3	4	Đáp án:
B:	3	4	5	6	
A:	5	3	4	6	0
1	3	5	4	6	1
2	3	4	5	6	2
3	3	4	5	6	2
4	3	4	5	6	2

CHƯƠNG 4 : HÌNH HỌC

§1 : MA TRẬN

- *Pointer: Con trỏ.*
- *Coordinates, grid: Tọa độ, lưới tọa độ.*

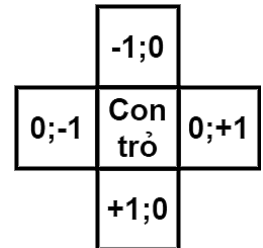
① BIỂU DIỄN MA TRẬN

Ma trận được biểu diễn bằng một mảng 2 chiều có n hàng m cột, với tọa độ của một điểm bất kỳ trong tọa độ: x, y, tương ứng với hàng, cột và biến đếm i, j.

② DI CHUYỂN CON TRỎ TRONG MA TRẬN

a. Tạo con trỏ

- Cách 0: Dùng hai biến để lưu tọa độ của con trỏ.
- Cách 2: Tạo một kiểu dữ liệu mới (pair,...) để lưu tọa độ con trỏ.



b. Di chuyển con trỏ

Ý tưởng:

- Con trỏ ứng với một vị trí (x; y) trong ma trận. Từ điểm này có thể có nhiều cách đi nhưng thường đi theo các ô kề cạnh: đi lên trên, phải, dưới, trái.
- Khi đi lên trên 0 đơn vị, giá trị x bị trừ 0. Tương tự với các hướng khác được vẽ trong hình bên.

Thực hiện:

- Tạo hai mảng dùng để lưu các đơn vị để con trỏ di chuyển.
 - Mảng X = {-1, 0, 1, 0}.
 - Mảng Y = {0, 1, 0, -1}.
- Với mỗi X[i] và Y[i] tương ứng, con trỏ có thể di chuyển theo các hướng như trên. VD: $i = 0 \Rightarrow x += -1, y += 1...$ Tương tự với $i = 1, 2, 3$.

c. Phạm vi di chuyển

Nguyên nhân: Không phải vị trí nào trong ma trận cũng đều có thể di chuyển ra 4 hướng như trên. VD nếu con trỏ đang ở ô (0; 0) thì không thể di chuyển lên trên hoặc sang trái.

Ý tưởng khắc phục:

- Dùng if giới hạn giá trị cho x và y của con trỏ: $0 \leq x \leq n, 0 \leq y \leq m$.
- Dùng đánh dấu: Tạo một ma trận dùng để đánh dấu những ô có thể đi. VD dùng mảng check kiểu bool, nếu $check[i][j] = true$ thì ô (i; j) có thể đi. Cách này làm điều kiện của if ngắn gọn hơn nhưng tốn bộ nhớ hơn. Khắc phục bằng cách gán thẳng cho mảng a một giá trị ngoại lai để đánh dấu.

Bài toán: Ma trận xoắn ốc

INPUT	OUTPUT																
Số nguyên dương N (N là số chính phương).	Một ma trận xoắn ốc theo chiều kim đồng hồ.																
16	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>12</td><td>13</td><td>14</td><td>5</td></tr><tr><td>11</td><td>16</td><td>15</td><td>6</td></tr><tr><td>10</td><td>9</td><td>8</td><td>7</td></tr></table>	1	2	3	4	12	13	14	5	11	16	15	6	10	9	8	7
1	2	3	4														
12	13	14	5														
11	16	15	6														
10	9	8	7														

- Ý tưởng:

- Khởi tạo:
 - Tạo một mảng 2 chiều tên ans dùng làm ma trận xuất ra màn hình.
 - Một biến con trỏ tên pointer.
 - Tạo một biến tên cnt dùng để làm giá trị gán vào ans. Ban đầu, ans = 1. Ở mỗi bước đi của pointer, cnt tăng 1 đơn vị.
- Nhận xét:
 - Ma trận bắt đầu từ 1 và kết thúc là N.
 - Ma trận có n hàng = n cột = $\sqrt{N}$.
 - Giới hạn lớn nhất của tọa độ (x, y) của pointer là n, và giới hạn nhỏ nhất là 1.
 - Có 4 hướng đi lần lượt là: phải, xuống, trái, lên.
 - Lưu ý: Do mảng trong C++ được đánh số thứ tự từ 0 chứ không phải từ 1 nên nếu muốn sử dụng toàn mảng thì phải xét giới hạn từ 1 đến n - 1.
- Di chuyển pointer: Lặp đi lặp lại các bước sau cho đến khi cnt đạt N:
 1. Đi sang phải: Lần lượt cộng 1 vào tọa độ y của pointer cho đến khi y = n thì chuyển sang bước 2.
 2. Đi xuống: Lần lượt cộng 1 vào tọa độ x của pointer cho đến khi x = n thì chuyển sang bước 3.
 3. Đi sang trái: Lần lượt trừ 0 vào tọa độ y của pointer cho đến khi y = 1 thì chuyển sang bước 4.
 4. Đi lên: Lần lượt trừ 0 vào tọa độ x của pointer cho đến khi x = 1 thì chuyển sang bước 1.
- Xuất mảng ans ra màn hình (cần nhắc lưu ý).

§2 : THUẬT TOÁN ĐƯỜNG QUÉT

- *Sweep line algorithm: Thuật toán đường quét.*

① CÁC KHÁI NIỆM CƠ BẢN TRÊN HỆ TRỤC TỌA ĐỘ DESCARTES

a. Vector

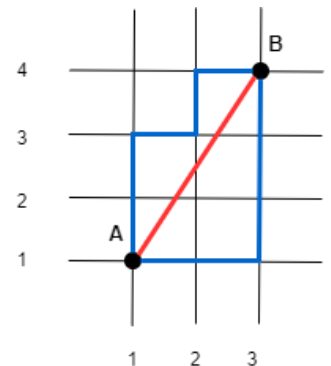
- **Vector:** Là một đoạn thẳng có hướng, với điểm gốc và điểm ngọn của hai điểm trong trục tọa độ hay ma trận.
- **Biểu thức tọa độ:** $A(x_A; y_A), B(x_B; y_B) \Rightarrow \overrightarrow{AB}(x_B - x_A; y_B - y_A)$.
- **Tích vô hướng:** $\vec{A}\vec{B} = x_A x_B + y_A y_B = |\vec{A}||\vec{B}| \cos(\vec{A}, \vec{B})$.

b. Khoảng cách giữa 2 điểm A và B

- **Khoảng cách Euclid** là độ dài của đoạn thẳng AB.

$$d_{Euclid}(A, B) = |\overrightarrow{AB}| = |\overrightarrow{BA}| = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}.$$
- **Khoảng cách Manhattan** là tổng độ dài các đường đi dọc hoặc ngang theo lưới tọa độ mà không vượt ngoài phần tạo ra bởi đoạn thẳng AB và hình chiếu của nó lên trục tung và trục hoành sao cho ngắn nhất có thể.

$$d_{Manhattan}(A, B) = |x_A - x_B| + |y_A - y_B|.$$
- Hình bên biểu diễn khoảng cách Euclid là đoạn thẳng màu đỏ có độ dài = $\sqrt{13}$. Hai đường màu xanh là hai trong nhiều đường đi Manhattan, và tất cả chúng đều có độ dài = 5.



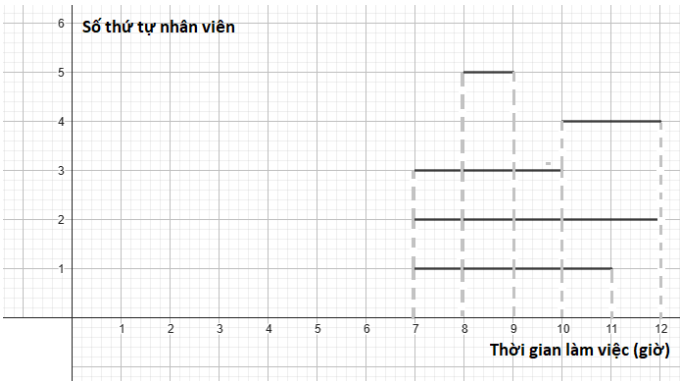
② THUẬT TOÁN ĐƯỜNG QUÉT

Giới thiệu: Thuật toán đường quét biểu diễn bài toán trên mặt phẳng tọa độ, với các điểm trên mặt phẳng đó tượng trưng cho các phần tử trong bài toán. Kết quả được tìm ra bằng cách duyệt các điểm theo chiều tăng hoặc giảm dần.

📌 Bài toán: Nhân viên

- Trong một công ty n có n nhân viên nhưng mỗi nhân viên có thời gian làm việc khác nhau (nhân viên thứ i làm việc từ S[i] đến E[i]). Tìm số nhân viên tối đa làm việc cùng lúc.

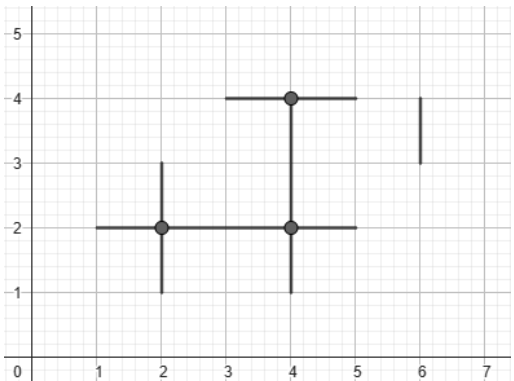
INPUT	OUTPUT	Giải thích test
-------	--------	-----------------

n S[i] E[i]	Như yêu cầu.	
5 8 9 10 12 7 10 7 12 7 11	4	Bài toán có thể được minh hoạ bằng biểu đồ như sau:  Có thể thấy trong thời gian 8 đến 9 giờ có 4 nhân viên cùng làm việc. Đây cũng là số lượng nhân viên làm việc cùng lúc nhiều nhất theo input.

- Ý tưởng:**
 - Tạo mảng emp gồm các pair, trong đó emp[i].first và emp[i].second lần lượt là thời điểm bắt đầu làm việc và kết thúc của nhân viên thứ i. Có mảng 1 chiều time dùng để lưu trữ các thời điểm.
 - Có biến kết quả ans = 0. Bắt đầu duyệt theo từng thời điểm:
 - Tại thời điểm time[i], kiểm tra xem có bao nhiêu emp[j] thoả $\text{emp}[j].\text{first} \leq \text{time}[i] \leq \text{emp}[j].\text{second}$. Gọi số lượng emp[j] thoả điều kiện là cnt.
 - ans = max(cnt, ans).

Bài toán: Số giao điểm

- Cho n đoạn thẳng song song với trục tung (nằm dọc) hoặc trục hoành (nằm ngang), đếm số lượng giao điểm tạo bởi những đoạn thẳng đó.
- INPUT:**
 - Dòng đầu: Chứa n là số lượng đoạn thẳng.
 - n dòng sau: Mỗi dòng chứa 4 số nguyên x_1, y_1 và x_2, y_2 , lần lượt là toạ độ điểm bắt đầu và kết thúc của đoạn thẳng.

INPUT	OUTPUT	Giải thích test
5 1 2 5 2 3 4 5 4 2 3 2 1 4 4 4 1 6 3 6 4	3	Bài toán có thể được minh hoạ bằng biểu đồ như sau: 

- Ý tưởng:**
 - Gọi V là mảng lưu các đoạn thẳng ngang, với V[i].first và V[i].second lần lượt là hoành độ của đoạn thẳng nằm ngang thứ i. Gọi H là mảng để lưu hoành độ của các đoạn nằm dọc.
 - Duyệt V: Ở mỗi V[i], kiểm tra xem có bao nhiêu H[j] thoả $V[i].\text{first} \leq H[j] \leq V[i].\text{second}$. Cộng dồn vào biến kết quả.

CHƯƠNG 5 : LÝ THUYẾT ĐỒ THỊ

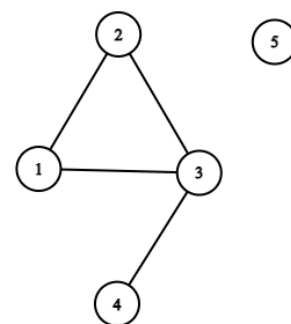
§1 : ĐẠI CƯƠNG

- **Graph:** Đồ thị.
- **(Un)directed graph:** Đồ thị (không) có hướng.
- **Edge, arc:** Cạnh, cung.
- **Node, point, vertex:** Đỉnh.

1 KHÁI NIỆM

Khái niệm:

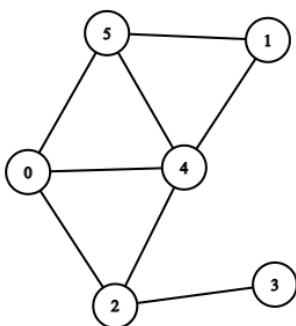
- Đồ thị (G) là tập của các đỉnh (V) và các cạnh/cung (E) .
- Ta thường gọi n (hoặc v) là số đỉnh và m (hoặc e) là số cạnh; u và v là kí hiệu 2 đỉnh.
- Một cạnh là tập hợp 2 đỉnh u và v (vì giữa 2 đỉnh thì có thể tạo ra 1 cạnh).
- Trong hình bên biểu diễn đồ thị G với:
 - Tập đỉnh $V = \{1; 2; 3; 4; 5\}$.
 - Tập cạnh $E = \{\{1, 2\}; \{1, 3\}; \{2, 3\}; \{3, 4\}\}$.
- Nếu tồn tại cạnh $\{u, v\}$, thì từ u có thể đi đến v , hay u kề v .



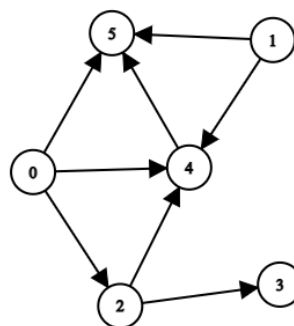
Phân loại:

- Đồ thị vô hướng: Có u kề v thì v cũng kề u .
- Đồ thị có hướng: Có u kề v nhưng chưa chắc v kề u (mỗi cạnh là $\overrightarrow{uv}$).

Lưu ý: Trong đồ thị có hướng, nếu u kề v và v cũng kề u thì $(u; v)$ và $(v; u)$ là 2 cạnh khác nhau.



Đồ thị vô hướng.



Đồ thị có hướng.

Bậc:

- **Bậc của đỉnh u :** Là số lượng cạnh nối với đỉnh u .
- **Bán bậc vào của đỉnh u ($\deg^-(u)$):** Là số lượng cạnh có hướng chĩa vào đỉnh u .
 - VD: Đỉnh 4 (đồ thị có hướng như trên) có $\deg^-(4) = 3$.
- **Bán bậc ra của đỉnh u ($\deg^+(u)$):** Là số lượng cạnh có hướng chĩa ra từ đỉnh u .
 - VD: Đỉnh 4 (đồ thị có hướng như trên) có $\deg^+(4) = 1$.

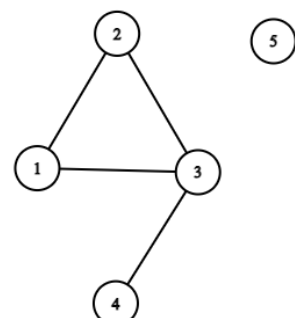
Độ dày của đồ thị:

- Một đồ thị có n đỉnh thì có tối đa $n(n - 1)$ cạnh.
- Một đồ thị mà số cạnh của nó ngang ngửa với số cạnh tối đa thì đồ thị đó là đồ thị dày.
- Ngược lại, đó là một đồ thị thưa.

2 BIỂU DIỄN ĐỒ THỊ

a. Danh sách cạnh

Danh sách cạnh thường được dùng trong INPUT.

INPUT mẫu	
Dòng đầu cho hai số nguyên N, M tương ứng với số đỉnh và số cạnh. M dòng tiếp theo, mỗi dòng chứa hai số u, v thể hiện đường đi hai chiều trực tiếp từ đỉnh u đến đỉnh v (đồ thị vô hướng).	
5 4 1 2 2 3 1 3 3 4 (INPUT mẫu minh họa đồ thị bên)	

b. Ma trận kề

Ma trận kề là cách biểu diễn đồ thị bằng ma trận hay mảng 2 chiều a . Tạm quy ước, nếu từ u đến v có cạnh nối thì $A[u][v] = 1$, nếu không thì $A[u][v] = 0$.

- **Ưu điểm:** Dễ dàng kiểm tra có tồn tại cạnh $\{u, v\}$ hay không bằng cách kiểm tra $A[u][v]$.
- **Nhược điểm:**
 - Tốn nhiều thời gian duyệt (phải duyệt hết n thì mới biết đỉnh u có cạnh nối đến đỉnh nào).
 - Tốn bộ nhớ (cần $n \times n$ phần tử để biểu diễn các cạnh, nếu là đồ thị vô hướng thì lãng phí một nửa ma trận).

$u \backslash v$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	0

Ma trận kề minh họa INPUT mẫu.

c. Danh sách kề

Danh sách kề là danh sách a mà $a[i]$ chứa toàn bộ các đỉnh v có cạnh nối với $A[i]$. Danh sách kề tối ưu so với hai cách trên nên đây thường được dùng để biểu diễn đồ thị, trong bài viết này, nếu không nói đến thì mặc định đang dùng danh sách kề.

Đỉnh u	Các đỉnh kề với u
1	2; 3
2	1; 3
3	2; 1; 4
4	3
5	

Danh sách kề minh họa INPUT mẫu.

Có thể sử dụng một mảng các vector để sử dụng danh sách kề. $A[u]$ là vector chứa các đỉnh có cạnh nối đến u .

1. `int n, m;`

```

2. vector<int> A[maxn]; //maxn là giá trị tối đa của n
3.
4. void readInput(){
5.     cin >> n >> m;
6.     for(int i = 0; i < n; ++i){
7.         int u, v;
8.         cin >> u >> v;
9.         A[u].push_back(v);
10.        A[v].push_back(u); //Trường hợp đồ thị vô hướng
11.    }
12. }

```

§2 : CÁC THUẬT TOÁN TÌM KIẾM TRÊN ĐỒ THỊ

- *Depth-First search (DFS)*: Tìm kiếm theo chiều sâu.
- *Breadth-First search (BFS)*: Tìm kiếm theo chiều rộng.
- Các thuật toán tìm kiếm trên đồ thị là các thuật toán giúp duyệt đồ thị.
- Có 2 thuật toán tìm kiếm là DFS và BFS tương tự nhau nhưng DFS đệ quy khá nhanh hơn BFS. Cả 2 đều liên quan đến những thuật toán đặc trưng khác trên đồ thị.

① DFS – THUẬT TOÁN TÌM KIẾM THEO CHIỀU SÂU

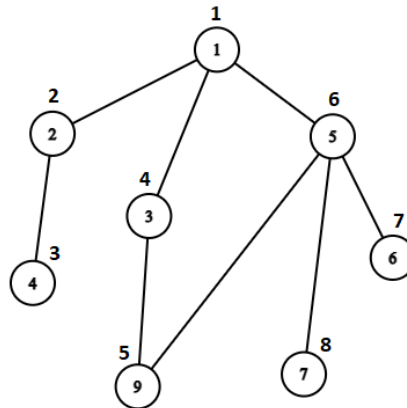
a.DFS có đệ quy

Các bước:

- Đánh dấu u (đỉnh hiện tại).
- Kiểm tra các đỉnh v kề với u. Nếu đỉnh v đó chưa được đi qua, DFS(v).

Hình dưới biểu diễn sự tìm kiếm của DFS. Các số trên mỗi đỉnh là số thứ tự mà đỉnh đó được thăm, với đỉnh xuất phát là 1.

Độ phức tạp: $O(n + m)$.



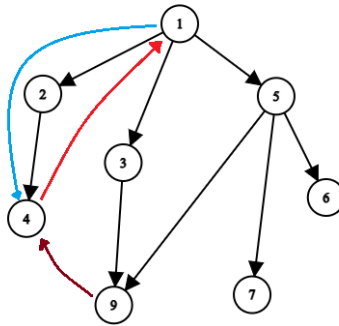
```

1. bool visited[maxn] = {false}; //Đánh dấu tất cả các đỉnh đều chưa được đi
2.
3. void DFS(int u){
4.     visited[u] = true;
5.     for(int v = 0; v < A[u].size(); ++v) //Có thể tối giản bằng for(int v : A[u])
6.         if(!visited[A[u][v]])
7.             DFS(A[u][v]);
8. }

```

Lưu ý: Nếu sử dụng for-each thì chỉ cần ghi là v thay vì A[u][v].

b. Cây DFS



- Thứ tự thăm các đỉnh trong đồ thị bằng DFS đã vô hình định hướng cho đồ thị đó và tạo thành một cây, gọi là cây tìm kiếm DFS.
- Cây DFS có:
 - **Đỉnh gốc:** Là đỉnh đầu tiên được thăm.
 - **Cung của cây** (màu đen): Nếu từ u DFS trực tiếp sang v, thì có cung của cây (u; v).
 - **Cung xuôi** (màu xanh dương): Nếu đỉnh v được thăm từ đỉnh u, nhưng từ đỉnh u không DFS trực tiếp sang v thì có cung xuôi (u; v) (u và v cùng nằm trên một nhánh).
 - **Cung ngược** (màu đỏ): Ngược lại với cung xuôi (u; v) là cung ngược (v; u).
 - **Cung chéo** (màu nâu): Có đỉnh u và v được DFS từ 2 nhánh khác nhau thì có cung chéo (u; v) hoặc cung chéo (v; u).

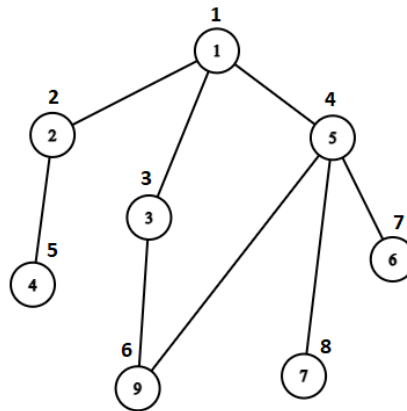
Lưu ý: Trong ảnh chỉ lấy đại diện vài cung chứ không thể hiện toàn bộ.

② BFS – THUẬT TOÁN TÌM KIẾM THEO CHIỀU RỘNG

Các bước:

- Khởi tạo: Đưa đỉnh bắt đầu vào queue.
- Lặp cho khi queue rỗng:
 - Lấy u ra khỏi queue, đánh dấu u.
 - Kiểm tra các đỉnh v kề với u. Nếu đỉnh v đó chưa được đi qua, đưa v vào queue.

Độ phức tạp: $O(n + m)$.



Đỉnh đang xét	Lấy ra	Queue	Đưa vào các đỉnh kề với đỉnh đang xét
		1	← 1.
1		2 3 5	← 3; 5.
2		3 5 4	← 4.
3		5 4 9	← 9.
5		4 9 6 7	← 6; 7.
4		9 6 7	
9		6 7	
6		7	
7			

Mô phỏng quá trình BFS (đỉnh bắt đầu là 1) (như hình trên).

```

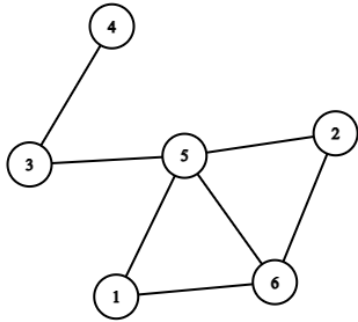
1. bool visited[maxn] = {false};
2. queue<int> qu;
3.
4. void BFS(int start){
5.     qu.push(start);
6.     while(!qu.empty()){
7.         int u = qu.front();
8.         qu.pop();
9.         visited[u] = true;
10.        for(int v = 0; v < A[u].size(); ++v)
11.            if(!visited[A[u][v]])
12.                qu.push(A[u][v]);
13.    }
14. }
```

§3 : CHU TRÌNH

- *Cycle*: Chu trình.
- *Graph coloring method*: Phương pháp tô màu đồ thị.

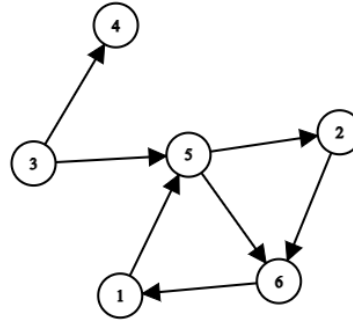
1 CHU TRÌNH

Chu trình là một đường đi có đỉnh đầu trùng với đỉnh cuối (đường đi khép kín). Mỗi đường đi là tập hợp các cạnh:



Danh sách các chu trình:

- Chu trình 1: $1 \rightarrow 5 \rightarrow 6 \rightarrow 1$.
- Chu trình 2: $5 \rightarrow 2 \rightarrow 6 \rightarrow 5$.
- Chu trình 3: $1 \rightarrow 5 \rightarrow 2 \rightarrow 6 \rightarrow 1$.



Danh sách các chu trình:

- Chu trình 1: $1 \rightarrow 5 \rightarrow 6 \rightarrow 1$.
- Chu trình 2: $1 \rightarrow 5 \rightarrow 2 \rightarrow 6 \rightarrow 1$.

2 KIỂM TRA CHU TRÌNH

a. Trên đồ thị vô hướng

Ý tưởng:

- Có $\text{parent}[u]$ là đỉnh cha của u (từ parent DFS/BFS trực tiếp sang u). Từ u , qua DFS/BFS tìm thấy v kề u , đã được thăm và không phải $\text{parent}[u] \Rightarrow$ Tồn tại cung ngược $(u; v) \Rightarrow$ Tồn tại chu trình: $(...) \leftarrow v \leftarrow u \leftarrow \text{parent}[u] \leftarrow (...)$.
- Thuật toán có thể được triển khai bằng DFS hoặc BFS.

Lưu ý:

- Thuật toán chỉ kiểm tra chu trình trên vùng liên thông chứ không kiểm tra trên toàn bộ đồ thị.
- Muốn kiểm tra trên toàn bộ đồ thị, ta kiểm tra trên tất cả vùng liên thông.

Độ phức tạp: $O(n + m)$.

```
1. int parent [maxn];
2. bool visited[maxn];
3.
4. //Hàm trả về true nếu trong vùng liên thông có chu trình
5. bool DFS(int u){
6.     visited[u] = true;
7.     for(int v = 0; v < A[u].size(); ++v)
8.         if(!visited[A[u][v]]){
9.             parent[A[u][v]] = u;
10.            if(DFS(A[u][v]))
11.                return true;
12.        } else if(A[u][v] != parent[u])
13.            return true;
14.     return false;
15. }
```

b. Trên đồ thị có hướng (phương pháp tô màu đồ thị)

Ý tưởng:

- Mỗi một đỉnh u trên cây DFS có 3 trạng thái (màu sắc) sau:
 - $\text{color}[u] = 0$: Đỉnh u chưa được thăm.
 - $\text{color}[u] = 2$: Đỉnh u đã được thăm (từ đỉnh u không thể thăm thêm đỉnh nào nữa).

- $color[u] = 1$: Đỉnh u đang được thăm dang dở (từ đỉnh u có thể thăm thêm các đỉnh khác).
- Có đỉnh u đang được thăm ($color[u] = 1$). Nếu có đỉnh v kề với đỉnh u mà v cũng đang được thăm ($color[v] = 1$) thì tồn tại cung ngược ($v; u$) $\Rightarrow$ Tồn tại chu trình $(...) \leftarrow v \leftarrow u \leftarrow (...)$. Kết thúc thuật toán.

Lưu ý:

- Thuật toán chỉ kiểm tra chu trình trên vùng liên thông chứ không kiểm tra trên toàn bộ đồ thị.
- Muốn kiểm tra trên toàn bộ đồ thị, ta kiểm tra trên tất cả vùng liên thông.

Độ phức tạp: $O(n + m)$.

```

1. //Thuật toán trả về true nếu vùng liên thông có chu trình
2. int color[maxn];
3.
4. bool DFS(int u){
5.     color[u] = 1;
6.     for(int v = 0; v < A[u].size(); ++v)
7.         if(color[A[u][v]] == 0){
8.             if(DFS(A[u][v]))
9.                 return true;
10.        } else if(color[A[u][v]] == 1)
11.            return true;
12.     color[u] = 2;
13.     return false;
14. }

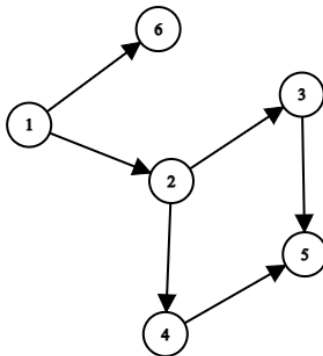
```

§4 : THỨ TỰ TOPO, SẮP XẾP TOPO

- *Topological ordering: Thứ tự topo.*

① THỨ TỰ TOPO

- Thứ tự topo là thứ tự trong danh sách các đỉnh thuộc đồ thị có hướng và không có chu trình, sao cho nếu u kề v thì u phải được xếp trước v . Nếu u kề với nhiều v , không phân biệt thứ tự các v .
- **Lưu ý:**
 - Một đồ thị có hướng có thể có nhiều thứ tự topo.
 - Chỉ đồ thị có hướng và không có chu trình thì mới có thứ tự topo.
- Đồ thị bên dưới có thể có thứ tự topo là 1; 2; 3; 4; 5; 6 hoặc 1; 6; 2; 4; 3; 5. VD có đỉnh 1 kề với đỉnh 2, nên 1 được xếp trước 2.



② THUẬT TOÁN KAHN

Ý tưởng:

- Kiểm tra và đếm số lượng bán bậc vào của các đỉnh.
- Đưa các đỉnh u có bán bậc vào $deg^-(u) = 0$ vào danh sách topo.
- Lần lượt xóa các đỉnh đó khỏi đồ thị (giảm bán bậc vào của các đỉnh kề với u đi 1) và thực hiện lại bước đầu.

Lưu ý: Thuật toán cho ra danh sách rỗng nếu đồ thị không có thứ tự topo.

Độ phức tạp: $O(n + m)$.

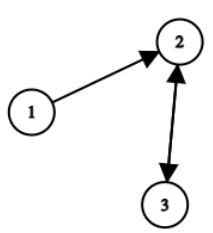
```

1. int n, m;
2. vector <int> A[maxn];
3. int in [maxn]; //Mảng lưu bán bậc vào của các đỉnh
4.
5. void readInput(){
6.     cin >> n >> m;
7.     for(int i = 1; i <= n; ++i){
8.         int u, v;
9.         cin >> u >> v;
10.        A[u].push_back(v);
11.        ++ in[v]; //Có cạnh u -> v suy ra: tăng deg-(v) lên 1
12.    }
13. }
14.
15. vector <int> topo;
16.
17. void kahn(){
18.     //Tìm đỉnh có deg-() = 0
19.     queue <int> q; //Danh sách các đỉnh chuẩn bị bị xoá khỏi đồ thị
20.     for(int u = 1; u <= n; ++u)
21.         if(in[u] == 0)
22.             q.push(u);
23.
24.     while(!q.empty()){
25.         //Coi như đỉnh đó đã bị xoá khỏi đồ thị, đưa vào danh sách topo
26.         int u = q.front(); q.pop();
27.         topo.push_back(u);
28.         //Duyệt các đỉnh kề với u
29.         for(int v = 0; v < A[u].size(); ++v){
30.             -- in[A[u][v]]; //Giảm deg-() của đỉnh đó
31.             if(in[A[u][v]] == 0) //Kiểm tra và đưa vào ds đỉnh chuẩn bị bị xoá
32.                 q.push(A[u][v]);
33.         }
34.     }
35. }

```

Bài toán: Đường đi dài nhất

- <https://lqdoj.edu.vn/problem/longestpath>
- Cho đồ thị vô hướng có n đỉnh và m cạnh (đồ thị không có chu trình có hướng). Tìm độ dài lớn nhất của đường đi có hướng (số cạnh của đồ thị).

INPUT	OUTPUT	Giải thích test
n m u v	Như yêu cầu.	
3 2 1 2 2 3	2	Đường đi có hướng dài nhất là $1 \rightarrow 2 \rightarrow 3$. 

- **Ý tưởng:**
 - Gọi $F[topo[i]]$ là đường đi dài nhất bắt đầu từ đỉnh $topo[0]$ (đỉnh đầu tiên) đến đỉnh $topo[i]$. Ban đầu, $F[i] = 0$.

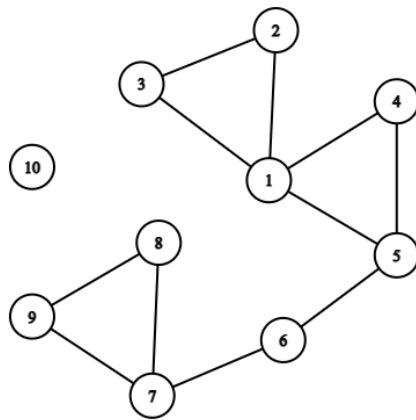
- Để đến được topo[i], cần xuất phát từ topo[j] với $j < i$ và topo[j] kề với topo[i]. Vậy, $F[\text{topo}[i]] = \max(F[\text{topo}[i]], F[\text{topo}[j]] + 1)$.
- Nếu sử dụng danh sách kề, ta chỉ cần duyệt qua tất cả đỉnh v kề với F[topo[i]] và áp dụng công thức như trên (với v tương ứng cho F[topo[j]]).
- Như vậy, cần sort topo trước khi thực hiện quy hoạch động nhằm tạo ra thứ tự lớn dần cho các đỉnh, đảm bảo khi ta cần tính toán đỉnh u thì các đỉnh v ($v < u$) đã được tính toán.

§5 : VÙNG LIÊN THÔNG

- **(Connected) component:** Vùng liên thông.
- **Strongly connected component (SCC):** Vùng liên thông mạnh.
- **Cut point:** Đỉnh khớp, đỉnh trụ.
- **Bridge, cut vertex:** Cạnh cầu.

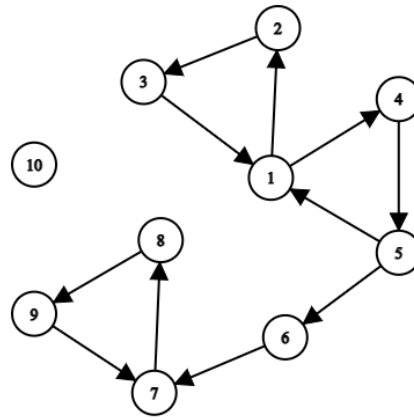
1 ĐỊNH NGHĨA

- Vùng liên thông / thành phần liên thông là tập hợp các đỉnh, mà từ một đỉnh bất kì thuộc vùng liên thông có thể đi đến tất cả các đỉnh còn lại của vùng liên thông đó. Nếu u và v cùng thuộc một vùng liên thông, ta nói u liên thông với v.
- **Trong đồ thị vô hướng:** Vùng liên thông có ít nhất 1 đỉnh.
- **Trong đồ thị có hướng:** Được chia ra thành vùng liên thông mạnh và vùng liên thông yếu:
 - **Vùng liên thông mạnh:** Có ít nhất 1 đỉnh.
 - **Vùng liên thông yếu:** Loại bỏ chiều của đồ thị có hướng để có đồ thị vô hướng. Vùng liên thông trong đồ thị vô hướng này là vùng liên thông yếu của đồ thị có hướng tương ứng (Điều kiện: Vùng liên thông yếu không trùng với vùng liên thông mạnh).
- Đồ thị vô hướng có 1 vùng liên thông được gọi là **đồ thị liên thông**.
- Đồ thị có hướng có 1 vùng liên thông mạnh được gọi là **đồ thị liên thông mạnh**, ngược lại là **đồ thị liên thông yếu**.
- VD:



Có 2 vùng liên thông là:

- {1; 2; 3; 4; 5; 6; 7; 8; 9}.
- {10}.



Vùng liên thông mạnh:

- {1; 2; 3; 4; 5}.
- {6}.
- {7; 8; 9}.
- {10}.

② TÌM VÙNG LIÊN THÔNG TRONG ĐỒ THỊ VÔ HƯỚNG BẰNG THUẬT TOÁN BFS/DFS

Các bước:

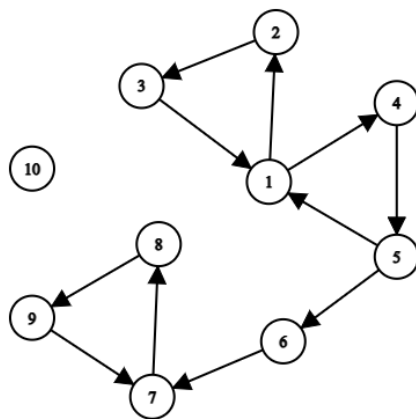
- Duyệt các đỉnh u . Nếu $visited[u] = false$, DFS/BFS(u).
- Kết thúc DFS/BFS, các đỉnh đang được đánh dấu hiện tại nằm trong một vùng liên thông.
- Có nhiều cách để nhận biết rằng vùng liên thông đó gồm có các đỉnh nào, chẳng hạn như tạo một vector lưu trữ các đỉnh trong mỗi vùng liên thông. Ở mỗi bước DFS/BFS, đưa v vào vector.

```
1. //Hàm trả về số lượng vùng liên thông bằng DFS
2. int ccCnt(){
3.     int ans = 0;
4.     for(int u = 1; u <= n; ++u)
5.         if(!visited[u]){
6.             DFS(u);
7.             ++ ans;
8.         }
9.     return ans;
10. }
```

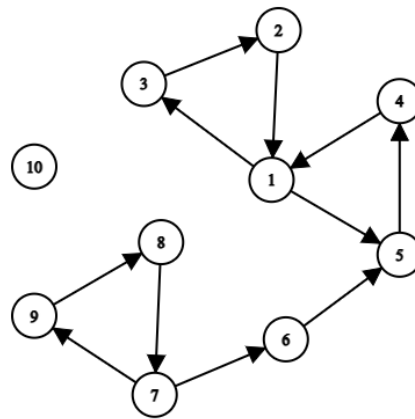
③ TÌM VÙNG LIÊN THÔNG TRONG ĐỒ THỊ CÓ HƯỚNG

a. Thuật toán Kosaraju

Tính chất của vùng liên thông mạnh: Nếu đảo chiều các cạnh của 1 vùng liên thông mạnh thì đó vẫn là vùng liên thông mạnh.



Đồ thị A.



Đồ thị rA là đồ thị đảo ngược của A.

Sau khi đảo chiều đồ thị thì các vùng liên thông mạnh vẫn không đổi.

Ý tưởng:

1. Lưu thứ tự duyệt DFS các đỉnh vào stack (DFS từ đỉnh đầu tiên, nếu từ đỉnh u không thể thêm bất kì đỉnh nào nữa thì đưa u vào stack).
2. Tạo ra đồ thị mới bằng cách đảo ngược các cạnh trong đồ thị ban đầu (thực hiện khi đọc input).
3. Lần lượt đưa các đỉnh u ra khỏi stack và DFS(u) trên đồ thị mới. Với mỗi đỉnh u có được danh sách các đỉnh được DFS từ u , đó là các đỉnh cùng nằm trong một vùng liên thông.

Độ phức tạp: $O(2(n + m))$.

```
1. #include <bits/stdc++.h>
2. #define maxn 100000
3. using namespace std;
4.
5. int n, m;
6. vector<int> A[maxn];
7. vector<int> rA[maxn]; //rA là đồ thị đảo ngược của A
8.
9. void readInput(){
```

```

10.     cin >> n >> m;
11.     for(int i = 1; i <= m; ++i){
12.         int u, v;
13.         cin >> u >> v;
14.         A[u].push_back(v);
15.         rA[v].push_back(u); //Thực hiện bước 1 ngay khi đọc input
16.     }
17. }
18.
19. bool visited[maxn];
20.
21. stack <int> stk; //stack chứa danh sách đỉnh
22.
23. //Bước 1:
24. void DFS1(int u){
25.     visited[u] = true;
26.     for(int v = 0; v < A[u].size(); ++v)
27.         if(!visited[A[u][v]])
28.             DFS1(A[u][v]);
29.     stk.push(u); //Nếu từ đỉnh u không thể thêm bất kì đỉnh nào nữa thì đưa u vào stack
30. }
31.
32. //Bước 3
33. void DFS2(int u){
34.     visited[u] = true;
35.     for(int v = 0; v < rA[u].size(); ++v)
36.         if(!visited[rA[u][v]])
37.             DFS2(rA[u][v]);
38.     cout << u << ' '; //Đặt cout ở cuối để xuất ra ưu tiên theo thứ tự đỉnh theo chiều của đồ thị gốc
39. }
40.
41. //Thuật toán xuất ra các vùng liên thông mạnh trong đồ thị
42. void kosaraju(){
43.     fill(&visited[1], &visited[n+1], false);
44.     for(int u = 1; u <= n; ++u)
45.         if(!visited[u])
46.             DFS1(u);
47.
48.     fill(&visited[1], &visited[n+1], false);
49.     while(!stk.empty()){
50.         int u = stk.top();
51.         stk.pop();
52.         if(!visited[u]){
53.             DFS2(u);
54.             cout << '\n';
55.         }
56.     }
57. }

```

b. Thuật toán Tarjan

Ý tưởng:

- Gọi num[u] là thứ tự mà đỉnh u được thăm trong cây DFS gốc start. Vậy, num[start] = 0 do start được DFS thăm trước tiên. Muốn biết num[u], ta tạo một biến độc lập với hàm DFS tên cnt, cho cnt lần lượt tăng một đơn vị mỗi khi ghé thăm một đỉnh mới, và ta gán num[u] = cnt.
- Gọi low[u] là đỉnh v sao cho từ u có thể đến được v và num[v] thấp nhất có thể, gọi đỉnh v này là đỉnh thấp nhất (num[start] = 0 do đỉnh thấp nhất có thể đến được start dĩ nhiên là start). Muốn biết low[u], ta làm như sau:
 - Ban đầu, giả sử đỉnh thấp nhất của u là u, nên low[u] = num[u].
 - Duyệt các đỉnh v kề với u:
 - Nếu đỉnh v đã từng đi qua (cung ngược (u, v)) ⇒ Có thể v là đỉnh thấp nhất ⇒ **low[u] = min(low[u], num[v])**.

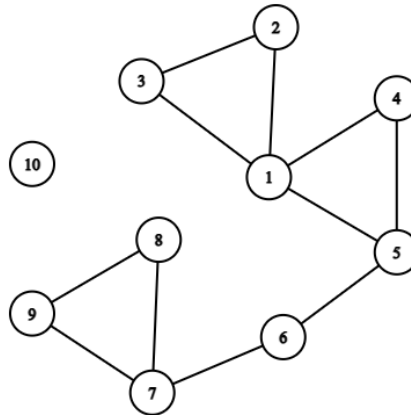
- Nếu đỉnh v chưa được đi qua (cung xuôi (u, v)), gọi DFS đỉnh v để xác định $\text{low}[v] \Rightarrow$ Có thể v là đỉnh thấp nhất $\Rightarrow \text{low}[u] = \min(\text{low}[u], \text{low}[v])$.
- Lưu ý: Để bảo toàn chiều của cung xuôi (u, v) sao cho từ v không duyệt ngược về u để làm hỏng công thức tính low , ta có thể:
 - Thêm tham số p cho hàm DFS ($\text{DFS}(u, p)$), với p tương ứng với u trong cung (u, v) và thêm điều kiện trong khi duyệt là $v \neq p$.
 - Nhận thấy rằng nếu có cung xuôi (u, v) thì $\text{num}[v] - 0 = \text{num}[u]$, cho nên ta có thể thêm điều kiện trong khi duyệt là $\text{num}[v] - 0 \neq \text{num}[u]$.

Độ phức tạp: $O(n + m)$.

4 ĐỈNH KHỚP, CẠNH CẦU

a. Khái niệm

- Đỉnh khớp / đỉnh trụ là một đỉnh mà khi xoá đỉnh đó ra khỏi đồ thị (cũng như xoá các cạnh nối với đỉnh đó) thì số vùng liên thông của đồ thị tăng lên.
- Cạnh cầu là một cạnh mà khi xoá cạnh đó ra khỏi đồ thị thì số vùng liên thông của đồ thị tăng lên.



Trong đồ thị trên, các đỉnh khớp là 1, 5, 6, 7; các cạnh cầu là (5; 6) và (6; 7).

- Lưu ý: Trong đồ thị có thể không có khớp hoặc cầu.

b. Tìm khớp và cầu

Phương pháp đơn giản nhất:

- Thử xoá một đỉnh / một cạnh của đồ thị rồi đếm xem số vùng liên thông của đồ thị đó có tăng lên không. Nếu có, thì đỉnh / cạnh vừa xoá là khớp / cầu.
- **Độ phức tạp:** $O(n(n + m))$ hoặc $O(m(n + m))$.

```

1. //Thuật toán xuất ra danh sách khớp
2. void cutNode(){
3.     memset(visited, false, sizeof(visited));
4.     int cc = ccCnt();
5.
6.     for(int i = 1; i <= n; ++i){
7.         memset(visited, false, sizeof(visited));
8.         visited[i] = true; //Coi như đã xoá đỉnh này khỏi đồ thị
9.         if(ccCnt() > cc)
10.             cout << i << ' ';
11.     }
12. }

```

```

1. int n, m;
2. vector <int> A[maxn];
3. vector <pair <int, int>> E; //vector lưu trữ cạnh

```



```

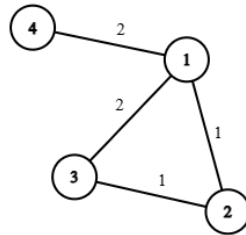
4.
5. void readInput(){
6.     cin >> n >> m;
7.     for(int i = 0; i < m; ++i){
8.         int u, v;
9.         cin >> u >> v;
10.        A[u].push_back(v);
11.        A[v].push_back(u);
12.        pair <int, int> e;
13.        e.first = u; e.second = v;
14.        E.push_back(e);
15.    }
16. }
17.
18. bool visited[maxn];
19.
20. void DFS(int u){
21.     visited[u] = true;
22.     for(int v = 0; v < A[u].size(); ++v)
23.         if(!visited[A[u][v]])
24.             DFS(A[u][v]);
25. }
26.
27. int ccCnt(){
28.     int ans = 0;
29.     for(int u = 1; u <= n; ++u)
30.         if(!visited[u]){
31.             DFS(u);
32.             ++ ans;
33.         }
34.     return ans;
35. }
36.
37. //cạnh (a, b) là cạnh bị xoá
38. void DFS2(int u, int a, int b){
39.     visited[u] = true;
40.     for(int v = 0; v < A[u].size(); ++v)
41.         if((u == a && A[u][v] == b) || (u == a && A[u][v] == b)) //ĐK
42.             continue;
43.         else if(!visited[A[u][v]])
44.             DFS2(A[u][v]);
45. }
46.
47. //Thuật toán xuất ra danh sách cầu
48. void cutVertex(){
49.     memset(visited, false, sizeof(visited));
50.     int cc = ccCnt();
51.
52.     for(int i = 0; i < m; ++i){
53.         memset(visited, false, sizeof(visited));
54.         int cnt = 0;
55.         for(int u = 1; u <= n; ++u)
56.             if(!visited[u]){
57.                 DFS2(u, E[i].first, E[i].second);
58.                 ++ cnt;
59.             }
60.         if(cnt > cc)
61.             cout << E[i].first << ' ' << E[i].second << '\n';
62.     }
63. }

```

Ứng dụng thuật toán Tarjan:

§6 : ĐƯỜNG ĐI NGẮN NHẤT

- **Weighted graph:** Đồ thị có trọng số.
- **Bài toán:** Xuất phát từ đỉnh s có thể có nhiều con đường đến đỉnh e , tìm đường đi ngắn nhất.
- **Trọng số:**
 - Trọng số của cạnh $(u; v)$ có thể hiểu là chi phí để đi từ đỉnh u đến đỉnh v (hoặc từ đỉnh v đến đỉnh u trong đồ thị vô hướng), gọi là $W[u][v]$ (kí hiệu theo mảng 2 chiều).
 - Xây dựng $W[u][v]$ có thể dùng mảng 2 chiều, nhưng sẽ tốn không gian (nhược điểm của ma trận kề). Ta có thể tối ưu bằng cách xây dựng theo ma trận kề như sau:
 - Gọi `vector<pair<int, int>> W[n]`.
 - Giả sử trọng số từ u đến v là w : `W[u].push_back({v, w})`.
 - Lúc bấy giờ, với đỉnh u , có `W[u].first = v`, và `W[u].second = w`.



VD đồ thị có trọng số (VD trọng số của cạnh $(4; 1)$ là 2m của cạnh $(1; 2)$ là 1).

- **Kĩ thuật truy vết:** Tìm những đỉnh nằm trên đường đi giữa s và e :
 - Xây dựng trace:
 - Gọi `trace[v]` là đỉnh được thăm liền trước đỉnh v .
 - Xét các đỉnh v kề với đỉnh u , `trace[u] = v`.
 - Xuất trace:
 1. Xuất đỉnh e .
 2. $e = \text{trace}[e]$.
 3. Lặp lại từ bước 1 cho đến khi $e = s$.

① THUẬT TOÁN LOANG (BFS)

a. Trên đồ thị không có trọng số

Áp dụng thuật toán BFS như thường.

b. Trên ma trận

Xuất phát từ ô $s(x_s, y_s)$.

Đếm số bước tối thiểu để di chuyển từ tọa độ ô $s(x_s, y_s)$ đến ô bất kỳ trong đồ thị:

- Gọi mảng 2 chiều d , với $d[i][j]$ là số bước ít nhất để từ ô (i, j) đến ô $s(x_s, y_s)$.
- Xét các ô (i', j') kề cạnh với ô (i, j) , $d[i'][j'] = d[i][j] + 1$.

② THUẬT TOÁN DIJKSTRA

Thuật toán Dijkstra cho biết đường đi ngắn nhất từ đỉnh xuất phát s cho đến mọi đỉnh trong đồ thị có trọng số không âm. Thuật toán Dijkstra có bản chất là quy hoạch động trên đồ thị.

Các bước:

- **Khởi tạo:**
 - Gọi $D[u]$ là độ dài đường đi ngắn nhất từ s đến v.
 - Ban đầu:
 1. $D[u] = +\infty$ ($+\infty$ là giá trị rất lớn mà không có trọng số nào trong đồ thị có thể đạt đến) (vì chưa biết từ s đến u phải đi như thế nào).
 2. Một đỉnh u được đánh dấu khi đã tìm ra đường đi nhỏ nhất từ s đến u, hay $D[u]$ nhỏ nhất. Cho nên, ban đầu không có đỉnh nào được đánh dấu.
 3. $D[s] = 0$ (Từ đỉnh s đến chính đỉnh s thì không cần đi qua cạnh nào, do đó $D[s] = 0$).
- **Lặp:**
 - **Chọn đỉnh để đi tiếp:**
 - Chọn đỉnh u chưa bị đánh dấu có $D[u]$ nhỏ nhất.
 - Đỉnh u này đã có $D[u]$ nhỏ nhất nên theo điều (2), ta đánh dấu đỉnh u.
 - **Cực tiểu hóa đường đi đến các đỉnh chưa cực tiểu (đỉnh v):** Xét các đỉnh v kề u. Có hai trường hợp:
 - Có thể đến đỉnh v mà không cần đi qua đỉnh u $\Rightarrow D[v]$ không đổi: $D[v] = D[v]$.
 - Đến đỉnh v mà cần đi qua đỉnh u $\Rightarrow D[v]$ có đổi: $D[v] = D[u] + W[u][v]$.Do cần tìm đường đi ngắn nhất, ta lấy min 2 trường hợp trên, hay $D[v] = \min(D[v], D[u] + W[u][v])$.
 - **Ngừng lặp:** Lặp đi lặp lại hai bước trên cho đến khi đạt một trong hai điều sau:
 - Tìm được đường đi ngắn nhất từ s đến đỉnh kết thúc e.
 - Tất cả các đỉnh đều đã được đánh dấu.

Độ phức tạp: $O(n^2)$.

INPUT	OUTPUT
n m s e u[i] v[i] w[i] (đồ thị có hướng) (với w[i] là trọng số của cạnh (u[i], v[i]))	Độ dài của đường đi ngắn nhất từ s đến e.

```
1. #include <bits/stdc++.h>
2. #define maxn 100000
3. using namespace std;
4.
5. int n, m;
6. vector <int> A[maxn];
7. vector <pair <int, int>> W[maxn];
8. //first là đỉnh v
9. //second là trọng số từ u đến v
10.
11. void readInput(){
12.     cin >> n >> m;
13.     for(int i = 0; i < n; ++i){
14.         int u, v, w;
15.         cin >> u >> v >> w;
16.         W[u].push_back({v, w});
17.     }
18. }
19.
20. #define infVal INT_MAX
21. bool visited[maxn] = {false};
22.
23. int dijkstra(int s, int e){
24.     int D[n+1];
25.     for(int i = 1; i <= n; ++i)
```

```

26.     D[i] = infVal;
27.     D[s] = 0;
28.
29.     while(true){
30.         //Chọn đỉnh u
31.         int u;
32.         int Du = infVal;
33.         for(int v = 1; v <= n; ++v)
34.             if(!visited[v] && D[v] < Du){
35.                 u = v;
36.                 Du = D[v];
37.             }
38.         visited[u] = true;
39.
40.         //Điều kiện kết thúc dijkstra
41.         if(u == e)
42.             return D[e];
43.
44.         //Cực tiểu hoá các đỉnh kề với u
45.         for(int i = 0; i < W[u].size(); ++i){
46.             int v = W[u][i].first;
47.             int w = W[u][i].second;
48.             if(!visited[v])
49.                 D[v] = min(D[v], D[u] + w);
50.         }
51.     }
52. }

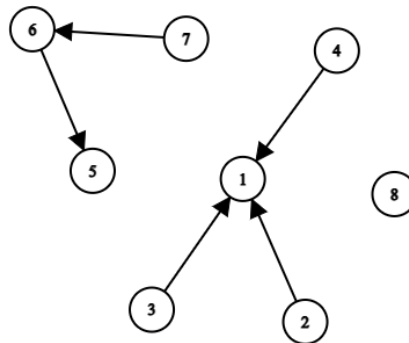
```

§7 : CÂY KHUNG CỰC TIỂU

- *Disjoint set union , Disjoint-set data structure, Union-find (DSU):* Cấu trúc dữ liệu các tập hợp rời nhau.
- *Minimum spanning tree (MST):* Cây khung cực tiểu, cây bao trùm cực tiểu.

❶ CẤU TRÚC DỮ LIỆU CÁC TẬP HỢP RỜI NHAU

- Cấu trúc dữ liệu các tập hợp rời nhau bao gồm các tập hợp, mỗi tập hợp gồm các đỉnh.
- Các tập hợp này rời nhau, tức là không có đỉnh chung (không giao nhau). Nếu giữa hai tập hợp có đỉnh chung, hai tập hợp đó được hợp nhất thành một.
- Ở mỗi tập hợp, chọn là một đỉnh u bất kì làm đỉnh cha. Gọi $\text{parent}[v]$ là đỉnh cha của v, nếu $\text{parent}[v] = u$ thì v và u cùng nằm trong một tập hợp, u là đỉnh cha của v, kí hiệu bằng cung có hướng $v \rightarrow u$.



Hình trên có 3 tập hợp:

- Tập hợp 1: {1; 2; 3; 4} có đỉnh cha là 1.
- Tập hợp 2: {5; 6; 7} có đỉnh cha là 5 ($\text{parent}[7] = 6, \text{parent}[6] = 5 \Rightarrow \text{parent}[7] = 5$).

- Tập hợp 3: {8} có đỉnh cha là 8.

a. Các thao tác

Thao tác tạo tập hợp:

- Có thể thấy ở tập hợp 3 chỉ có 1 đỉnh duy nhất nên đỉnh cha của nó là chính nó. Vậy ban đầu, đỉnh cha của mỗi đỉnh là chính nó.

```
1. void makeSet(){
2.     for(int u = 1; u <= n; ++u)
3.         parent[u] = u;
4. }
```

Thao tác tìm đỉnh cha của u:

- Nếu $\text{parent}[u] = u$, trả về kết quả cha của nó là chính nó.
- Nếu $\text{parent}[u] \neq u$, tìm cha của $\text{parent}[u]$ ($\text{find}(\text{parent}[u])$).

```
1. int find(int u){
2.     if(u == parent[u])
3.         return u;
4.     else
5.         return find(parent[u]);
6. }
```

- VD tập hợp 3: Muốn tìm đỉnh cha của 7, gọi $\text{find}(7) \rightarrow \text{find}(6) \rightarrow \text{find}(5)$. Ở $u = 5$, $\text{parent}[u] = u = 5$ nên lần lượt trả về 5 cho $\text{find}(5)$, $\text{find}(6)$ và $\text{find}(7)$. Vậy, $\text{find}(7) = 5$.

Thao tác gộp 2 đỉnh u, v thành 1 tập hợp:

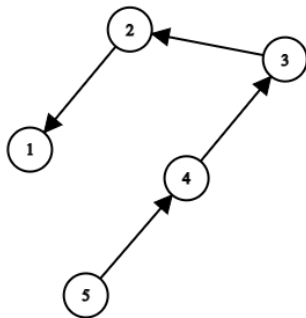
- Nếu $\text{find}(u) = \text{find}(v) \Rightarrow u$ và v có chung cha $\Rightarrow$ không cần gộp.
- Nếu không, gọi $pu = \text{find}(u)$, $p_v = \text{find}(v)$, cho $\text{parent}[pu] = p_v$ (cha của cha của đỉnh u không còn là chính nó nữa mà đã trở thành cha của đỉnh v , vậy u và v đã gộp thành một tập hợp) hoặc $\text{parent}[p_v] = pu$.

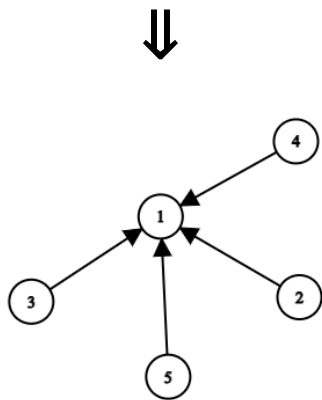
```
1. void merge(int u, int v){
2.     int pu = find(u);
3.     int pv = find(v);
4.     if(pu != pv)
5.         parent[pu] = pv;
6. }
```

b. Tối ưu các thao tác

Phương pháp nén đường đi:

- Xét tập hợp bên, nếu ta muốn tìm cha của 5, tức $\text{find}(5)$, ta phải gọi $\text{find}(5) \rightarrow \text{find}(4) \rightarrow \text{find}(3) \rightarrow \text{find}(2) \rightarrow \text{find}(1)$. Và mỗi lần muốn tìm $\text{find}(5)$, ta đều tốn thời gian để đệ quy đến $\text{find}(1)$.





- Nén đường đi là phương pháp rút ngắn cây lại. Cụ thể, trước khi return ở `find(u)`, ta gán `parent[u] = find(parent[u])` trước.
- Thông thường, khi muốn tìm cha của 5, ta gọi `find(5) → find(4) → find(3) → find(2) → find(1)`. Ở `find(1)`, `parent[1] = 1` nên `find(1) = 1` và bắt đầu return:
 - Return về `find(2)`, gán `parent[2] = 1`,
 - Return về `find(3)`, gán `parent[3] = 1`,
 - Return về `find(4)`, gán `parent[4] = 1`,
 - Return về `find(5)`, gán `parent[5] = 1`, kết thúc thuật toán.
- Như vậy, chỉ mất 1 lần gọi đệ quy từ `find(5)` đến `find(1)`, ta đã thành công rút gọn bớt cây để `parent` của mọi `u` trên quá trình đệ quy đều thành 1 đỉnh cha cuối cùng. Kể từ đó về sau, khi gọi `find` một đỉnh nào đó có cha là 1, ta chỉ tốn 2 lần đệ quy.

```

1. int find(int u){
2.     if(u == parent[u])
3.         return u;
4.     else
5.         return parent[u] = find(parent[u]);
6. }

```

Phương pháp gộp tập hợp nhỏ vào tập hợp lớn:

- Ở thao tác gộp 2 đỉnh `u, v` thành 1 tập hợp, ta đã nói gộp `parent[pu] = pv` hay `parent[pv] = pu` đều được, nhưng điều này có ảnh hưởng đến thời gian thực hiện thực toán.
- Giả sử ta gộp `parent[pu] = pv`, mà `pu` thuộc một tập hợp có nhiều đỉnh hơn tập hợp chứa `pv` ⇒ Ta cần phải cập nhật lại đỉnh cha của tất cả các đỉnh trong tập `pu` (và nó nhiều hơn so với tập hợp chứa `pv`).
- Suy ra, muốn gộp 2 đỉnh `u` và `v` mà muốn tối ưu thời gian, ta so sánh kích thước của tập chứa đỉnh `pu` và `pv`, gọi là `sz[pu]` và `sz[pv]`. Nếu `sz[pu] < sz[pv]` ⇒ ta gộp `pu` vào `pv` (`parent[pu] = pv`) và ngược lại.

```

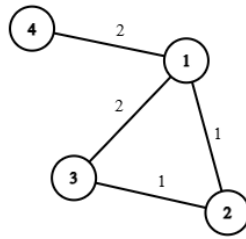
1. int parent[maxn];
2. int sz[maxn]; //Mảng chứa kích thước tập hợp
3.
4. void makeSet(){
5.     for(int u = 1; u <= n; ++u){
6.         parent[u] = u;
7.         sz[u] = 1; //Ban đầu, kích thước mỗi tập hợp = 1
8.     }
9. }
10.
11. int find(int u){
12.     if(u == parent[u])
13.         return u;
14.     else
15.         return parent[u] = find(parent[u]);
16. }
17.
18. void merge(int u, int v){
19.     int pu = find(u);
20.     int pv = find(v);
21.     if(pu != pv){
22.         if(sz[pu] > sz[pv]) //Hoán đổi lại sao cho sz[pu] <= sz[pv]
23.             swap(pu, pv);
24.         parent[pu] = pv;
25.         sz[pv] += sz[pu]; //Cập nhật lại sz[pv]
26.     }
27. }

```

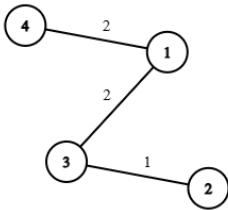
② KHÁI NIỆM CÂY KHUNG CỰC TIỂU

- Trong đồ thị vô hướng và liên thông thì tồn tại cây khung.
- Cây khung là một phần của đồ thị (bao gồm các đỉnh và cạnh của đồ thị đó), có tính chất như sau:
 - Cây khung chứa tất cả các đỉnh của đồ thị.
 - Cây khung chứa $n - 1$ cạnh, với n là số đỉnh của đồ thị ban đầu.
 - Cây khung liên thông.
 - Cây khung không có chu trình.
- Trong đồ thị vô hướng, liên thông và có trọng số, cây khung cực tiểu là cây khung mà tổng trọng số của nó là nhỏ nhất.

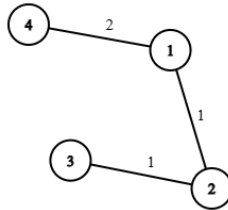
VD: Đồ thị sau có 3 cây khung và 1 cây khung cực tiểu:



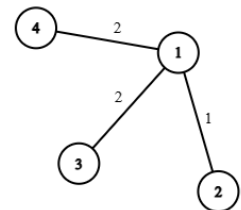
Cây khung thứ 1:



Cây khung thứ 2:



Cây khung thứ 3:



Đây là cây khung cực tiểu vì nó tổng trọng số của các cạnh nhỏ nhất.

③ TÌM CÂY KHUNG CỰC TIỂU

b. Thuật toán Kruskal

Ý tưởng:

1. Khởi tạo:
 - Gọi MST là danh sách chứa các cạnh của cây khung cực tiểu.
 - Gọi d là độ dài trọng số của cây khung MST,
 - Gọi E là danh sách chứa các cạnh của đồ thị (mỗi cạnh gồm 2 đỉnh u, v và trọng số w).
2. Sắp xếp E theo chiều tăng dần của trọng số.
3. Lặp đi lặp lại các bước sau khi $\text{MST.size}() < n - 1$ và E khác rỗng:
 - Lấy ra e là cạnh đầu tiên của E (cũng là cạnh nhỏ nhất vì E đã được sắp xếp theo chiều tăng dần).

- Nếu đưa e vào MST mà không tạo thành chu trình thì đưa e vào MST, tăng độ dài cây khung MST (nếu 2 đỉnh u và v của cạnh e có thể gộp lại thành 1 tập hợp thì đưa e vào MST sẽ không tạo thành chu trình).

Độ phức tạp: $O(m \log m + m \log n)$, với $O(m \log m)$ là độ phức tạp khi sắp xếp các cạnh bằng thuật toán Intro sort.

Nhược điểm: Thuật toán chủ yếu thao tác với các cạnh của đồ thị, nên nếu như đồ thị có quá nhiều cạnh (đồ thị dày) thì thuật toán tỏ ra không hiệu quả.

```

1. //Tạo struct lưu cạnh (u, v) có trọng số w
2. struct data{
3.     int u, v;
4.     int w;
5. };
6.
7. int n, m;
8.
9. int parent[maxn];
10. int sz[maxn];
11.
12. void makeSet(){
13.     for(int u = 1; u <= n; ++u){
14.         parent[u] = u;
15.         sz[u] = 1;
16.     }
17. }
18.
19. int find(int u){
20.     if(u == parent[u])
21.         return u;
22.     else
23.         return parent[u] = find(parent[u]);
24. }
25.
26. //Nếu u và v có thể gộp lại, gộp lại rồi trả về true
27. bool merge(int u, int v){
28.     int pu = find(u);
29.     int pv = find(v);
30.     if(pu == pv)
31.         return false;
32.     if(sz[pu] > sz[pv])
33.         swap(pu, pv);
34.     parent[pu] = pv;
35.     sz[pv] += sz[pu];
36.     return true;
37. }
38.
39. vector <data> E;
40.
41. void readInput(){
42.     cin >> n >> m;
43.     for(int i = 0; i < m; ++i){
44.         int u, v, w;
45.         cin >> u >> v >> w;
46.         data e;
47.         e.u = u; e.v = v; e.w = w;
48.         E.push_back(e);
49.     }
50. }
51.
52. //Hàm cmp sắp xếp E tăng dần
53. bool cmp(data a, data b){
54.     return a.w < b.w;
55. }
56.
57. void kruskal(){
58.     //B1
59.     vector <data> MST;

```



```

60.     int d = 0;
61.     makeSet();
62.
63.     //B2
64.     sort(E.begin(), E.end(), cmp);
65.
66.     //B3
67.     for(int i = 0; i < m && MST.size() < n-1; ++i){
68.         data e = E[i];
69.         if(merge(e.u, e.v)){ //Nếu 2 đỉnh u và v có thể đưa vào MST
70.             MST.push_back(e);
71.             d += e.w;
72.         }
73.     }
74. }

```

Lưu ý dòng điều kiện được highlight: Giả sử ở cuối lần lặp, $MST.size() = n - 1$ (đã thỏa điều kiện và nên thoát ra khỏi vòng lặp) nhưng vòng lặp sẽ vẫn tiếp tục lặp vì $MST.size() \leq n-1$. Vậy điều kiện đúng ở đây là $MST.size() < n-1$.

c.Thuật toán Prim